STARLET — служебные рутины

Справочное руководство

STARLET-RM-0171-RUS

STARLET Version 1.7.1 *** 31-OCT-2026

Copyright © 2026 StarLet Squad and Ruslan R. Laishev.

StarLet - свободная программа, распространяется на условиях Apache License, Version 2.0. Она поставляется БЕЗ КАКИХ-ЛИБО ГАРАНТИЙ.

Содержание

Предисловие **6**
Кому адресовано . . . 6
Структура документа . . . 7
Связанные документы . . . 8
Изменения в версии 1.7.1 . . . 9
Изменения в версии 1.7.0 . . . 10
Товарные знаки . . . 11
Условные обозначения . . . 12

Глава 1. Введение в рутины **13**
1.1 Зачем эти рутины . . . 13
1.2 Соглашение об условных значениях . . . 15
1.3 Типы аргументов . . . 16
1.4 Именование . . . 17
1.5 Безопасность потоков . . . 18
1.6 Как читается статья . . . 19

Глава 2. Служебные рутины **20**
2.1 Условные значения и сообщения . . . 21
STS$K_* – Константы серьёзности . . . 22
$DEFFAC, $DEFMSG, $DEFSEV – Построение условного значения . . . 23
$FAC, $MSG, $SEV – Разбор условного значения . . . 24
$STS_ERROR – Проверка на ошибку . . . 25
UTIL$S_INVARG, UTIL$S_INQUE, UTIL$S_NOLOCK, UTIL$S_DUPFAC – Условные значения библиотеки . . . 26
EMSG_RECORD, EMSG_RECORD_DESC – Типы каталога сообщений . . . 27
__util$inimsg – Регистрация каталога сообщений . . . 28
__util$getmsg – Поиск записи сообщения . . . 29
__util$putmsg – Сигнализация сообщения (формат printf) . . . 30
__util$putmsgd – Сигнализация сообщения с местом вызова . . . 31
__util$putmsg_fao – Сигнализация сообщения (управляющая строка FAO) . . . 32
__util$putmsgd_fao – Сигнализация сообщения (FAO) с местом вызова . . . 33
$PUTMSG – Сигнализация сообщения (макрос) . . . 34
$PUTMSG_FAO – Сигнализация сообщения, FAO (макрос) . . . 35
2.2 Журнал, трассировка и вывод . . . 36
__util$log – Запись в журнал (формат printf) . . . 37
__util$logd – Запись в журнал с местом вызова . . . 38
__util$log_fao – Запись в журнал (управляющая строка FAO) . . . 39
__util$logd_fao – Запись в журнал (FAO) с местом вызова . . . 40
$LOG – Запись в журнал (макрос) . . . 41
$LOG_FAO – Запись в журнал, FAO (макрос) . . . 42
$IFLOG – Запись в журнал, если флаг установлен (макрос) . . . 43
__util$log2buf – Форматирование записи журнала в буфер . . . 44
__util$out – Запись строки в дескриптор журнала . . . 45
__util$deflog – Направление журнала в файл и на хост syslog . . . 46
__util$setlogfd – Направление журнала в открытый дескриптор . . . 47

__util$rewindlogfile – Усечение файла журнала за пределом . . . 48
__util$rotatelogfile – Ротация файла журнала за пределом . . . 49
__util$syslog – Отправка сообщения на хост syslog . . . 50
__util$trace – Запись трассировки . . . 51
$TRACE, $IFTRACE – Трассировка (макросы) . . . 52
__util$dumphex – Запись шестнадцатеричного дампа . . . 53
$DUMPHEX – Запись шестнадцатеричного дампа (макрос) . . . 54
2.3 Счётные строки и работа со строками . . . 55
ASC, ASC$K_SZ – Тип счётной строки . . . 55
$ASCDECL, $ASCINI, $ASCLEN, $ASCPTR, $ASC, $ASCNIL – Макросы счётных строк . . . 56
__util$str2asc – Копирование строки Си в счётную строку . . . 57
__util$asc2str – Копирование счётной строки в строку Си . . . 58
__util$cpyasc – Копирование счётной строки . . . 59
__util$cmpasc – Сравнение счётных строк . . . 60
__util$cmpasc_blind – Сравнение счётных строк без учёта регистра . . . 61
__util$strelem – Взятие элемента строки с разделителями . . . 62
__util$uint32_2_dec – Преобразование беззнакового целого в десятичный текст 63
__util$trim – Удаление ведущих и замыкающих пробельных символов . . . 64
__util$collapse – Удаление всех пробельных символов . . . 65
__util$uncomment – Отсечение комментария от строки . . . 66
__util$strcat_rx – Конкатенация списка строк . . . 67
__util$strstr – Поиск счётной строки в счётной строке . . . 68
CRLF, CRLFCRLF, CRLFCRLF_LW – Константы ограничителя строки . . . 69
2.4 Память . . . 70
__util$movc5 – Перемещение символов с заполнением . . . 70
__util$fill – Заполнение буфера образцом . . . 71
__util$bzero – Очистка памяти . . . 72
__util$iszero – Проверка памяти на нули . . . 73
__util$strxor – Исключающее ИЛИ данных с ключом . . . 74
$ARRSZ – Число элементов массива . . . 75
2.5 Сопоставление с образцом . . . 76
__util$pattern_match – Сопоставление строки с образцом-шаблоном . . . 76
2.6 Шестнадцатеричные числа, UUID, base64 и CRC . . . 77
__util$hex2bin – Преобразование шестнадцатеричного текста в двоичные данные . . . 77
__util$bin2hex – Преобразование двоичных данных в шестнадцатеричный текст 78
$BIN2HEX – Преобразование двоичных данных в шестнадцатеричные (макрос) 79
__util$faohex – Форматирование октетов в одну строку шестнадцатеричного дампа . . . 80
__util$uuid2str – Преобразование UUID в текст . . . 81
__util$base64 – Кодирование октетов в base64 . . . 82
__util$unbase64 – Декодирование base64 . . . 83
__util$crc32c – Вычисление CRC-32 . . . 84
2.7 Ключевые слова и символы . . . 85
KWDENT, UTIL$K_LOOKUP_* – Типы таблицы ключевых слов . . . 85
__util$lookup_key – Поиск ключевого слова в таблице . . . 86
SYM_REC, $SYM_REC_INI, $SYM_REC_EOL – Типы таблицы символов . . . 88
__util$val2sym – Перевод значения в его имя . . . 89
__util$mask2sym – Перевод маски в имена . . . 90
2.8 Параметры и конфигурация . . . 91
OPTS, OPTS$K_*, OPTS_NULL – Типы таблицы параметров . . . 91
__util$getparams – Чтение параметров из командной строки . . . 93
__util$readconfig – Чтение параметров из файла . . . 95
__util$showparams – Запись параметров в журнал . . . 96
2.9 Очереди, блокировки и атомарные операции . . . 97
ENTRY, __QUEUE, QUEUE_INITIALIZER – Типы очереди . . . 98
__util$lockspin – Захват спин-блокировки . . . 99

__util$unlockspin – Освобождение спин-блокировки . . . 100
$LOCK_LONG, $UNLOCK_LONG – Макросы блокировки . . . 101
__util$insqtail – Вставка элемента в хвост . . . 102
__util$insqhead – Вставка элемента в голову . . . 103
__util$remqhead – Удаление элемента из головы . . . 104
__util$remqtail – Удаление элемента из хвоста . . . 105
__util$remqent – Удаление заданного элемента . . . 106
__util$movqhead – Перемещение элемента в голову . . . 107
__util$movqtail – Перемещение элемента в хвост . . . 108
__util$q_unlink – Изъятие элемента из цепочки очереди . . . 109
__util$clrqueue – Опустошение очереди . . . 110
$INSQHEAD, $INSQTAIL, $REMQHEAD, $REMQTAIL, $REMQENT, $MOVQHEAD, $MOVQTAIL, $CLRQUE – Макросы очереди . . . 111
$CMP_STORE_LONG – Сравнение и обмен . . . 112
$ATOMIC_INC, $ATOMIC_DEC – Атомарные увеличение и уменьшение . . . 113
2.10 Диапазоны, минимум и максимум . . . 114
__util$min, __util$max, $MIN, $MAX – Меньшее и большее из двух целых . . . 114
__util$isinrange, $ISINRANGE – Проверка значения на принадлежность замкнутому диапазону . . . 115
__util$isinrange2, $ISINRANGE2 – Проверка значения на принадлежность замкнутому диапазону (вторая форма) . . . 116
__util$range, $RANGE – Ограничение значения диапазоном . . . 117
2.11 Списки элементов . . . 118
ILE2, ILE3 – Типы элемента списка элементов . . . 118
$ILE2INI, $ILE3INI, $ILENULL – Макросы списка элементов . . . 119
2.12 Помощники времени . . . 120
__util$timbuf – Разложение местного времени . . . 121
__util$add_time – Сложение двух времён . . . 122
__util$sub_time – Вычитание двух времён . . . 123
__util$cmp_time – Сравнение двух времён . . . 124
2.13 Пустая рутина . . . 125
__util$nope – Ничего не делать . . . 125
Прочие определения . . . 126
3.1 Соглашения процедур FAO . . . 127
3.2 Директивы FAO . . . 129
__util$fao – Форматированный вывод ASCII (список параметров) . . . 135
__util$faol – Форматированный вывод ASCII (массив параметров) . . . 137
$FAO_DESC – Инициализация выходного дескриптора . . . 139
3.3 Формат двоичного времени . . . 140
__util$gettim – Получение двоичного времени . . . 141
__util$bintim – Преобразование времени ASCII в двоичное время . . . 142
__util$asctim – Преобразование двоичного времени в ASCII . . . 144
__util$add_times – Сложение двух двоичных времён . . . 146
__util$sub_times – Вычитание двух двоичных времён . . . 147
__util$add_time – Сложение двух значений timespec . . . 148
__util$sub_time – Вычитание двух значений timespec . . . 149
__util$cmp_time – Сравнение двух значений timespec . . . 150
__util$timbuf – Разбор времени в календарную структуру . . . 151
3.4 FAO в вызовах журналирования и сообщений . . . 152
3.5 Отличия от OpenVMS . . . 153
3.6 Изменения в версии 1.7.0 . . . 154

Глава 4. Процедуры командного языка 155
4.1 Таблицы команд . . . 156
4.2 Типы значений . . . 159
4.3 Флаги и параметры . . . 161
4.4 Правила разбора . . . 162
4.5 Ключевые слова . . . 164

4.6 Вложенные глаголы . . . 165
__cli$parse – Разбор командной строки . . . 166
__cli$dispatch – Вызов процедуры действия . . . 168
__cli$get_value – Получение значения . . . 169
__cli$get_keyword – Приведение значения ключевого слова . . . 171
__cli$present – Проверка наличия элемента . . . 173
__cli$cleanup – Освобождение контекста . . . 174
__cli$show_verbs – Вывод таблицы глаголов . . . 175
__cli$help – Вывод экрана справки . . . 176
__cli$show_ctx – Вывод контекста . . . 177
4.7 Изменения в версии 1.7.0 . . . 178

Глава 5. Процедуры протокола TLV 179
5.1 Раскладка PDU . . . 180
__tlv$hset – Установка заголовка PDU . . . 182
__tlv$hget – Получение заголовка PDU . . . 183
__tlv$encode_tag – Упаковка тега . . . 185
__tlv$decode_tag – Распаковка тега . . . 186
__tlv$put – Добавление записи . . . 187
__tlv$get – Получение значения . . . 189
__tlv$lookup – Поиск записи . . . 191
__tlv$dump – Вывод PDU . . . 193
5.2 Изменения в версии 1.7.0 . . . 194

Приложение A. Условные значения и серьёзность 195
A.1 Раскладка условного значения . . . 196
A.2 Значения серьёзности . . . 197
A.3 Макросы . . . 198
A.4 Значения библиотечных модулей . . . 199

Приложение B. Указатель процедур и макросов 202

Предисловие

Кому адресовано

Руководство адресовано программисту, который пишет на Си с библиотекой StarLet и хочет знать точно, для одной рутины или одного макроса, что она принимает, что делает и что возвращает. Предполагается C11 и инструментарий POSIX или Windows. OpenVMS не предполагается, хотя читатель, знающий библиотеку времени исполнения OpenVMS (Run-Time Library), найдёт знакомыми состав, имена и условные значения.

Руководство справочное. Его читают по статьям, а не подряд. Об обоснованиях устройства и о разобранных программах см. *Руководство программиста STARLET* (*STARLET Programmer's Guide*).

Структура документа

Часть	Содержание
Глава 1	Введение: соглашение об условных значениях, типы аргументов, именование, безопасность потоков, как читается статья
Глава 2	Служебные рутины (`utility_routines`): сообщения, журнал, строки, память, сопоставление с образцом, кодировки, ключевые слова, параметры, очереди, диапазоны, списки элементов
Глава 3	Службы FAO и времени (`starlet`)
Глава 4	Рутины командного языка (`cli_routines`)
Глава 5	Рутины протокола TLV (`avproto`)
Приложение A	Условные значения и серьёзность
Приложение B	Указатель рутин и макросов

Связанные документы

- *Руководство программиста STARLET* (*STARLET Programmer's Guide*), номер издания STARLET-PG-0171: устройство библиотеки, правила кодирования, которым она следует, и разобранные примеры.

Исходные тексты рутин: `utility_routines.h` и `utility_routines.c`, `starlet.h` и `starlet.c`, `cli_routines.h` и `cli_routines.c`, `avproto.h` и `avproto.c`. Каждая рутина файлов `.c` имеет в заголовке блок `FUNCTIONAL DESCRIPTION`. Регрессионные тесты в каталоге `tests/` показывают настоящие вызовы. Если руководство и поведение кода расходятся, прав код, а руководство подлежит исправлению.

Изменения в версии 1.7.1

Версия 1.7.1 закрывает восемь пунктов, оставшихся от 1.7.0: `__tlv$lookup` проверяет длину записи фиксированного размера; рутины журнала возвращают `STS$K_ERROR` для неопределённой серьёзности; `$LOG_FAO` и `$PUTMSG_FAO` помечают в записи неверный статус FAO (`[FAO  BADDIR]`); `__util$hex2bin` проверяет ввод; `OPTS$K_INT` из 1, 2 и 8 октетов беззнаковый, из 4 октетов и `int` – знаковый; парные кавычки снимаются со значения параметра; `__cli$get_keyword` возвращает `STS$K_WARN` для пустого значения; комментарий `__util$deflog` исправлен. Каждая статья описывает поведение версии 1.7.1, а Приложение D *Руководства программиста STARLET* (STARLET-PG-0171) перечисляет то, что может заметить потребитель.

Изменения в версии 1.7.0

Версия 1.7.0 исправляет немало дефектов версии 1.6.11 и вместе с ними меняет несколько значений и прототипов. Каждая статья руководства описывает поведение версии 1.7.1. Список того, что может заметить потребитель библиотеки, с причиной каждого изменения, приведён в Приложении D *Руководства программиста STARLET* (STARLET-PG-0171).

Товарные знаки

OpenVMS, VMScluster, Alpha и Digital - товарные знаки Hewlett Packard Enterprise Development LP или VMS Software, Inc. Linux - зарегистрированный товарный знак Линуса Торвальдса в США и других странах. Windows - товарный знак группы компаний Microsoft. Android - товарный знак Google LLC. Arm - товарный знак Arm Limited. Intel - товарный знак Intel Corporation. Все остальные названия продуктов и компаний - товарные знаки их владельцев.

StarLet - независимая реализация. Она не содержит кода OpenVMS и не связана с перечисленными здесь владельцами, не одобрена и не поддерживается ими; их названия употреблены лишь затем, чтобы сказать, по какому образцу построена библиотека и где она работает.

Условные обозначения

Статья руководства описывает одну рутину или один макрос. Она построена как статья справочника по библиотеке времени исполнения:

Заголовок	Значение
название	Имя рутины и, после двух дефисов, что она делает
Формат	Прототип на Си или макрос, точно как в заголовочном файле
Возвращает	Тип значения и что оно означает
Аргументы	По пункту на аргумент: тип, доступ, механизм, описание
Описание	Что делает рутина, её граничные случаи и ограничения, как их ведёт себя код
Возвращаемые условные значения	Таблица значений, которые возвращает рутина, и когда
Пример	Короткий фрагмент на Си

Доступ бывает: *только чтение* - рутина не меняет то, что обозначает аргумент; *только запись* - рутина записывает туда значение и старого не читает; *изменение* - рутина читает значение и записывает новое. *Механизм* бывает *по значению* или *по ссылке*, то есть передаётся адрес.

Имена рутин и макросов набраны `моноширинным шрифтом`. В сплошном тексте рутина пишется с пустыми скобками, `__util$strstr()`. Знак доллара - часть имени, как в OpenVMS; в Си с GCC, Clang или MinGW-w64 он ничем не особенный.

Даты имеют вид `31-OCT-2026`. Число вида `0x1F` - шестнадцатеричное.

Глава 1. Введение в рутины

1.1 Зачем эти рутины

Рутины `utility_routines` - это небольшие устойчивые части, которые каждая программа проекта в конце концов пишет для себя сама: журнал, сообщение, очередь, таблица параметров, контрольная сумма. Здесь они написаны один раз. Вот что в них сделано правильно, и код это показывает:

Каждый вызов сообщает, чем он кончился, и единым способом. Рутина, которая может потерпеть неудачу, возвращает условное значение (condition value). Его младший бит установлен при успехе и сброшен при неудаче, поэтому вся библиотека проверяется одной проверкой, `1 & status`, - проверкой OpenVMS. Ничто не сигнализируется через `errno`, через нулевой указатель в одном месте и отрицательное число в другом. Статус несёт свою серьёзность (severity), так что вызывающий отличает предупреждение от ошибки, а ошибку от фатальной без всякой таблицы.

Длина переносится, а не отыскивается. Строка, которая может содержать нулевой октет, - это `ASC`, счётная строка (counted string); буфер - это адрес и длина; кодировщики и декодировщики, `__util$strstr()`, `__util$trim()` и остальные принимают длину и не вызывают снова и снова `strlen()`. Рутина, пишущая в буфер, принимает его размер и не пишет за его пределы; рутины журнала усекают запись, а не переполняют буфер, и тесты (`tests/test_log_overflow.c`, `tests/test_strings.c`, `tests/test_queue.c`) это удерживают.

Один каталог сообщений, одна форма сообщения. Программа один раз регистрирует свои сообщения через `__util$inimsg()`, а `$PUTMSG_FAO()` находит сообщение по его условному значению и печатает его в форме OpenVMS, `%FACILITY-S-IDENT, text`. Текст сообщения написан в одном месте, а не в каждом месте, где оно сигнализируется, и статус, пересекающий границу модуля, остаётся тем же сообщением.

Один форматер для журналов, сообщений и вывода. Рутины `_FAO`, `$LOG_FAO()` и `$PUTMSG_FAO()`, форматируют службой FAO модуля `starlet`. Одни и те же управляющие строки служат записи журнала, сообщению каталога и строке, собираемой в буфере. Макросы приводят каждый параметр к нужному типу и подсчитывают их, так что вызывающий пишет значения и больше ничего.

Запись журнала говорит, откуда она. Каждая запись несёт время с миллисекундами и номер потока. Запись отладочной сборки несёт модуль, рутину и строку вызова, `[module\routine:line]`, которые макрос берёт из `__MODULE__`, `__FUNCTION__` и `__LINE__`. Журнал нескольких потоков читается по потокам; запись находится в исходном тексте по её строке.

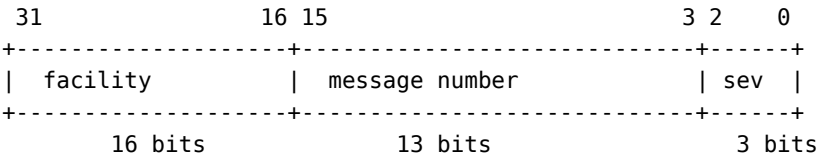
Малые, статические и без зависимостей. Большинство рутин - `static inline` в одном заголовке и не требуют никакой библиотеки. Остальные находятся в одном исходном файле, которому нужны библиотека Си и, для `__util$fao()` и ей подобных, модуль `starlet` того же пакета. На частых путях нет выделения памяти: буферы принадлежат вызывающему или стеку. Код собирается для Linux на x86-64 и ARMv8, для Windows через MinGW-w64 и для Android, а контрольная сумма использует

команды процессора, где они есть.

Проверено тестами. Каталог `tests/` содержит регрессионную программу для раскладки условного значения, каталога сообщений, записи журнала и её переполнения, ротации файла журнала, журнала, перенаправленного на дескриптор, сопоставления с образцом, поиска ключевых слов, base64, контрольной суммы, очередей, помощников для строк и памяти и рутин параметров. Найденный дефект получает тест раньше, чем исправление.

1.2 Соглашение об условных значениях

Условное значение (condition value) - беззнаковое 32-разрядное целое. Его возвращают рутины библиотеки, и им же названы сообщения каталога. Оно устроено как в OpenVMS:



Серьёзность (severity) - три младших бита, и это единственная часть, которую несут простые константы.

Константа	Значение	Буква	Смысл
`STS$K_WARN`	0	W	Предупреждение: сделано, но посмотрите
`STS$K_SUCCESS`	1	S	Успех
`STS$K_ERROR`	2	E	Ошибка: не сделано
`STS$K_INFO`	3	I	Информация; считается успехом
`STS$K_FATAL`	4	F	Фатальная ошибка
`STS$K_UNDEF`, `STS$K_UNDEF5`	5	?	Не серьёзность; серьёзность вне диапазона показывается так, а рутины журнала возвращают для неё `STS$K_ERROR`
`STS$K_UNDEF6`, `STS$K_UNDEF7`	6, 7	?	Не используются

Проверка - это младший бит. Условное значение с установленным младшим битом - успех; со сброшенным - неудача. Поэтому `STS$K_SUCCESS` (1) и `STS$K_INFO` (3) проверку проходят, а `STS$K_WARN` (0), `STS$K_ERROR` (2) и `STS$K_FATAL` (4) - нет. Проверка пишется `1 & status`:

```
if ( !(1 & (status = __util$base64(src, srclen, dst, dstsz, &dstlen))) )
    return  $LOG(STS$K_ERROR, "base64 failed, status = %d", status);
```

Не сравнивайте статус с `STS$K_SUCCESS`, если статья рутины не говорит, что значение именно такое: рутина может вернуть `STS$K_INFO`, а статус, построенный через `$DEFMSG()`, несёт номер сообщения в старших битах.

`$DEFFAC()`, `$DEFMSG()` и `$DEFSEV()` строят условное значение из частей, а `$FAC()`, `$MSG()` и `$SEV()` разбирают его. `$MSG()` обратен `$DEFMSG()`; он берёт 13 бит номера сообщения и не больше. `$STS_ERROR()` истинно для ошибки или фатальной ошибки. Макросы описаны в разделе 2.1.

Несколько рутин не входят в соглашение, и их статьи об этом говорят: `__util$pattern_match()` возвращает значение истинности, `__util$crc32c()` возвращает контрольную сумму, а рутина, преобразующая или измеряющая, возвращает длину. Рутина, возвращающая длину, возвращает её в `int`, и отрицательное значение или нуль не есть ошибка, если статья не говорит иного.

1.3 Типы аргументов

Счётные строки. Тип `ASC` - счётная строка OpenVMS, ASCIC: один октет длины и до 255 октетов текста.

```
typedef struct _asc { unsigned char len; char sts[ASC$K_SZ]; } ASC;    /* ASC$K_SZ is 255
 ↪  */
```

Текст не обязан заканчиваться нулевым октетом и может его содержать. Многие рутины библиотеки оставляют после текста лишний нуль для удобства Си, и там, где они так делают, статья об этом говорит. Макросы `$ASCDECL()`, `$ASCINI()`, `$ASCLEN()`, `$ASCPTR()`, `$ASC()` и `$ASCNIL` объявляют, инициализируют и разбирают `ASC`. Макрос `$ASC()` раскрывается в длину и адрес, так что `ASC` печатается через `"%.*s"`.

Буферы с длинами. Буфер передаётся как адрес и длина, в этом порядке, а если рутина пишет в него - как адрес, размер и адрес переменной, получающей длину результата. Выходная длина устанавливается и тогда, когда буфер оказался мал, чтобы вызывающий узнал, каким буфер должен был быть (`__util$base64()`, `__util$unbase64()`).

Списки элементов. Типы `ILE2` и `ILE3` - элементы списка элементов (item list) OpenVMS: код, длина и адрес, а для `ILE3` ещё и адрес слова, получающего возвращённую длину. Список кончается элементом из нулей (`$ILENULL`). Типы определены здесь, чтобы у программы было одно их определение; рутины, принимающие список, описаны в других главах. Обратите внимание на раскладку `ILE3`: код идёт первым, а длина вторым - наоборот, чем в `ILE3` OpenVMS; см. раздел 2.11.

Таблицы. Таблица - массив, который кончается элементом-ограничителем или размер которого передаётся. Таблицы библиотеки: `OPTS` (параметры, кончается `OPTS_NULL`), `KWDENT` (ключевые слова, размер передаётся), `SYM_REC` (символы, кончается `$SYM_REC_EOL`) и `EMSG_RECORD` (сообщения, размер в дескрипторе).

Указатели. Аргумент-указатель, являющийся *выходным*, передаётся по ссылке и на нуль не проверяется, если статья не говорит иного. Макросы, оборачивающие рутины очередей, приводят свои аргументы к нужному типу, так что принимается любой указатель на структуру, начинающуюся с `ENTRY`.

1.4 Именование

`__util$name`. Рутина называется `__util$` и слово в нижнем регистре; это префикс средства (facility) библиотеки, как `LIB$` в OpenVMS. Имя - допустимый идентификатор Си в компиляторах GCC, Clang и MinGW-w64; знак доллара ничем не особенный.

`$NAME`. Макрос, оборачивающий рутину, или константа средства начинается со знака доллара и пишется в верхнем регистре: `$LOG()`, `$INSQTAIL()`, `$ASCDECL()`. Константа группы называется `GROUP$K_NAME`, маска - `GROUP$M_NAME`: `STS$K_ERROR`, `OPTS$K_STR`, `UTIL$K_LOOKUP_NCASE`, `STS$M_SYSLOG`.

`__MODULE__` и `__FAC__`. Программа называет себя рутинам журнала двумя макросами, которые определяются *до* включения `utility_routines.h`:

```
#define __MODULE__  "CONF$"     /* the module: shows in a debug record, default NULL    */
#define __FAC__     "MYAPP"     /* the facility: shows in every record, default "UTILS" */
#include "utility_routines.h"
```

`__MODULE__` - строка, обычно имя исходного файла или модуля; если она не определена, то равна `NULL`, и отладочная запись показывает только рутину и строку. `__FAC__` - имя средства, которое в записи стоит между `%` и буквой серьёзности, `%MYAPP-E:`. Если оно не определено, оно равно `"UTILS"`.

Отладка и выпуск. Макросы `$LOG()`, `$LOG_FAO()`, `$PUTMSG()`, `$PUTMSG_FAO()`, `$TRACE()` и `$IFTRACE()` различаются в зависимости от `NDEBUG`. Без `NDEBUG` (отладочная сборка) они вызывают варианты `d`, `__util$logd()` и подобные, с `__MODULE__`, `__FUNCTION__` и `__LINE__`, и запись показывает, где был вызов. С `NDEBUG` они вызывают простые рутины, а `$TRACE()` и `$IFTRACE()` компилируются в ничто.

Компиляторы. Заголовок использует расширение GNU `, ## __VA_ARGS__` и встроенные функции `__sync_*` и `__atomic_*`. Он написан для GCC, Clang и MinGW-w64. Рекомендуется `-std=gnu11`.

Компоновка. Рутины, не являющиеся `static inline`, находятся в `utility_routines.c`; программа компонуется с библиотекой `libstarlet`. Рутины FAO находятся в модуле `starlet`, и рутины `_FAO` этой главы их вызывают: включайте `starlet.h` там, где используются `$LOG_FAO()` или `$PUTMSG_FAO()`, потому что макросы приводят свои параметры к `fao_prm_t`, определённому там. Начиная с 1.7.0 `starlet.h` сам включает `utility_routines.h`, так что два заголовка можно включать в любом порядке.

1.5 Безопасность потоков

Библиотека рассчитана на программы из нескольких потоков. Что гарантируется:

- **Журнал безопасен.** Запись строится в стеке вызывающего и пишется одним вызовом `write()`, поэтому записи разных потоков не смешиваются внутри записи. Дата и номер потока в префиксе кэшируются в локальной памяти потока (thread-local storage), для каждого потока, без блокировки.
- **Дескриптор журнала не защищён.** `__util$deflog()`, `__util$setlogfd()` и `__util$rotatelogfile()` меняют дескриптор, в который пишут все потоки. Вызывайте `__util$deflog()` или `__util$setlogfd()` при запуске, до старта потоков. `__util$rotatelogfile()` можно безопасно вызывать из потока, пока другие пишут в журнал; ротацию делает только один поток за раз, остальные возвращаются сразу.
- **Каталог сообщений.** Начиная с 1.7.0 `__util$inimsg()` безопасна для потоков: регистрации упорядочиваются, а список публикуется лишь тогда, когда дескриптор полон, так что одновременный `__util$getmsg()` видит либо старый список, либо новый. Регистрация при запуске по-прежнему аккуратнее. `__util$getmsg()` и рутины, сигнализирующие сообщения, список только читают.
- **Очереди взаимоблокируемы.** Рутины очередей берут спин-блокировку (spin lock) в заголовке очереди. Элемент вставляется, удаляется или подсчитывается атомарно по отношению к другим вызовам очередей той же очереди. Ограничения отдельных рутин - в их статьях (раздел 2.9).
- **Контрольная сумма** выбирает свой механизм при первом вызове. Два потока, вызвавшие её впервые, решают одинаково, так что гонка безвредна.
- **`__util$syslog()`** держит сокет в переменной `static`. Сокет создаётся один раз, под защитой, так что два потока, делающие первый вызов одновременно, безопасны (1.7.0).
- **Рутины параметров** (`__util$getparams()`, `__util$readconfig()`) пишут переменные таблицы и не рассчитаны на вызов из нескольких потоков; их используют один раз, в `main()`.

Рутины `static inline`, работающие только со своими аргументами, безопасны настолько, насколько безопасны их аргументы: два потока могут вызвать `__util$trim()` для двух буферов, но не для одного.

1.6 Как читается статья

Статья начинается с имени рутины и краткого названия. **Формат** - прототип, как он стоит в заголовке. **Возвращает** даёт тип и смысл значения. В разделе **Аргументы** аргумент описан так:

имя – `тип`; доступ; механизм. Что это.

так что `src -- void *; только чтение; по ссылке` - это адрес данных, которые рутина читает. **Описание** говорит, что рутина делает, чего не делает и каковы ограничения: размер, диапазон, условие, при котором она ничего не делает. **Возвращаемые условные значения** перечисляют значения и случаи. Статья рутины, не возвращающей условное значение, говорит об этом там же и перечисляет то, что она возвращает. **Пример** - фрагмент, который компилируется, если поместить его в функцию; он предполагает включения и макросы раздела 1.4 и намеренно короток.

Макрос документируется так же. Его **Формат** показывает макрос, как он определён; если макрос раскрывается в вызов рутины, статья говорит, в какой, а аргументы макроса описаны как аргументы макроса, а не рутины.

Глава 2. Служебные рутины

Эта глава описывает каждую рутину и макрос, объявленные в `utility_routines.h` и определённые там (`static inline`) или в `utility_routines.c`. Они сгруппированы по назначению, в таком порядке:

Раздел	Группа
2.1	Условные значения и сообщения
2.2	Журнал, трассировка и вывод
2.3	Счётные строки и работа со строками
2.4	Память
2.5	Сопоставление с образцом
2.6	Шестнадцатеричные числа, UUID, base64 и CRC
2.7	Ключевые слова и символы
2.8	Параметры и конфигурация
2.9	Очереди, блокировки и атомарные операции
2.10	Диапазоны, минимум и максимум
2.11	Списки элементов
2.12	Помощники времени
2.13	Пустая рутина

Разделение с модулем `starlet`. Рутины `__util$fao()`, `__util$faol()`, `__util$bintim()`, `__util$asctim()`, `__util$gettim()`, `__util$add_times()` и `__util$sub_times()` имеют префикс `__util$`, но *определены в `starlet.c`* и объявлены в `starlet.h`. Они описаны в главе 3. `__util$faohex()`, `__util$timbuf()`, `__util$add_time()`, `__util$sub_time()` и `__util$cmp_time()` определены в `utility_routines.*` и описаны здесь (разделы 2.6 и 2.12). Заметьте разницу между `__util$add_time()` (эта глава: две `struct timespec`) и `__util$add_times()` (глава 3: `fao_time_t`).

Рутины `__util$trace()`, `__util$dumphex()`, рутины журнала и рутины сообщений пишут в **дескриптор журнала**: стандартный вывод, пока `__util$deflog()` или `__util$setlogfd()` его не изменит.

2.1 Условные значения и сообщения

Константы и макросы, строящие и разбирающие условное значение, и каталог сообщений. Раскладка условного значения - в разделе 1.2.

STS$K_* – Константы серьёзности

Формат

```
enum {
    STS$K_WARN  = 0,
    STS$K_SUCCESS   = 1,
    STS$K_ERROR = 2,
    STS$K_INFO  = 3,
    STS$K_FATAL = 4,
    STS$K_UNDEF = 5,
    STS$K_UNDEF5    = 5,
    STS$K_UNDEF6    = 6,
    STS$K_UNDEF7    = 7,
    STS$K_MAX   = 8
};

#define STS$M_SYSLOG    16
#define STS$C__WARN "W"
#define STS$C__SUCCESS  "S"
#define STS$C__ERROR    "E"
#define STS$C__INFO "I"
#define STS$C__FATAL    "F"
#define STS$C__UNDEF    "?"
```

Описание

Серьёзности условного значения. `STS$K_MAX` - конец списка, число серьёзностей. `STS$M_SYSLOG` - флаг, а не серьёзность: его добавляют по ИЛИ к аргументу серьёзности `__util$log()`, `__util$log_fao()` и `$LOG()`, чтобы запись была также отправлена в системный журнал (см. `__util$log()`). `STS$C__*` - буквы, показываемые в записи.

$DEFFAC, $DEFMSG, $DEFSEV – Построение условного значения

Формат

```
#define $DEFFAC(code)   ((code) << 16)
#define $DEFMSG(code)   ( ((code) << 3) & 0x0000FFF8UL )
#define $DEFSEV(code)   ((code) & 0x7)
```

Возвращает

Целочисленное выражение: поле, сдвинутое на своё место в условном значении.

Аргументы

- *code* – целое; только чтение; по значению. Для `$DEFFAC()` номер средства, до 16 бит; для `$DEFMSG()` номер сообщения, 13 бит; для `$DEFSEV()` серьёзность, `STS$K_*`.

Описание

Три макроса строят три поля условного значения; поля объединяются по ИЛИ. `$DEFMSG()` отбрасывает биты номера сообщения выше 13-го. `$DEFFAC()` не маскирует: номер средства должен умещаться в 16 бит.

Пример

```
#define MYAPP$K_FACNO   0x0123
#define MYAPP$_OPENFAIL ($DEFFAC(MYAPP$K_FACNO) | $DEFMSG(1) | $DEFSEV(STS$K_ERROR))
#define MYAPP$_STARTED  ($DEFFAC(MYAPP$K_FACNO) | $DEFMSG(2) | $DEFSEV(STS$K_INFO))
```

$FAC, $MSG, $SEV – Разбор условного значения

Формат

```
#define $FAC(code)  ((code) >> 16)
#define $MSG(code)  ( ((code) >> 3) & 0x00001FFFUL )
#define $SEV(code)  ((code) & 0x7)
```

Возвращает

Целочисленное выражение: номер средства, номер сообщения, серьёзность.

Аргументы

- *code* – беззнаковое целое; только чтение; по значению. Условное значение.

Описание

`$FAC()`, `$MSG()` и `$SEV()` обратны `$DEFFAC()`, `$DEFMSG()` и `$DEFSEV()`. Передавайте им значение типа `unsigned`: `$FAC()` сдвигает без маскирования, и отрицательное `int` расширило бы знак.

Пример

```
unsigned l_sts = MYAPP$_OPENFAIL;

if ( $SEV(l_sts) == STS$K_ERROR && $FAC(l_sts) == MYAPP$K_FACNO )
    $LOG(STS$K_INFO, "message number %u", $MSG(l_sts));
```

$STS_ERROR – Проверка на ошибку

Формат

```
#define $STS_ERROR(code)    ( ($SEV(code) == STS$K_ERROR) || ($SEV(code) == STS$K_FATAL) )
```

Возвращает

Не нуль, если серьёзность *code* есть `STS$K_ERROR` или `STS$K_FATAL`; иначе нуль.

Аргументы

- *code* – беззнаковое целое; только чтение; по значению. Условное значение. Макрос вычисляет его дважды.

Описание

Истинно для двух серьёзностей, означающих, что операция не удалась. Предупреждение здесь не ошибка, хотя его младший бит сброшен. Для проверки успеха используйте `1 & code`.

UTIL$S_INVARG, UTIL$S_INQUE, UTIL$S_NOLOCK, UTIL$S_DUPFAC – Условные значения библиотеки

Формат

```
#define UTIL$S_INVARG   0x102
#define UTIL$S_INQUE    0x202
#define UTIL$S_NOLOCK   0x302
#define UTIL$S_DUPFAC   0x400
```

Описание

Условные значения рутин вставки в очередь и `__util$inimsg()`. Они следуют раскладке раздела 1.2: номер сообщения лежит выше трёх младших битов, составляющих серьёзность.

Значение	Серьёзность	Смысл
`UTIL$S_INVARG` (0x102)	`STS$K_ERROR`	Нулевой или иным образом непригодный аргумент
`UTIL$S_INQUE` (0x202)	`STS$K_ERROR`	Элемент уже находится в другой очереди
`UTIL$S_NOLOCK` (0x302)	`STS$K_ERROR`	Спин-блокировка очереди не захвачена
`UTIL$S_DUPFAC` (0x400)	`STS$K_WARN`	Каталог этого средства уже зарегистрирован

Значения очереди возвращают `__util$insqtail()` и, начиная с 1.7.0, также `__util$insqhead()`. В 1.6.11 `UTIL$S_NOLOCK` было равно 0x202 и не отличалось от `UTIL$S_INQUE`; `UTIL$S_DUPFAC` новое. У всех четырёх младший бит сброшен. `UTIL$S_DUPFAC` - предупреждение, поэтому `$STS_ERROR()` для него ложно, но и `1 & status` ложно.

EMSG_RECORD, EMSG_RECORD_DESC – Типы каталога сообщений

Формат

```
typedef struct __emsg_record__  {
        int     sts;
        struct {
        unsigned char   textl;
            char    text[254];
        };
        const char  *facname;
        const char  *mnemonic;
    } EMSG_RECORD;

typedef struct __emsg_record_desc__ {
        struct __emsg_record_desc__ *link;
        unsigned        msgnr;
        unsigned        facno;
        struct __emsg_record__  *msgrec;
        const char      *facname;
    } EMSG_RECORD_DESC;
```

Описание

`EMSG_RECORD` - одно сообщение: его условное значение (*sts*), его текст как ASCIC (*textl* и *text*; текст используется и как строка Си, поэтому за ним должен следовать нулевой октет), имя средства (*facname*) и мнемоника (*mnemonic*). Последние два заполняет `__util$inimsg()`; таблица их задавать не обязана, разве что *mnemonic* можно задать как строковое представление имени условия, `"MYAPP$_OPENFAIL"`, из которого `__util$inimsg()` оставляет то, что следует за `$_`, `"OPENFAIL"`.

`EMSG_RECORD_DESC` описывает каталог одного средства: *msgrec* - массив, *msgnr* - число его элементов, *facno* - номер средства (число, задаваемое `$DEFFAC()`), *facname* - имя, показываемое в `%FACNAME-S-`. *link* принадлежит библиотеке. Дескриптор и массив должны оставаться действительными всё время работы программы.

Текст сообщения - формат printf для `__util$putmsg()` и управляющая строка FAO для `__util$putmsg_fao()`. Поэтому каталог пишется для одного из двух.

Пример

```
#define MYAPP$_MSGENT(a_sts, a_txt) { (int) (a_sts), { sizeof(a_txt) - 1, a_txt }, NULL,
 ↪  #a_sts }

static EMSG_RECORD s_msgs [] = {
    MYAPP$_MSGENT(MYAPP$_OPENFAIL,  "Cannot open !AZ, errno = !SL"),
    MYAPP$_MSGENT(MYAPP$_STARTED,   "Started, version !AZ"),
};

static EMSG_RECORD_DESC s_msgdsc = { NULL, $ARRSZ(s_msgs), MYAPP$K_FACNO, s_msgs, "MYAPP"
 ↪  };
```

__util$inimsg – Регистрация каталога сообщений

Формат

```
unsigned    __util$inimsg   (EMSG_RECORD_DESC *msgdsc);
```

Возвращает

`unsigned`: условное значение. В 1.6.11 обе неудачи были простым `STS$K_WARN`.

Аргументы

- *msgdsc* – `EMSG_RECORD_DESC *`; изменение; по ссылке. Дескриптор каталога.

Описание

Регистрирует каталог, чтобы рутины сообщений находили его записи. Рутина сортирует массив *msgrec* на месте по средству и номеру сообщения, проставляет *facname* в каждую запись, усекает каждую *mnemonic* до части после `$_` и связывает дескриптор в голову списка каталогов. Поэтому массив должен быть доступен для записи и не должен быть объектом `const`.

Средство регистрируется один раз. Второй дескриптор с тем же *facno* отвергается, и ничего не меняется. Рутина безопасна для потоков (1.7.0): регистрации упорядочиваются, а список публикуется лишь тогда, когда дескриптор полон (раздел 1.5).

Возвращаемые условные значения

Значение	Смысл
`STS$K_SUCCESS`	Каталог зарегистрирован
`UTIL$S_INVARG` (0x102)	*msgdsc* равен нулю
`UTIL$S_DUPFAC` (0x400)	Каталог средства с тем же номером уже зарегистрирован; ничего не изменено

Пример

```
unsigned l_sts;

if ( !(1 & (l_sts = __util$inimsg(&s_msgdsc))) && (UTIL$S_DUPFAC != l_sts) )
    $LOG(STS$K_ERROR, "catalogue is not registered, status = %#x", l_sts);
```

__util$getmsg – Поиск записи сообщения

Формат

```
unsigned    __util$getmsg   (unsigned sts, EMSG_RECORD **outmsg);
```

Возвращает

`unsigned`: условное значение.

Аргументы

- *sts* – `unsigned`; только чтение; по значению. Условное значение нужного сообщения.
- *outmsg* – `EMSG_RECORD **`; только запись; по ссылке. Получает адрес записи.

Описание

Находит каталог средства из *sts*, затем запись двоичным поиском. Сравниваются средство и номер сообщения; серьёзность *sts* игнорируется, так что одна запись служит сообщению, сигнализируемому как информация в одном месте и как предупреждение в другом. Если запись не найдена, *outmsg* не изменяется.

Возвращаемые условные значения

Значение	Смысл
`STS$K_SUCCESS`	Найдена; *outmsg* содержит запись
`STS$K_ERROR`	Каталог средства не зарегистрирован или в нём нет такого номера сообщения

Пример

```
EMSG_RECORD *l_rec;

if ( 1 & __util$getmsg(MYAPP$_STARTED, &l_rec) )
    $LOG(STS$K_INFO, "the format is '%.*s'", l_rec->textl, l_rec->text);
```

__util$putmsg – Сигнализация сообщения (формат printf)

Формат

```
unsigned    __util$putmsg   (unsigned sts, ...);
```

Возвращает

`unsigned`: аргумент *sts*.

Аргументы

- *sts* – `unsigned`; только чтение; по значению. Условное значение сообщения.
- *...* – аргументы формата текста сообщения, по значению.

Описание

Пишет одну запись в дескриптор журнала: префикс (дата, время, номер потока), затем сообщение. Запись каталога находится через `__util$getmsg()`; её текст используется как **формат printf** со следующими аргументами. Оболочка `%FACILITY-S-` не добавляется.

Если запись не найдена, пишется текст по умолчанию: `%NONAME-<S>-NOMSG, Message number: <hex> [fac: ..., sev: ..., msgno: ...]`, с буквой серьёзности *sts*. Строка начинается со знака процента, как и всякая другая строка этой формы. Запись длиннее 2047 октетов обрезается. Рутина возвращает *sts*, так что функция может кончаться `return $PUTMSG(...)`. Это рутина, которую `$PUTMSG()` вызывает в сборке выпуска.

Возвращаемые условные значения

Эта рутина возвращает свой аргумент *sts* и никакого собственного условного значения.

__util$putmsgd – Сигнализация сообщения с местом вызова

Формат

```
unsigned    __util$putmsgd  (unsigned sts, const char *__mod, const char *__fi, unsigned
↪  __li, ...);
```

Возвращает

`unsigned`: аргумент *sts*.

Аргументы

- *sts* – `unsigned`; только чтение; по значению. Условное значение.
- *__mod* – `const char *`; только чтение; по ссылке. Имя модуля; может быть нулевым.
- *__fi* – `const char *`; только чтение; по ссылке. Имя рутины.
- *__li* – `unsigned`; только чтение; по значению. Номер строки.
- *...* – аргументы формата текста сообщения.

Описание

Как `__util$putmsg()`, но после префикса вставлено место вызова: `[module\routine:line]`, если *__mod* не нуль, и `[routine:line]`, если нуль. `$PUTMSG()` вызывает эту рутину в отладочной сборке.

Возвращаемые условные значения

Эта рутина возвращает свой аргумент *sts*.

__util$putmsg_fao – Сигнализация сообщения (управляющая строка FAO)

Формат

```
unsigned    __util$putmsg_fao   (unsigned sts, int prmcnt, ...);
```

Возвращает

`unsigned`: аргумент *sts*.

Аргументы

- *sts* – `unsigned`; только чтение; по значению. Условное значение.
- *prmcnt* – `int`; только чтение; по значению. Число следующих параметров FAO, от 0 до 16. Большее число принимается за 16, отрицательное - за 0.
- *...* – `fao_prm_t`, по значению. Параметры FAO. Каждый должен передаваться как `fao_prm_t`.

Описание

Пишет одну запись: префикс, оболочку, текст. Запись находится через `__util$getmsg()`; её текст - **управляющая строка FAO**, форматируемая `__util$faol()` (глава 3). Оболочка имеет форму OpenVMS:

```
%FACILITY-S-MNEMONIC, text        if the record has a mnemonic
%FACILITY-S-text                  if it has not
```

`S` - буква серьёзности *sts* (а не серьёзности, хранимой в записи). Имя средства - *facname* каталога или `NONAME`. Если запись не найдена, пишется текст по умолчанию из `__util$putmsg()`. Текст обрезается по буферу (2047 октетов). Меньше параметров, чем директив в управляющей строке, безопасно: форматирование останавливается там, где кончаются параметры.

Если форматирование текста не удалось (неверная директива, мало параметров, обрезанное поле или вывод), запись всё равно пишется, а после текста стоит метка `[FAO BADDIR]`, `[FAO NOPRM]`, `[FAO FLDOVF]`, `[FAO BUFOVF]` или `[FAO 0x...]`, как делает `__util$log_fao()`; возвращаемое значение не меняется. То же верно для `__util$putmsgd_fao()`.

Вызывайте её через `$PUTMSG_FAO()`, который приводит параметры к типу и подсчитывает их.

Возвращаемые условные значения

Эта рутина возвращает свой аргумент *sts*.

__util$putmsgd_fao – Сигнализация сообщения (FAO) с местом вызова

Формат

```
unsigned    __util$putmsgd_fao  (unsigned sts, const char *__mod, const char *__fi,
↪  unsigned __li, int prmcnt, ...);
```

Возвращает

`unsigned`: аргумент *sts*.

Аргументы

- *sts* – `unsigned`; только чтение; по значению. Условное значение.
- *__mod* – `const char *`; только чтение; по ссылке. Имя модуля; может быть нулевым.
- *__fi* – `const char *`; только чтение; по ссылке. Имя рутины.
- *__li* – `unsigned`; только чтение; по значению. Номер строки.
- *prmcnt* – `int`; только чтение; по значению. Число параметров FAO, от 0 до 16.
- *...* – `fao_prm_t`, по значению. Параметры FAO.

Описание

Как `__util$putmsg_fao()`, но между префиксом и оболочкой стоит `[module\routine:line]` (или `[routine:line]`). `$PUTMSG_FAO()` вызывает эту рутину в отладочной сборке.

Возвращаемые условные значения

Эта рутина возвращает свой аргумент *sts*.

$PUTMSG – Сигнализация сообщения (макрос)

Формат

```
#define $PUTMSG(sts, ...)   __util$putmsgd(sts, __MODULE__, __FUNCTION__ , __LINE__ , ##
 ↪  __VA_ARGS__)   /* debug   */
#define $PUTMSG(sts, ...)   __util$putmsg(sts, ## __VA_ARGS__)                 /* NDEBUG
 ↪  */
```

Возвращает

`unsigned`: *sts*.

Аргументы

- *sts* – `unsigned`; только чтение; по значению. Условное значение.
- *...* – аргументы формата printf текста сообщения.

Описание

Сигнализирует сообщение каталога с текстом в формате printf. Без `NDEBUG` запись показывает место вызова.

Пример

```
return  $PUTMSG(MYAPP$_OPENFAIL, l_path, errno);    /* for a catalogue whose texts are
 ↪  printf formats */
```

$PUTMSG_FAO – Сигнализация сообщения, FAO (макрос)

Формат

```
#define $PUTMSG_FAO(sts, ...)   __util$putmsgd_fao(sts, __MODULE__, __FUNCTION__ ,
 ↪  __LINE__ ,   /* debug  */
                FAO$_NARG(__VA_ARGS__) FAO$_EACH(__VA_ARGS__))
#define $PUTMSG_FAO(sts, ...)   __util$putmsg_fao(sts, FAO$_NARG(__VA_ARGS__)
 ↪  FAO$_EACH(__VA_ARGS__))   /* NDEBUG */
```

Возвращает

`unsigned`: *sts*.

Аргументы

- *sts* – `unsigned`; только чтение; по значению. Условное значение.
- *…* – от 0 до 16 значений для директив FAO. Пишутся как есть.

Описание

Макрос подсчитывает значения через `FAO$_NARG()` и приводит каждое к `fao_prm_t` через `FAO$_EACH()`, так что вызывающий пишет значения, а не счётчик и не приведения типов. Вспомогательные макросы `FAO$_C`, `FAO$_E0` … `FAO$_E16`, `FAO$_GET`, `FAO$_NARG` и `FAO$_EACH` входят в этот механизм и напрямую не вызываются. Больше 16 значений не компилируется. Указатель, переданный для `!AZ`, преобразуется к `fao_prm_t` приведением. Включайте `starlet.h`.

Пример

```
#include "starlet.h"

$PUTMSG_FAO(MYAPP$_OPENFAIL, "/etc/myapp.conf", (long) errno);
```

2.2 Журнал, трассировка и вывод

Все рутины этого раздела пишут в **дескриптор журнала**. При запуске программы это стандартный вывод. `__util$deflog()` направляет его в файл, `__util$setlogfd()` - в уже открытый дескриптор.

Запись. Каждая запись, кроме записи `__util$out()`, начинается с одного и того же префикса:

```
DD-MM-YYYY HH:MM:SS.fff TTTTTT
```

Дата и время местные. *fff* - доля секунды в настоящих миллисекундах, от 000 до 999, на всех платформах (в 1.6.11 это были наносекунды, делённые на 2^20, и значения шли от 000 до 953). Время берётся из грубых часов системы (`CLOCK_REALTIME_COARSE` в Linux), разрешение которых - несколько миллисекунд. *TTTTTT* - номер потока (`gettid()` в Linux), выровненный вправо до шести позиций; он запрашивается один раз для каждого потока. После `fork()` потомок по-прежнему показывает номер родителя.

Затем рутины добавляют, в таком порядке, место вызова (только отладочные формы), оболочку `%FACILITY-S:` и текст:

Рутина	Запись после префикса
`__util$log()`, `__util$log_fao()`	`%FAC-S: text`
`__util$logd()`, `__util$logd_fao()`	`[module\routine:line] %FAC-S:  text` (два пробела)
`__util$trace()`	`[module\routine:line]` с дополнением пробелами до позиции 72, затем текст
`__util$dumphex()`	`[routine:line] Dump of N octets follows:` и строки дампа

Запись кончается переводом строки и пишется одним `write()`. Запись ограничена 2047 октетами (1023 для `__util$trace()`); более длинная обрезается. Ошибка `write()` игнорируется.

__util$log – Запись в журнал (формат printf)

Формат

```
unsigned    __util$log  (const char *fac, unsigned severity, const char *fmt, ...);
```

Возвращает

`unsigned`: серьёзность.

Аргументы

- *fac* – `const char *`; только чтение; по ссылке. Имя средства для оболочки.
- *severity* – `unsigned`; только чтение; по значению. `STS$K_*`, возможно с добавлением по ИЛИ `STS$M_SYSLOG`.
- *fmt* – `const char *`; только чтение; по ссылке. Формат printf.
- *...* – аргументы *fmt*.

Описание

Пишет `prefix %fac-S: text`. Буква *S* - буква для `$SEV(severity)`.

Сначала снимается `STS$M_SYSLOG`. Затем серьёзность выше `STS$K_FATAL` (4) недопустима: показывается буква `?`, а возвращается `STS$K_ERROR` (2, младший бит сброшен): неопределённая серьёзность – ошибка вызывающего и не должна читаться как успех. До 1.7.0 возвращалось `STS$K_UNDEF` (5), у которого младший бит установлен. До 1.6.11 сам флаг делал серьёзность «вне диапазона», так что запись с `STS$M_SYSLOG` возвращала 5 при любой серьёзности.

Если к *severity* добавлен по ИЛИ `STS$M_SYSLOG`, текст дополнительно отправляется через `syslog()` библиотеки Си. Приоритет - `LOG_INFO` для серьёзности с установленным младшим битом (успех, информация) и `LOG_ERR` для остальных. В Windows недоступно.

`$LOG()` вызывает эту рутину в сборке выпуска.

Возвращаемые условные значения

Рутина возвращает записанную серьёзность со снятым `STS$M_SYSLOG`, так что `return $LOG(STS$K_ERROR, ...)` возвращает ошибку, как и `return $LOG(STS$K_ERROR | STS$M_SYSLOG, ...)`.

Значение	Смысл
`STS$K_WARN`, `STS$K_SUCCESS`, `STS$K_ERROR`, `STS$K_INFO`, `STS$K_FATAL`	Заданная серьёзность
`STS$K_ERROR` (2)	Серьёзность не определена (5, 6 или 7 в младших битах); в записи показано `?`

Пример

```
__util$log("MYAPP", STS$K_INFO, "started, pid = %d", (int) getpid());
```

__util$logd – Запись в журнал с местом вызова

Формат

```
unsigned    __util$logd (const char *fac, unsigned severity, const char *fmt, const char
↪  *mod, const char *__func, unsigned __line, ...);
```

Возвращает

`unsigned`: записанная серьёзность (`STS$M_SYSLOG` сброшен; `STS$K_ERROR` для неопределённой, 5-7).

Аргументы

- *fac* – `const char *`; только чтение; по ссылке. Имя средства.
- *severity* – `unsigned`; только чтение; по значению. `STS$K_*`.
- *fmt* – `const char *`; только чтение; по ссылке. Формат printf.
- *mod* – `const char *`; только чтение; по ссылке. Имя модуля; может быть нулевым, и тогда запись показывает `[routine:line]`.
- *__func* – `const char *`; только чтение; по ссылке. Имя рутины.
- *__line* – `unsigned`; только чтение; по значению. Номер строки.
- *...* – аргументы *fmt*.

Описание

Пишет `prefix [mod\func:line] %fac-S:  text` (`prefix [func:line] ...` для нулевого *mod*). `$LOG()` вызывает эту рутину в отладочной сборке. Серьёзность обрабатывается так же, как в `__util$log()`, включая `STS$M_SYSLOG`: запись уходит и в системный журнал, и в отладочной сборке, и в сборке выпуска (1.6.11 этого не делала).

Возвращаемые условные значения

Записанная серьёзность, как её возвращает `__util$log()`: `STS$M_SYSLOG` в неё не входит, а значение за пределами `STS$K_FATAL` есть `STS$K_ERROR` (до 1.7.0 – `STS$K_UNDEF`).

Пример

```
#define __MODULE__ "CONF$"
...
$LOG(STS$K_ERROR, "no file '%s'", l_path);  /* 31-10-2026 12:00:00.123 4711
↪  [CONF$\main:42] %UTILS-E:  no file 'x' */
```

__util$log_fao – Запись в журнал (управляющая строка FAO)

Формат

```
unsigned    __util$log_fao  (const char *a_fac, unsigned a_sev, const char *a_ctl, int
↪  a_prmcnt, ...);
```

Возвращает

`unsigned`: серьёзность.

Аргументы

- *a_fac* – `const char *`; только чтение; по ссылке. Имя средства.
- *a_sev* – `unsigned`; только чтение; по значению. `STS$K_*`, возможно с добавлением по ИЛИ `STS$M_SYSLOG`.
- *a_ctl* – `const char *`; только чтение; по ссылке. Управляющая строка FAO.
- *a_prmcnt* – `int`; только чтение; по значению. Число параметров FAO, от 0 до 16 (большее принимается за 16, меньшее 0 - за 0).
- *...* – `fao_prm_t`, по значению. Параметры.

Описание

Близнец `__util$log()` для FAO: пишет `prefix %fac-S: text`, причём текст форматируется `__util$faol()`. Проверка диапазона серьёзности и правило `STS$M_SYSLOG` те же, что у `__util$log()`, включая возвращаемое значение; в `syslog()` попадает уже отформатированное тело. Вызывайте через `$LOG_FAO()`. Раскрытие директив описано в главе 3.

Если `__util$faol()` вернула статус со сброшенным младшим битом (неверная директива, мало параметров, поле или вывод обрезаны), запись всё равно пишется, а после текста стоит метка `[FAO BADDIR]`, `[FAO NOPRM]`, `[FAO FLDOVF]` или `[FAO BUFOVF]`, либо `[FAO 0x...]` с шестнадцатеричным статусом для другого. Текст, заполнивший запись, обрезается, чтобы оставить место метке. Возвращаемое значение не меняется, так что `return $LOG_FAO(STS$K_ERROR, ...)` по-прежнему возвращает серьёзность. (До 1.7.0 статус отбрасывался.)

Возвращаемые условные значения

Как у `__util$log()`.

__util$logd_fao – Запись в журнал (FAO) с местом вызова

Формат

```
unsigned    __util$logd_fao (const char *a_fac, unsigned a_sev, const char *a_ctl, const
 ↪  char *a_mod, const char *a_func, unsigned a_line, int a_prmcnt, ...);
```

Возвращает

`unsigned`: записанная серьёзность (`STS$M_SYSLOG` сброшен; `STS$K_ERROR` для неопределённой, 5-7).

Аргументы

- *a_fac* – `const char *`; только чтение; по ссылке. Имя средства.
- *a_sev* – `unsigned`; только чтение; по значению. `STS$K_*`.
- *a_ctl* – `const char *`; только чтение; по ссылке. Управляющая строка FAO.
- *a_mod* – `const char *`; только чтение; по ссылке. Имя модуля; может быть нулевым (см. `__util$logd()`).
- *a_func* – `const char *`; только чтение; по ссылке. Имя рутины.
- *a_line* – `unsigned`; только чтение; по значению. Номер строки.
- *a_prmcnt* – `int`; только чтение; по значению. Число параметров FAO, от 0 до 16.
- *...* – `fao_prm_t`, по значению. Параметры.

Описание

Близнец `__util$logd()` для FAO: `prefix [mod\func:line] %fac-S:  text`. `$LOG_FAO()` вызывает её в отладочной сборке. `STS$M_SYSLOG` учитывается, как в `__util$log_fao()`.

Возвращаемые условные значения

Как у `__util$logd()`.

$LOG – Запись в журнал (макрос)

Формат

```
#define $LOG(severity, fmt, ...)    __util$logd(__FAC__, severity, fmt, __MODULE__,
 ↪  __FUNCTION__ , __LINE__ , ## __VA_ARGS__)   /* debug  */
#define $LOG(severity, fmt, ...)    __util$log(__FAC__, severity, fmt, ## __VA_ARGS__)
 ↪  /* NDEBUG */
```

Возвращает

`unsigned`: серьёзность (см. вызываемую рутину).

Аргументы

- *severity* – `STS$K_*`; только чтение; по значению.
- *fmt* – строковый литерал или `const char *`; только чтение; по ссылке. Формат printf.
- *...* – аргументы *fmt*.

Описание

Обычный способ писать в журнал. Средство - `__FAC__`; модуль - `__MODULE__`. Определите `__MODULE__` как строку до включения заголовка: отладочная сборка с умолчательным `NULL` показывает только рутину и строку, `[routine:line]`.

Пример

```
if ( 0 > (l_fd = open(l_path, O_RDONLY)) )
    return  $LOG(STS$K_ERROR, "open('%s') -> errno %d", l_path, errno);
```

$LOG_FAO – Запись в журнал, FAO (макрос)

Формат

```
#define $LOG_FAO(severity, ctl, ...)    __util$logd_fao(__FAC__, severity, ctl,
 ↪  __MODULE__, __FUNCTION__ , __LINE__ ,   /* debug  */
                    FAO$_NARG(__VA_ARGS__) FAO$_EACH(__VA_ARGS__))
#define $LOG_FAO(severity, ctl, ...)    __util$log_fao(__FAC__, severity, ctl,
 ↪  FAO$_NARG(__VA_ARGS__) FAO$_EACH(__VA_ARGS__))   /* NDEBUG */
```

Возвращает

`unsigned`: серьёзность.

Аргументы

- *severity* – `STS$K_*`; только чтение; по значению.
- *ctl* – строка; только чтение; по ссылке. Управляющая строка FAO.
- *...* – от 0 до 16 значений для директив, пишутся как есть.

Описание

Как `$PUTMSG_FAO()`, но для журнала: макрос подсчитывает значения и приводит каждое к `fao_prm_t`. Включайте `starlet.h`.

Пример

```
#include "starlet.h"

$LOG_FAO(STS$K_INFO, "!AZ: !UL octets in !UL blocks", l_name, (unsigned long) l_size,
 ↪  (unsigned long) l_blocks);
```

$IFLOG – Запись в журнал, если флаг установлен (макрос)

Формат

```
#define $IFLOG(f, severity, fmt, ...)   f ? __util$logd(__FAC__, severity, fmt,
↪  __MODULE__, __FUNCTION__ , __LINE__ , ## __VA_ARGS__) : severity
```

Возвращает

Значение выражения: результат `__util$logd()`, если *f* истинно, и *severity*, если нет.

Аргументы

- *f* – целочисленное выражение; только чтение; по значению. Флаг.
- *severity*, *fmt*, *...* – как у `$LOG()`.

Описание

Пишет в журнал, только если *f* истинно. Макрос не заключён в скобки: берите в скобки всё использование, если оно часть большего выражения (приведение вида `(void) $IFLOG(...)` относится только к *f*), и берите в скобки *f*, если это не простое имя. Макрос всегда вызывает `__util$logd()`, и в отладочной сборке, и в сборке выпуска, так что ***MODULE*** используется в обеих.

Пример

```
$IFLOG(g_verbose, STS$K_INFO, "queue length %u", l_count);
```

__util$log2buf – Форматирование записи журнала в буфер

Формат

```
unsigned    __util$log2buf  (void *out, int outsz, int * outlen, const char *fac,
↪  unsigned severity, const char *fmt, ...);
```

Возвращает

`unsigned`: записанная серьёзность, как её возвращает `__util$log()`.

Аргументы

- *out* – `void *`; только запись; по ссылке. Буфер.
- *outsz* – `int`; только чтение; по значению. Его размер в октетах.
- *outlen* – `int *`; только запись; по ссылке. Получает длину записи без завершающего нуля.
- *fac* – `const char *`; только чтение; по ссылке. Имя средства.
- *severity* – `unsigned`; только чтение; по значению. `STS$K_*`.
- *fmt* – `const char *`; только чтение; по ссылке. Формат printf.
- *...* – аргументы *fmt*.

Описание

Строит `prefix %fac-S:text` в буфере вызывающего так, как `__util$log()` записал бы её, но без пробела после двоеточия, без перевода строки и с нулевым октетом в конце. В журнал ничего не пишется. Запись обрезается по размеру: не более *outsz* - 1 октетов, затем нуль. `STS$M_SYSLOG` игнорируется (в системный журнал ничего не отправляется), а серьёзность вне диапазона от 0 до 4 показывается как `?`. Если *outsz* меньше 1, буфер не затрагивается и *outlen* устанавливается в 0. *outlen* не должен быть нулевым.

Возвращаемые условные значения

Рутина возвращает записанную серьёзность (`STS$M_SYSLOG` снят, `STS$K_ERROR` для неопределённой, 5-7); собственного условного значения она не сигнализирует.

Пример

```
char l_msg [256];
int  l_len;

__util$log2buf(l_msg, sizeof(l_msg), &l_len, "MYAPP", STS$K_WARN, "disk %d is full", 3);
```

__util$out – Запись строки в дескриптор журнала

Формат

```
unsigned    __util$out  (char *fmt, ...);
```

Возвращает

`unsigned`: `STS$K_SUCCESS`.

Аргументы

- *fmt* – `char *`; только чтение; по ссылке. Формат printf.
- *...* – аргументы *fmt*.

Описание

Пишет отформатированный текст и перевод строки, без префикса и без оболочки: для заставки, таблицы, текста подсказки. Строка обрезается на 1023 октетах. Результат `write()` не проверяется.

Возвращаемые условные значения

Значение	Смысл
`STS$K_SUCCESS`	Всегда

Пример

```
__util$out("MYAPP version %s", "1.0");
```

__util$deflog – Направление журнала в файл и на хост syslog

Формат

```
int __util$deflog   (const char *, const char *);
```

Возвращает

`int`: условное значение.

Аргументы

- *logfile* – `const char *`; только чтение; по ссылке. Путь к файлу журнала либо нуль или пустая строка, чтобы оставить журнал где он есть.
- *sloghost* – `const char *`; только чтение; по ссылке. Адрес IPv4 сервера syslog в точечной форме либо нуль.

Описание

Если *sloghost* не нуль, адрес запоминается для `__util$syslog()` с портом 514. Адрес должен быть адресом IPv4 в точечной форме. Всё иное (имя хоста, адрес IPv6) заносится в журнал как ошибка и игнорируется: хост syslog остаётся прежним, а без хоста рутина syslog ничего не делает. В 1.6.11 плохой адрес запоминался как нули. **Рутина лишь запоминает хост.** Рутины журнала ничего ему не отправляют, включая `STS$M_SYSLOG` (этот флаг вызывает локальный `syslog()` библиотеки C); запись попадает на хост только через `__util$syslog()`.

Если *logfile* не нуль и не пуст, файл открывается на дозапись (создаётся с режимом 0644, если его нет; дочерними процессами не наследуется). Старый файл предыдущего вызова закрывается. Новый дескриптор становится дескриптором журнала, а **стандартный вывод и стандартный вывод ошибок перенаправляются в тот же файл** через `dup2()`. Затем пишется запись “Log file ‘…’ has been opened.”. Имя файла запоминается для `__util$rotatelogfile()`; имя в 1024 октета и более не запоминается и отключает ротацию.

Если *logfile* нуль или пуст, вызов меняет только хост и возвращает `STS$K_SUCCESS`. Вызывайте рутину один раз при запуске, до того как какой-либо поток пишет в журнал. Программа, чей стандартный вывод несёт данные, использует вместо неё `__util$setlogfd()`.

Возвращаемые условные значения

Значение	Смысл
`STS$K_SUCCESS` (1)	Файл не запрашивался либо файл открыт и запись сделана
`STS$K_ERROR` (2)	Файл открыть не удалось; журнал остаётся где был, и туда пишется запись с `errno`

Пример

```
if ( !(1 & __util$deflog("/var/log/myapp.log", NULL)) )
    return  STS$K_ERROR;
```

__util$setlogfd – Направление журнала в открытый дескриптор

Формат

```
int __util$setlogfd (int);
```

Возвращает

`int`: условное значение.

Аргументы

- *a_fd* – `int`; только чтение; по значению. Открытый дескриптор.

Описание

Делает *a_fd* дескриптором журнала. Никакой файл не открывается, стандартные потоки не затрагиваются, и дескриптор впоследствии не закрывается. После этого вызова журнал не наш файл, поэтому `__util$rotatelogfile()` ничего не делает. Типичное применение: `__util$setlogfd(STDERR_FILENO)` в фильтре, чей стандартный вывод несёт данные.

Возвращаемые условные значения

Значение	Смысл
`STS$K_SUCCESS`	Сделано
`STS$K_ERROR`	*a_fd* отрицателен; журнал не изменён

Пример

```
__util$setlogfd(STDERR_FILENO);
```

__util$rewindlogfile – Усечение файла журнала за пределом

Формат

```
int __util$rewindlogfile    (size_t);
```

Возвращает

`int`: условное значение.

Аргументы

- *limit* – `size_t`; только чтение; по значению. Размер в октетах, за которым файл усекается. Нуль отключает.

Описание

Если дескриптор журнала не стандартный вывод и *limit* не нуль, текущая позиция дескриптора сравнивается с *limit*. Достигнув его или перейдя, файл усекается до нуля и перематывается. Старые записи теряются. Позиция читается как `int`: рутина рассчитана на пределы меньше 2 ГБ. В Windows ничего не делается (позиция перематывается, но файл не усекается). Для файла, который должен сохранять свою историю, лучше `__util$rotatelogfile()`.

Возвращаемые условные значения

Значение	Смысл
`STS$K_SUCCESS`	Сделано либо нечего делать
`STS$K_ERROR`	Позицию дескриптора прочитать не удалось (например, это канал)

__util$rotatelogfile – Ротация файла журнала за пределом

Формат

```
int __util$rotatelogfile    (size_t, unsigned);
```

Возвращает

`int`: условное значение.

Аргументы

- *a_limit* – `size_t`; только чтение; по значению. Размер в октетах, за которым файл ротируется. Нуль: никогда.
- *a_keep* – `unsigned`; только чтение; по значению. Сколько старых файлов хранить, от 0 до 99.

Описание

Вызывайте её время от времени (достаточно раз в несколько секунд) в долгоработающей программе. Если журнал открыт `__util$deflog()` и вырос до *a_limit* октетов, файл переименовывается в `<file>.1`, более старые в `<file>.2` … `<file>.<a_keep>` (самый старый отбрасывается), а новый `<file>` открывается на том же самом дескрипторе журнала, вместе со стандартным выводом и стандартным выводом ошибок. Никакие данные не копируются. Запись, сделанная другим потоком в этот момент, попадает целиком либо в старый файл, либо в новый. Ротацию делает только один поток; поток, обнаруживший ротацию в ходе, сразу возвращает `STS$K_SUCCESS`.

При *a_keep* = 0 файл усекается на месте, и размер проверяется при каждом вызове.

Рутина ничего не делает и возвращает `STS$K_SUCCESS`, если *a_limit* равен нулю, если журнал не файл, открытый `__util$deflog()` (это стандартный вывод либо дескриптор, установленный `__util$setlogfd()`), либо если файл ещё не достиг предела. После успешной ротации пишется запись “Log file … has been rotated” или “… truncated”.

Возвращаемые условные значения

Значение	Смысл
`STS$K_SUCCESS`	Нечего делать либо ротацию делает другой поток
`STS$K_INFO` (3)	Файл ротирован или усечён; успех (младший бит установлен)
`STS$K_ERROR`	*a_keep* больше 99, либо `fstat()`, `rename()`, `open()` или `dup2()` не удались; журнал продолжает идти в имевшийся у него файл

Пример

```
for ( ;; )
    {
    __util$rotatelogfile(10 * 1024 * 1024, 5);  /* 10 MB, keep 5 */
    sleep(5);
    }
```

__util$syslog – Отправка сообщения на хост syslog

Формат

```
unsigned    __util$syslog   (int fac, int sev, const char *tag, const char *msg, int
↪  msglen);
```

Возвращает

`unsigned`: условное значение.

Аргументы

- *fac* – `int`; только чтение; по значению. Код средства syslog (`LOG_USER` и так далее).
- *sev* – `int`; только чтение; по значению. Код серьёзности syslog (`LOG_DEBUG` и так далее).
- *tag* – `const char *`; только чтение; по ссылке. Метка, как имя программы.
- *msg* – `const char *`; только чтение; по ссылке. Текст.
- *msglen* – `int`; только чтение; по значению. Его длина.

Описание

Отправляет одну датаграмму UDP, в форме RFC 3164, на хост, заданный `__util$deflog()`, порт 514: `<fac+sev> tag: text`. Метка дополняется до 16 позиций. Сокет создаётся при первом вызове и сохраняется (в Windows тогда же запускается Winsock). Сокет создаётся один раз, под защитой, так что первый вызов может прийти из нескольких потоков одновременно. Рутины журнала эту рутину не вызывают; вызывайте её сами.

Если `__util$deflog()` не получила хост syslog, отправлять некуда, и рутина ничего не делает и возвращает `STS$K_SUCCESS`. Она ничего не пишет в стандартный вывод ошибок, не вызывает ни `perror()`, ни `abort()` и никогда не пишет собственного сообщения, поскольку её могла вызвать сама запись в журнал. Неудавшееся создание сокета повторяется при следующем вызове.

Возвращаемые условные значения

Значение	Смысл
`STS$K_SUCCESS`	Датаграмма отправлена либо хост не задан и делать нечего
`STS$K_ERROR`	Сокет создать не удалось либо `sendmsg()` не удался

Пример

```
__util$deflog(NULL, "192.168.1.10");
__util$syslog(LOG_USER, LOG_INFO, "myapp", "started", 7);
```

__util$trace – Запись трассировки

Формат

```
void    __util$trace    (int cond, const char *fmt, const char *mod, const char *func,
↪  unsigned line, ...);
```

Возвращает

Ничего (`void`).

Аргументы

- *cond* – `int`; только чтение; по значению. Запись делается, только если это не нуль.
- *fmt* – `const char *`; только чтение; по ссылке. Формат printf.
- *mod* – `const char *`; только чтение; по ссылке. Имя модуля; нуль допустим.
- *func* – `const char *`; только чтение; по ссылке. Имя рутины.
- *line* – `unsigned`; только чтение; по значению. Номер строки.
- *...* – аргументы *fmt*.

Описание

Пишет `prefix [mod\func:line]`, затем дополняет пробелами до позиции 72, чтобы тексты записей выстраивались в столбец, затем текст. При нулевом *mod* форма `[func:line]`. Если *mod* не нуль, модуль и рутина разделены одной обратной косой чертой, `[mod\func:line]`, как в других рутинах (1.6.11 писала две). Запись обрезается на 1023 октетах. Обычно вызывается через `$TRACE()` и `$IFTRACE()`.

Возвращаемые условные значения

Нет; рутина не возвращает значения.

$TRACE, $IFTRACE – Трассировка (макросы)

Формат

```
#define $TRACE(fmt, ...)        __util$trace(1, fmt, __MODULE__, __FUNCTION__, __LINE__ ,
 ↪  ## __VA_ARGS__)   /* debug  */
#define $IFTRACE(cond, fmt, ...)    __util$trace(cond, fmt, __MODULE__, __FUNCTION__,
 ↪  __LINE__ , ## __VA_ARGS__)    /* debug  */
#define $TRACE(fmt, ...)        __util$nope()                               /* NDEBUG */
#define $IFTRACE(cond, fmt, ...)    ((void) sizeof(cond), __util$nope())
 ↪  /* NDEBUG */
```

Возвращает

`void`.

Аргументы

- *cond* – целочисленное выражение. Трассировка пишется, если оно не нуль.
- *fmt, ...* – формат printf и его аргументы.

Описание

Трассировка предназначена для отладки. В сборке с `NDEBUG` оба макроса компилируются в ничто, а *cond* названо в `sizeof`, чтобы переменная, используемая лишь для трассировки, не считалась неиспользуемой; оно не вычисляется. Макросы не переопределяются, если программа определила их раньше.

Пример

```
$TRACE("entering, count = %u", l_count);
$IFTRACE(g_trace, "state %d", l_state);
```

__util$dumphex – Запись шестнадцатеричного дампа

Формат

```
void    __util$dumphex  (const char *__fi, unsigned __li, const void *src, unsigned short
↪  srclen);
```

Возвращает

Ничего.

Аргументы

- *__fi* – `const char *`; только чтение; по ссылке. Имя рутины.
- *__li* – `unsigned`; только чтение; по значению. Номер строки.
- *src* – `const void *`; только чтение; по ссылке. Данные.
- *srclen* – `unsigned short`; только чтение; по значению. Число октетов, до 65535.

Описание

Пишет заголовочную запись `prefix [fi:li] Dump of N octets follows:`, затем данные по 16 октетов в строке, каждая строка в 80 октетов вместе с переводом строки:

```
\t+0010:  58 59 5a ...                                          XYZ
```

Строка содержит смещение в шестнадцатеричном виде, 16 столбцов по две шестнадцатеричные цифры, два пробела и те же октеты как символы, точку для непечатаемых. Последняя строка дополняется пробелами.

Возвращаемые условные значения

Нет; рутина не возвращает значения.

$DUMPHEX – Запись шестнадцатеричного дампа (макрос)

Формат

```
#define $DUMPHEX(s,l)   __util$dumphex(__FUNCTION__, __LINE__ , s, l)
```

Возвращает

`void`.

Аргументы

- *s* – адрес; только чтение; по ссылке. Данные.
- *l* – целое; только чтение; по значению. Число октетов.

Описание

Вызывает `__util$dumphex()` с именем текущей рутины и строкой.

Пример

```
unsigned char l_hdr [24];

$DUMPHEX(l_hdr, sizeof(l_hdr));
```

2.3 Счётные строки и работа со строками

ASC, ASC$K_SZ – Тип счётной строки

Формат

```
#define ASC$K_SZ    255

typedef struct _asc
{
    unsigned char   len;
    char    sts[ASC$K_SZ];
} ASC;
```

Описание

Счётная строка (counted string): *len* - число октетов текста, от 0 до 255, а *sts* содержит текст. Текст не обязан кончаться нулевым октетом и может его содержать. Рутины, создающие `ASC` из строки Си, также записывают нулевой октет после текста, чтобы *sts* можно было передать функции Си; для строки в полные 255 октетов места для него нет (см. `__util$str2asc()`). `__ASC_TYPE__` определяется заголовком, чтобы показать, что тип определён здесь.

$ASCDECL, $ASCINI, $ASCLEN, $ASCPTR, $ASC, $ASCNIL – Макросы счётных строк

Формат

```
#define $ASCDECL(a,sts) ASC a = {(unsigned char)sizeof(sts)-1,(sts)}
#define $ASCINI(a)  ((unsigned char) sizeof(a))-1,(a)
#define $ASCLEN(a)  (((ASC *) a)->len)
#define $ASCPTR(a)  (((ASC *) a)->sts)
#define $ASC(a)     (((ASC *) a)->len),(((ASC *) a)->sts)
#define $ASCNIL     0,0
```

Аргументы

- *a* – для `$ASCDECL()` имя объявляемой переменной; для `$ASCINI()` строковый литерал; для `$ASCLEN()`, `$ASCPTR()` и `$ASC()` указатель на `ASC`.
- *sts* – только `$ASCDECL()`: строковый литерал.

Описание

`$ASCDECL(name, "text")` объявляет и инициализирует `ASC` из строкового литерала. `$ASCINI("text")` раскрывается в первые два инициализатора `ASC`, длину и текст, для использования внутри фигурных скобок (например, в таблице `OPTS` или `KWDENT`). `$ASCLEN(p)` и `$ASCPTR(p)` дают длину и текст `ASC`, на который указывает *p*; им можно присваивать. `$ASC(p)` раскрывается в два значения, длину и адрес, для преобразования `"%.*s"`. `$ASCNIL` - два нуля, инициализаторы пустого `ASC`.

Макросы принимают *указатель*. Для переменной пишите `$ASCLEN(&l_name)`.

Пример

```
$ASCDECL(l_cmd, "SHOW");
ASC l_copy;

__util$cpyasc(&l_cmd, &l_copy);
$LOG(STS$K_INFO, "command '%.*s' (%u octets)", $ASC(&l_copy), $ASCLEN(&l_copy));
```

__util$str2asc – Копирование строки Си в счётную строку

Формат

```
inline static int   __util$str2asc  (const char * a_src, void * a_dst);
```

Возвращает

`int`: длина записанного текста, не более 254.

Аргументы

- *a_src* – `const char *`; только чтение; по ссылке. Строка, завершённая нулём.
- *a_dst* – `void *`; только запись; по ссылке. Адрес `ASC`.

Описание

Копирует *a_src* в текст `ASC`, устанавливает его длину и записывает нулевой октет после текста. `ASC` вмещает `ASC$K_SZ` (255) октетов, а нулевому октету нужен один из них, поэтому более длинный текст обрезается на 254 октетах, и возвращаемая длина равна 254. За пределы `ASC` ничего не записывается (в 1.6.11 текст в 255 октетов и более ставил нулевой октет на один октет за ним).

Возвращаемые условные значения

Нет; рутина возвращает длину.

Пример

```
ASC l_name;

__util$str2asc("config.sys", &l_name);
```

__util$asc2str – Копирование счётной строки в строку Си

Формат

```
inline static int   __util$asc2str  (ASC * src, char * dst);
```

Возвращает

`int`: длина текста.

Аргументы

- *src* – `ASC *`; только чтение; по ссылке. Счётная строка.
- *dst* – `char *`; только запись; по ссылке. Буфер; он должен вмещать длину плюс один.

Описание

Копирует текст *src* в *dst* и записывает после него нулевой октет.

Возвращаемые условные значения

Нет; рутина возвращает длину.

__util$cpyasc – Копирование счётной строки

Формат

```
inline static int   __util$cpyasc   (const ASC * a_src, const ASC * a_dst);
```

Возвращает

`int`: скопированная длина.

Аргументы

- *a_src* – `const ASC *`; только чтение; по ссылке. Источник.
- *a_dst* – `const ASC *`; только запись; по ссылке. Приёмник. Он объявлен `const`, но всё равно записывается: передавайте адрес неконстантного `ASC`.

Описание

Копирует длину и текст через `memmove()`, так что области могут перекрываться. Нулевой октет после текста не записывается.

Возвращаемые условные значения

Нет; рутина возвращает длину.

__util$cmpasc – Сравнение счётных строк

Формат

```
inline static int   __util$cmpasc   (ASC * s1, ASC * s2);
```

Возвращает

`int`: нуль, если строки равны; иначе не нуль.

Аргументы

- *s1*, *s2* – `ASC *`; только чтение; по ссылке. Строки.

Описание

Если длины различаются, результат - длина *s1* минус длина *s2*. Если они равны, результат - значение `memcmp()` над текстом. Поэтому порядок определяется сначала длиной и **не** является алфавитным. Используйте рутину для проверки на равенство.

Возвращаемые условные значения

Нет; рутина возвращает результат сравнения.

__util$cmpasc_blind – Сравнение счётных строк без учёта регистра

Формат

```
inline static int   __util$cmpasc_blind (const ASC * s1, const ASC * s2);
```

Возвращает

`int`: нуль, если строки равны без учёта регистра; иначе не нуль.

Аргументы

- *s1*, *s2* – `const ASC *`; только чтение; по ссылке. Строки.

Описание

Как `__util$cmpasc()`, но текст сравнивается через `strncasecmp()`. В Windows он сравнивается через `_stricmp()`, который читает до нулевого октета, поэтому в Windows текст должен быть завершён нулём.

Возвращаемые условные значения

Нет; рутина возвращает результат сравнения.

__util$strelem – Взятие элемента строки с разделителями

Формат

```
inline static int   __util$strelem  (const ASC *a_src, int a_separator, int a_idx, ASC
↪  *a_dst);
```

Возвращает

`int`: длина найденного элемента; 0, если его нет.

Аргументы

- *a_src* – `const ASC *`; только чтение; по ссылке. Строка.
- *a_separator* – `int`; только чтение; по значению. Октет-разделитель.
- *a_idx* – `int`; только чтение; по значению. Номер элемента, с 0.
- *a_dst* – `ASC *`; только запись; по ссылке. Получает элемент. Устанавливается всегда: пустой `ASC`, когда такого элемента нет.

Описание

Вырезает элемент номер *a_idx* из строки элементов, разделённых *a_separator*, и записывает его в *a_dst*. Рутина соответствует `F$ELEMENT` языка DCL. Она верна для любого индекса (1.6.11 была надёжна только для элементов 0 и 1 и могла прочитать за концом строки). Поиск никогда не выходит за пределы строки. За элементом следует нулевой октет, когда для него есть место. Отрицательный индекс или индекс за последним элементом даёт пустой *a_dst* и длину 0. Пустой элемент тоже имеет длину 0, так же как отсутствующий: в `"a,b,,d"` элемент 2 пуст, и по результату его не отличить от элемента 7.

Возвращаемые условные значения

Нет; рутина возвращает длину.

Пример

```
$ASCDECL(l_list, "alpha,beta");
ASC l_item;

if ( __util$strelem(&l_list, ',', 1, &l_item) )
    $LOG(STS$K_INFO, "second: %.*s", $ASC(&l_item));
```

__util$uint32_2_dec – Преобразование беззнакового целого в десятичный текст

Формат

```
inline  static int  __util$uint32_2_dec (unsigned int a_src, char *a_dst, size_t a_dstsz,
↪  size_t *a_retlen);
```

Возвращает

`int`: число записанных цифр.

Аргументы

- *a_src* – `unsigned int`; только чтение; по значению. Значение.
- *a_dst* – `char *`; только запись; по ссылке. Буфер.
- *a_dstsz* – `size_t`; только чтение; по значению. Размер буфера; не должен быть 0.
- *a_retlen* – `size_t *`; только запись; по ссылке. Получает длину; может быть нулевым.

Описание

Записывает десятичные цифры *a_src* и завершающий нуль. Значение 0 даёт “0”. Если цифры не помещаются, сохраняются старшие: 4000000000 в буфер из 8 октетов даёт “4000000”. При *a_dstsz*, равном 0, отладочная сборка останавливается на `assert()`.

Возвращаемые условные значения

Нет; рутина возвращает длину.

__util$trim – Удаление ведущих и замыкающих пробельных символов

Формат

```
inline static int   __util$trim (char *src, int srclen);
```

Возвращает

`int`: длина оставшегося текста; 0, если *src* нуль, *srclen* равно 0 или там были только пробельные символы.

Аргументы

- *src* – `char *`; изменение; по ссылке. Текст.
- *srclen* – `int`; только чтение; по значению. Его длина.

Описание

Удаляет пробельные символы (`isspace()`: пробел, табуляция, перевод строки, вертикальная табуляция, перевод формата, возврат каретки) с обоих концов текста на месте. Текст сдвигается к началу буфера, и после него записывается нулевой октет. Октеты передаются в `isspace()` как `char`: не используйте рутину для текста с октетами выше 0x7F на платформе, где `char` знаковый.

Возвращаемые условные значения

Нет; рутина возвращает длину.

Пример

```
char l_buf [] = "  value \n";
int  l_len = __util$trim(l_buf, (int) strlen(l_buf));   /* "value", 5 */
```

__util$collapse – Удаление всех пробельных символов

Формат

```
inline static int   __util$collapse (char *src, int srclen);
```

Возвращает

`int`: длина оставшегося текста; 0, если *src* нуль или *srclen* равно 0.

Аргументы

- *src* – `char *`; изменение; по ссылке. Текст.
- *srclen* – `int`; только чтение; по значению. Его длина.

Описание

Удаляет каждый октет, для которого `isspace()` истинно, смыкая промежутки. Если что-то было удалено, нулевой октет записывается там, где стоял последний удалённый, так что текст становится строкой Си. Если ничего не было удалено, места для нулевого октета нет и он не записывается: рутина никогда не пишет за пределы данных ей *srclen* октетов (в 1.6.11 она записывала нуль в *src*[*srclen*], и буфер должен был быть на октет длиннее).

Возвращаемые условные значения

Нет; рутина возвращает длину.

__util$uncomment – Отсечение комментария от строки

Формат

```
inline static int   __util$uncomment    (char *src, int srclen, char marker);
```

Возвращает

`int`: длина текста перед комментарием; *srclen*, если комментария не было; 0, если *src* нуль или *srclen* равно 0.

Аргументы

- *src* – `char *`; изменение; по ссылке. Строка.
- *srclen* – `int`; только чтение; по значению. Её длина.
- *marker* – `char`; только чтение; по значению. Октет, начинающий комментарий, например `'!'`.

Описание

Находит первый *marker*, не находящийся внутри заключённой в кавычки части, записывает туда нулевой октет и возвращает его смещение. Часть в двойных кавычках (`"..."`) или в одинарных (`'...'`) пропускается целиком: кавычка другого вида внутри неё - обычный октет, экранирования нет, а оставшаяся открытой кавычка тянется до конца строки, так что ничего не отсекается. (В 1.6.11 маркер внутри кавычек тоже начинал комментарий.) Строка, начинающаяся с маркера, даёт 0. Текст перед комментарием не усекается по краям пробелов; после неё вызывайте `__util$trim()`.

Возвращаемые условные значения

Нет; рутина возвращает длину.

Пример

```
char l_line [] = "-port=80   ! the port";

__util$uncomment(l_line, (int) strlen(l_line), '!');    /* "-port=80   " */
```

__util$strcat_rx – Конкатенация списка строк

Формат

```
inline static   int __util$strcat_rx    (void * dst, int dstsz, int * dstlen, ...);
```

Возвращает

`int`: условное значение.

Аргументы

- *dst* – `void *`; только запись; по ссылке. Область для результата.
- *dstsz* – `int`; только чтение; по значению. Размер области.
- *dstlen* – `int *`; только запись; по ссылке. Получает длину результата.
- *...* – пары аргументов: адрес `char *` и длина **`int`** (не `size_t`), оканчивающиеся нулевым адресом. Пара с длиной 0 пропускается.

Описание

Копирует куски один за другим в *dst* и записывает после них нулевой октет. Один октет области оставлен для нуля, поэтому куски могут заполнить её ровно до *dstsz* - 1 октетов, и это успех. Если кусок не помещается, копирование там останавливается: предыдущие куски сохраняются, *dstlen* содержит их длину, а текст завершён нулём. В *dst*[*dstsz*] никогда ничего не пишется. Отрицательная длина - кусок, который не помещается. Нулевые *dst* или *dstlen* или *dstsz* меньше 1 - ошибка (1.6.11 возвращала ошибку и писала за область, когда куски заполняли её ровно).

Возвращаемые условные значения

Значение	Смысл
`STS$K_SUCCESS`	Все куски скопированы
`STS$K_ERROR`	Кусок не поместился или аргумент неверен

Пример

```
char l_path [128];
int  l_len;

__util$strcat_rx(l_path, sizeof(l_path), &l_len, "/etc/", 5, "myapp", 5, ".conf", 5,
 ↪  (char *) NULL);
```

__util$strstr – Поиск счётной строки в счётной строке

Формат

```
char *  __util$strstr   (char *s1, size_t s1len, char *s2, size_t s2len);
```

Возвращает

`char *`: адрес первого вхождения; нуль, если его нет.

Аргументы

- *s1* – `char *`; только чтение; по ссылке. Область поиска.
- *s1len* – `size_t`; только чтение; по значению. Её длина.
- *s2* – `char *`; только чтение; по ссылке. Образец.
- *s2len* – `size_t`; только чтение; по значению. Его длина.

Описание

Как `strstr()`, но с длинами, так что ни одному операнду не нужен нулевой октет и любой может его содержать. Нулевой адрес, длина 0 или *s2len* больше *s1len* дают нуль. Сравнение точное, регистр существен.

Возвращаемые условные значения

Нет; рутина возвращает адрес или нуль.

Пример

```
char *l_p = __util$strstr(l_buf, l_buflen, CRLFCRLF, 4);    /* the end of an HTTP header
 ↪  */
```

CRLF, CRLFCRLF, CRLFCRLF_LW – Константы ограничителя строки

Формат

```
#define CRLFCRLF_LW 0x0a0d0a0d
#define CRLFCRLF    "\r\n\r\n"
#define CRLF        "\r\n"
```

Описание

Возврат каретки и перевод строки как строка; то же дважды как строка; то же дважды как longword в порядке байтов x86 или ARM в режиме little-endian, для проверки четырёх октетов одним сравнением.

2.4 Память

__util$movc5 – Перемещение символов с заполнением

Формат

```
static inline int   __util$movc5    (unsigned short *srclen, char *source, unsigned char
 ↪  *fill, unsigned short *dstlen, char *dest);
```

Возвращает

`int`: условное значение.

Аргументы

- *srclen* – `unsigned short *`; только чтение; по ссылке. Длина источника.
- *source* – `char *`; только чтение; по ссылке. Источник.
- *fill* – `unsigned char *`; только чтение; по ссылке. Октет заполнения; нулевой адрес означает нулевой октет.
- *dstlen* – `unsigned short *`; только чтение; по ссылке. Длина приёмника.
- *dest* – `char *`; только запись; по ссылке. Приёмник.

Описание

Команда MOVC5 архитектуры VAX как рутина, по образцу `LIB$MOVC5`. `min(*srclen, *dstlen)` октетов копируется из источника в приёмник через `memmove()`, так что области могут перекрываться. Если приёмник длиннее источника, остаток заполняется октетом заполнения. Если короче, лишняя часть источника не перемещается. Длины - от 0 до 65535.

Возвращаемые условные значения

Значение	Смысл
`STS$K_SUCCESS`	Всегда

Пример

```
unsigned short l_srclen = 5, l_dstlen = 16;
unsigned char  l_fill = ' ';
char l_name [16];

__util$movc5(&l_srclen, "alpha", &l_fill, &l_dstlen, l_name);   /* "alpha" and 11 spaces
 ↪  */
```

__util$fill – Заполнение буфера образцом

Формат

```
static inline   int __util$fill (void * dst, int dstsz, const void * src, int srcsz);
```

Возвращает

`int`: условное значение.

Аргументы

- *dst* – `void *`; только запись; по ссылке. Буфер.
- *dstsz* – `int`; только чтение; по значению. Его размер.
- *src* – `const void *`; только чтение; по ссылке. Образец.
- *srcsz* – `int`; только чтение; по значению. Его размер; должен быть больше 0.

Описание

Заполняет *dst* копиями образца одна за другой; последняя копия обрезается по размеру. Образец нулевого (или отрицательного) размера никогда ничего бы не заполнил и отвергается (в 1.6.11 рутина на нём зацикливалась навсегда).

Возвращаемые условные значения

Значение	Смысл
`STS$K_SUCCESS`	Заполнено
`STS$K_ERROR`	*dst* или *src* нуль, *dstsz* отрицателен или *srcsz* не больше 0

__util$bzero – Очистка памяти

Формат

```
inline static int __util$bzero  (void * bufp, size_t bufsz);
```

Возвращает

`int`: условное значение.

Аргументы

- *bufp* – `void *`; только запись; по ссылке. Область.
- *bufsz* – `size_t`; только чтение; по значению. Её размер.

Описание

Записывает нуль в каждый из *bufsz* октетов (это `memset()` в нуль). В 1.6.11 последние *bufsz* по модулю 4 октетов оставались как были; теперь очищается каждый октет, при любых длине и выравнивании. Длина 0 - успех, и на адрес при этом не смотрят.

Возвращаемые условные значения

Значение	Смысл
`STS$K_SUCCESS`	Сделано
`STS$K_ERROR`	*bufp* нуль, а *bufsz* не 0

__util$iszero – Проверка памяти на нули

Формат

```
inline static int __util$iszero (void * bufp, int bufsz);
```

Возвращает

`int`: условное значение.

Аргументы

- *bufp* – `void *`; только чтение; по ссылке. Область.
- *bufsz* – `int`; только чтение; по значению. Её размер.

Описание

Проверяет область по восемь октетов (через `memcpy()`, так что невыровненный адрес допустим), а затем оставшиеся от 0 до 7 октетов по одному. Проверяется каждый октет на каждой платформе, в том числе там, где `long` имеет 4 октета (в 1.6.11 там до 4 октетов могли остаться непроверенными). Смотреть в нулевой области или при длине 0 нечего, и обе считаются целиком нулевыми.

Возвращаемые условные значения

Значение	Смысл
`STS$K_SUCCESS`	Каждый октет нулевой (либо длина 0, либо *bufp* нуль)
`STS$K_WARN`	Найден ненулевой октет

Пример

```
if ( __util$iszero(l_key, sizeof(l_key)) == STS$K_SUCCESS )
    $LOG(STS$K_WARN, "the key is not set");
```

__util$strxor – Исключающее ИЛИ данных с ключом

Формат

```
static inline   int __util$strxor   (void * key, int keysz, void * src, int srcsz, void *
↪  dst, int dstsz, int * ctx);
```

Возвращает

`int`: условное значение.

Аргументы

- *key* – `void *`; только чтение; по ссылке. Ключ.
- *keysz* – `int`; только чтение; по значению. Его длина; должна быть больше 0.
- *src* – `void *`; только чтение; по ссылке. Данные.
- *srcsz* – `int`; только чтение; по значению. Их длина.
- *dst* – `void *`; только запись; по ссылке. Результат; может совпадать с *src*.
- *dstsz* – `int`; только чтение; по значению. Размер области результата.
- *ctx* – `int *`; изменение; по ссылке. Позиция в ключе, сохраняемая между вызовами. Может быть нулевым.

Описание

Применяет XOR к `min(srcsz, dstsz)` октетам *src* с ключом, повторяя ключ по мере надобности. Если *ctx* не нуль, позиция в ключе берётся из него в начале и записывается обратно в конце, так что поток можно обрабатывать частями: результат тот же, что для целого сразу. Начинайте с `*ctx` = 0. Значение `*ctx` выше *keysz* отвергается.

Возвращаемые условные значения

Значение	Смысл
`STS$K_SUCCESS`	Сделано
`STS$K_ERROR`	`*ctx` больше *keysz*

Пример

```
int l_pos = 0;

__util$strxor(l_key, l_keylen, l_buf, l_n, l_buf, l_n, &l_pos); /* in place, a piece of a
↪  stream */
```

$ARRSZ – Число элементов массива

Формат

```
#define $ARRSZ(a)   (sizeof(a)/sizeof(a[0]))
```

Возвращает

`size_t`: число элементов.

Аргументы

- *a* – массив (не указатель); только чтение. Макрос вычисляется в константу; сам массив не вычисляется.

Пример

```
KWDENT l_tbl [] = { ... };

for ( size_t i = 0; i < $ARRSZ(l_tbl); i++ )
    ;
```

2.5 Сопоставление с образцом

__util$pattern_match – Сопоставление строки с образцом-шаблоном

Формат

```
int __util$pattern_match    (char * a_str, char * a_pattern);
```

Возвращает

`int`: **не** условное значение. 1, если строка соответствует, 0, если нет (также для нулевого аргумента).

Аргументы

- *a_str* – `char *`; только чтение; по ссылке. Проверяемая строка, завершённая нулём.
- *a_pattern* – `char *`; только чтение; по ссылке. Образец, завершённый нулём.

Описание

Вся проверяемая строка должна соответствовать всему образцу. В образце:

Октет	Соответствует
`*`	Любой последовательности октетов, в том числе пустой
`%` или `?`	Ровно одному октету
любой другой	Самому себе; сравнение учитывает регистр

Экранирования нет: обратная косая черта - обычный октет, а `?` всегда подстановочный знак. Сопоставитель итеративный; его работа ограничена произведением длин, и он не рекурсивен. Подстановочные знаки имеют силу только в **образце**: `*`, `%` или `?` в проверяемой строке - обычный октет. Строка `*ba` соответствует образцу `*a`, а также образцу `*b*`. (В 1.6.11 `*` строки против `*` образца принималась буквально, и `*ba` не соответствовало `*a`.)

Возвращаемые условные значения

Нет; рутина возвращает 1 при соответствии и 0 в противном случае, а также 0 для нулевого аргумента.

Пример

```
if ( __util$pattern_match(l_name, "SYS$*.COM") )
    $LOG(STS$K_INFO, "'%s' is a system procedure", l_name);
```

2.6 Шестнадцатеричные числа, UUID, base64 и CRC

__util$hex2bin – Преобразование шестнадцатеричного текста в двоичные данные

Формат

```
inline  static int __util$hex2bin   (const void * a_srchex, void * a_dstbin, unsigned
↪  short a_srchexlen);
```

Возвращает

`int`: число записанных октетов, *a_srchexlen* / 2; -1 при неверном вводе.

Аргументы

- *a_srchex* – `const void *`; только чтение; по ссылке. Текст в любом регистре.
- *a_dstbin* – `void *`; только запись; по ссылке. Результат.
- *a_srchexlen* – `unsigned short`; только чтение; по значению. Число цифр; оно должно быть чётным.

Описание

Преобразует две цифры в один октет, в любом регистре. Ввод проверяется заранее: **нечётное** число цифр или символ, не являющийся шестнадцатеричной цифрой (пробел, `0x`, знак), заставляет рутину вернуть -1 **ничего не записав**, так что получатель никогда не заполнен наполовину. (До 1.7.0 цифры не проверялись, и “abc” давало 0x0A 0xBC.) Текст не обязан кончаться нулём; читаются только *a_srchexlen* октетов. При *a_srchexlen* 0 ничего не записывается и результат равен 0.

Возвращаемые условные значения

Нет; рутина возвращает длину или -1.

Пример

```
unsigned char l_id [4];

__util$hex2bin("c0a80102", l_id, 8);
```

__util$bin2hex – Преобразование двоичных данных в шестнадцатеричный текст

Формат

```
inline  static int __util$bin2hex   (const void * a_srcbin, void * a_dsthex, unsigned
↪  short a_srcbinlen);
```

Возвращает

`int`: число записанных символов, 2 * *a_srcbinlen*, без нуля.

Аргументы

- *a_srcbin* – `const void *`; только чтение; по ссылке. Октеты.
- *a_dsthex* – `void *`; только запись; по ссылке. Текст. Он должен вмещать 2 * *a_srcbinlen* + 1 октетов.
- *a_srcbinlen* – `unsigned short`; только чтение; по значению. Число октетов.

Описание

Записывает для каждого октета две шестнадцатеричные цифры в нижнем регистре и завершающий нуль.

Возвращаемые условные значения

Нет; рутина возвращает длину.

$BIN2HEX – Преобразование двоичных данных в шестнадцатеричные (макрос)

Формат

```
#define $BIN2HEX(s,d,l) __util$bin2hex((char*) s, (char*) d, (unsigned short) l)
```

Описание

Вызывает `__util$bin2hex()` с приведениями типов. Аргументы как у рутины; *s* - источник, *d* - приёмник, *l* - длина источника.

Пример

```
char l_hex [2 * 16 + 1];

$BIN2HEX(l_uuid, l_hex, 16);
```

__util$faohex – Форматирование октетов в одну строку шестнадцатеричного дампа

Формат

```
int __util$faohex   (const void *src, unsigned short srclen, char *out, unsigned short
↪  outsz, unsigned short *outlen);
```

Возвращает

`int`: число записанных символов; 0, если буфер слишком мал или адрес нулевой.

Аргументы

- *src* – `const void *`; только чтение; по ссылке. Октеты.
- *srclen* – `unsigned short`; только чтение; по значению. Сколько их; больше 16 принимается за 16.
- *out* – `char *`; только запись; по ссылке. Буфер.
- *outsz* – `unsigned short`; только чтение; по значению. Его размер; не менее 67 октетов.
- *outlen* – `unsigned short *`; только запись; по ссылке. Получает длину. Необязательный: может быть нулевым (в 1.6.11 нулевой адрес разыменовывался).

Описание

Форматирует до 16 октетов как 16 столбцов по две шестнадцатеричные цифры и пробел (48 символов), два пробела и 16 символов тех же октетов, с точкой для непечатаемых: всего 66 символов и нулевой октет. Неиспользованные столбцы - пробелы. Рутина никогда не читает за пределами *srclen* октетов (или за пределами 16 из них).

Возвращаемые условные значения

Нет; рутина возвращает длину. 0 означает, что *outsz* меньше 67 либо *out* или *src* нулевой.

Пример

```
char l_line [80];
unsigned short l_len;

__util$faohex(l_buf, 16, l_line, sizeof(l_line), &l_len);
```

__util$uuid2str – Преобразование UUID в текст

Формат

```
inline static int   __util$uuid2str (void * uuid, char * buf, int bufsz);
```

Возвращает

`int`: длина записанного текста.

Аргументы

- *uuid* – `void *`; только чтение; по ссылке. 16 октетов.
- *buf* – `char *`; только запись; по ссылке. Буфер.
- *bufsz* – `int`; только чтение; по значению. Число символов, которое он может принять, без нуля.

Описание

Записывает UUID в виде `xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx` в нижнем регистре, в порядке октетов, без смены порядка байтов, и завершающий нуль. Записывается не более *bufsz* символов; за ними идёт нуль, поэтому буфер должен вмещать *bufsz* + 1 октетов (37 для всего текста).

Возвращаемые условные значения

Нет; рутина возвращает длину (36 для целого UUID).

Пример

```
char l_text [37];

__util$uuid2str(l_uuid, l_text, 36);
```

__util$base64 – Кодирование октетов в base64

Формат

```
int __util$base64   (void *src, int srclen, void *dst, int dstsz, int *dstlen);
```

Возвращает

`int`: условное значение.

Аргументы

- *src* – `void *`; только чтение; по ссылке. Данные.
- *srclen* – `int`; только чтение; по значению. Их длина в октетах.
- *dst* – `void *`; только запись; по ссылке. Выходной буфер.
- *dstsz* – `int`; только чтение; по значению. Его размер.
- *dstlen* – `int *`; только запись; по ссылке. Получает длину текста; не должен быть нулевым.

Описание

Кодирует алфавитом RFC 4648 (A-Z, a-z, 0-9, `+`, `/`), дополняя `=` до кратного четырём числа символов: 4 символа на каждые 3 октета, с округлением вверх. *dstlen* устанавливается в нужную длину даже если *dstsz* слишком мал, и тогда ничего не записывается. Нулевой октет записывается после текста, если за текстом есть для него место; в *dstlen* он не учитывается.

Возвращаемые условные значения

Значение	Смысл
`STS$K_SUCCESS`	Закодировано
`STS$K_ERROR`	*dstlen* нулевой; длина отрицательна; *src* или *dst* нулевой при ненулевой длине; *dstsz* слишком мал (тогда *dstlen* содержит нужный размер)

Пример

```
char l_text [64];
int  l_len;

if ( 1 & __util$base64("Aladdin:open sesame", 19, l_text, sizeof(l_text), &l_len) )
    $LOG(STS$K_INFO, "%.*s", l_len, l_text);
```

__util$unbase64 – Декодирование base64

Формат

```
int __util$unbase64 (void *src, int srclen, void *dst, int dstsz, int *dstlen);
```

Возвращает

`int`: условное значение.

Аргументы

- *src* – `void *`; только чтение; по ссылке. Текст.
- *srclen* – `int`; только чтение; по значению. Его длина в символах.
- *dst* – `void *`; только запись; по ссылке. Выходной буфер.
- *dstsz* – `int`; только чтение; по значению. Его размер.
- *dstlen* – `int *`; только запись; по ссылке. Получает число декодированных октетов; не должен быть нулевым.

Описание

Декодирует текст по RFC 4648. Дополнение `=` принимается в конце и может отсутствовать; последняя группа из двух или трёх символов декодируется как есть. Декодер строгий: символ вне алфавита (в том числе пробельный), `=` не в конце, больше двух таких знаков, дополнение, не завершающее группу, или последняя группа из одного символа - ошибка. Результат нулём не завершается. *dstlen* устанавливается в нужный размер до проверки текста, а вывод пишется по ходу проверки: после ошибки буфер может содержать часть результата.

Возвращаемые условные значения

Значение	Смысл
`STS$K_SUCCESS`	Декодировано
`STS$K_ERROR`	*dstlen* нулевой; неверные аргументы, как в `__util$base64()`; искажённый текст; *dstsz* слишком мал

__util$crc32c – Вычисление CRC-32

Формат

```
unsigned    __util$crc32c   (unsigned int crc, const void *buf, size_t buflen);
```

Возвращает

`unsigned`: контрольная сумма. **Не** условное значение.

Аргументы

- *crc* – `unsigned int`; только чтение; по значению. 0 для начала либо результат предыдущего вызова для продолжения.
- *buf* – `const void *`; только чтение; по ссылке. Данные.
- *buflen* – `size_t`; только чтение; по значению. Их длина.

Описание

Вычисляет CRC-32 по IEEE 802.3 и zlib (отражённый полином 0xEDB88320), а **не** по полиному Кастаньоли, на который намекает имя. Имя сохранено, поскольку от него зависят программы; комментарий в заголовке теперь говорит, что это за функция. Контрольное значение для “123456789” - 0xCBF43926. Результат одинаков на всех платформах. Вычисляется самым быстрым механизмом, который есть у процессора: умножение без переносов на x86-64 (PCLMULQDQ и SSE 4.1), команды CRC32 на ARMv8 под Linux, иначе таблицы slice-by-8. Механизм выбирается при первом вызове; переменная окружения `STARLET_CRC32=SOFTWARE` принуждает к таблицам. Принимается буфер любого выравнивания и длины. Длинный буфер можно передавать частями: результат одной части есть *crc* следующей.

Возвращаемые условные значения

Нет; рутина возвращает контрольную сумму.

Пример

```
unsigned l_crc = 0;

l_crc = __util$crc32c(l_crc, l_part1, l_len1);
l_crc = __util$crc32c(l_crc, l_part2, l_len2);
```

2.7 Ключевые слова и символы

KWDENT, UTIL$K_LOOKUP_* – Типы таблицы ключевых слов

Формат

```
#define UTIL$K_LOOKUP_NCASE (1 << 0)
#define UTIL$K_LOOKUP_ABBREV    (1 << 1)

typedef struct _kwdent
{
    ASC kwd;
    union   {
        int val;
        void *  ptr;
        };
} KWDENT;
```

Описание

Элемент таблицы ключевых слов: ключевое слово как счётная строка и значение, которое оно обозначает, целое или адрес (они делят одно место). Таблица - массив элементов; сортировать её не нужно. Два элемента с одним значением - *синонимы*, например `SHOW` и `SH`. Заполняйте все элементы одинаково: синонимы определяются сравнением всего объединения.

`UTIL$K_LOOKUP_NCASE` заставляет сравнение игнорировать регистр. `UTIL$K_LOOKUP_ABBREV` позволяет строке быть началом ключевого слова.

Пример

```
enum { K_SHOW = 1, K_SET, K_SETUP };

static KWDENT s_cmds [] = {
    { {$ASCINI("SHOW")},    {.val = K_SHOW} },
    { {$ASCINI("SH")},  {.val = K_SHOW} },
    { {$ASCINI("SET")}, {.val = K_SET} },
    { {$ASCINI("SETUP")},   {.val = K_SETUP} },
};
```

__util$lookup_key – Поиск ключевого слова в таблице

Формат

```
inline  static  int __util$lookup_key   (const char *a_src, int a_srclen, KWDENT *a_ktbl,
↪  int a_ktblsz, KWDENT **a_kwd, int a_flags);
```

Возвращает

`int`: условное значение.

Аргументы

- *a_src* – `const char *`; только чтение; по ссылке. Искомая строка; она не обязана кончаться нулём.
- *a_srclen* – `int`; только чтение; по значению. Её длина.
- *a_ktbl* – `KWDENT *`; только чтение; по ссылке. Таблица.
- *a_ktblsz* – `int`; только чтение; по значению. Число элементов в таблице.
- *a_kwd* – `KWDENT **`; только запись; по ссылке. Получает адрес найденного элемента, адрес внутри таблицы. Не изменяется, когда рутина возвращает `STS$K_ERROR`.
- *a_flags* – `int`; только чтение; по значению. 0 либо `UTIL$K_LOOKUP_NCASE`, `UTIL$K_LOOKUP_ABBREV` или оба.

Описание

Без `UTIL$K_LOOKUP_ABBREV` строка должна равняться ключевому слову по всей длине: `SETU` не есть `SETUP`, а `SET` находится независимо от своего места в таблице. С `UTIL$K_LOOKUP_NCASE` регистр игнорируется; без него сравнение точное.

С `UTIL$K_LOOKUP_ABBREV` строка может быть началом ключевого слова, как DCL принимает команду:

- ключевое слово, написанное полностью, побеждает, чьим бы началом ни была ещё эта строка (`SET` выбирает `SET`, хотя начинает `SETUP`);
- иначе, если строка является началом только одного ключевого слова, выбирается оно;
- ключевые слова с одним значением (синонимы) считаются за одно: `S` для `SHOW` и `SH` выбирает первое из них;
- если строка является началом ключевых слов с разными значениями, сокращение неоднозначно. Рутина возвращает `STS$K_WARN`, а *a_kwd* - первый подходящий элемент.

Пустая строка (*a_srclen* не больше 0) - ошибка. Строка длиннее ключевого слова никогда ему не соответствует. Результат не зависит от порядка элементов, за исключением того, что *a_kwd* - первое совпадение для неоднозначного сокращения и для синонимов. Заметьте, что `STS$K_WARN` равно 0: младший бит сброшен, так что `1 & status` ложно, хотя *a_kwd* установлен.

Возвращаемые условные значения

Значение	Смысл
`STS$K_SUCCESS`	Найдено; *a_kwd* - элемент
`STS$K_WARN`	Только ABBREV: сокращение неоднозначно; *a_kwd* - первый подходящий элемент
`STS$K_ERROR`	Строка пуста или не соответствует ни одному ключевому слову; *a_kwd* не изменён

Пример

```
KWDENT *l_kwd;
int     l_flags = UTIL$K_LOOKUP_NCASE | UTIL$K_LOOKUP_ABBREV;

switch ( __util$lookup_key("sh", 2, s_cmds, $ARRSZ(s_cmds), &l_kwd, l_flags) )
    {
    case STS$K_SUCCESS:
        $LOG(STS$K_INFO, "command %d", l_kwd->val);
        break;

    case STS$K_WARN:
        $LOG(STS$K_WARN, "ambiguous, could be %.*s", $ASC(&l_kwd->kwd));
        break;

    default:
        $LOG(STS$K_ERROR, "unknown command");
    }
```

SYM_REC, $SYM_REC_INI, $SYM_REC_EOL – Типы таблицы символов

Формат

```
typedef struct __sym_rec__ {
    unsigned long long  val;
    unsigned char       len;
            char    *sym;
} SYM_REC;

#define $SYM_REC_INI(s) {s, sizeof(#s)-1,  #s}
#define $SYM_REC_EOL    {0, 0, NULL}
```

Описание

Таблица символов сопоставляет значение или бит маски с его именем. `$SYM_REC_INI(name)` создаёт элемент из константы и её собственного имени как строки (имя превращается в строку, так что макрос в нём не раскрывается). `$SYM_REC_EOL` завершает таблицу. Таблица для `__util$mask2sym()` кончается на первом элементе, значение которого 0 или имя нулевое; таблица для `__util$val2sym()` кончается на первом элементе, длина которого 0 или имя нулевое. Завершайте оба вида `$SYM_REC_EOL`; не помещайте символ со значением 0 в таблицу маски.

Пример

```
enum { F_READ = 1, F_WRITE = 2, F_EXEC = 4 };

static const SYM_REC s_flags [] = {
    $SYM_REC_INI(F_READ), $SYM_REC_INI(F_WRITE), $SYM_REC_INI(F_EXEC), $SYM_REC_EOL
};
```

__util$val2sym – Перевод значения в его имя

Формат

```
static inline int   __util$val2sym  (unsigned long long val, const SYM_REC *sym, char
↪  *out, int outsz);
```

Возвращает

`int`: длина записанного имени; 0, если значения нет в таблице.

Аргументы

- *val* – `unsigned long long`; только чтение; по значению. Значение.
- *sym* – `const SYM_REC *`; только чтение; по ссылке. Таблица.
- *out* – `char *`; только запись; по ссылке. Буфер.
- *outsz* – `int`; только чтение; по значению. Его размер.

Описание

Записывает имя первого элемента, значение которого равно *val*, и завершающий нуль. Если такого элемента нет или имя не помещается (нужно, чтобы *outsz* было больше его длины), записывается пустая строка и результат равен 0. Нулевой октет записывается в любом случае, поэтому *out* должен вмещать не менее одного октета.

Возвращаемые условные значения

Нет; рутина возвращает длину.

__util$mask2sym – Перевод маски в имена

Формат

```
static inline int   __util$mask2sym (unsigned long long mask, const SYM_REC *sym, char
↪  *out, int outsz);
```

Возвращает

`int`: длина записанного текста.

Аргументы

- *mask* – `unsigned long long`; только чтение; по значению. Маска.
- *sym* – `const SYM_REC *`; только чтение; по ссылке. Таблица битов.
- *out* – `char *`; только запись; по ссылке. Буфер.
- *outsz* – `int`; только чтение; по значению. Его размер.

Описание

Записывает имена всех элементов, значение которых имеет общий с *mask* бит, через разделитель `|`, в порядке таблицы, и завершающий нуль. Проверка наличия места учитывает разделитель и нулевой октет: имя записывается, только если помещаются оно, предшествующий `|` и нуль. Когда имя не помещается, рутина останавливается; записанное остаётся, и текст всегда завершён нулём. Буфер меньше 1 октета не затрагивается, длина равна 0. (В 1.6.11 проверка не учитывала разделитель и нулевой октет, и буфер мог быть переполнен.)

Возвращаемые условные значения

Нет; рутина возвращает длину.

Пример

```
char l_text [64];

__util$mask2sym(F_READ | F_EXEC, s_flags, l_text, sizeof(l_text));  /* "F_READ|F_EXEC" */
```

2.8 Параметры и конфигурация

OPTS, OPTS$K_*, OPTS_NULL – Типы таблицы параметров

Формат

```
#define OPTS$K_INT  0
#define OPTS$K_STR  1
#define OPTS$K_OPT  2
#define OPTS$K_PORT 3
#define OPTS$K_IP4  4
#define OPTS$K_CONF 5
#define OPTS$K_PWD  6

typedef struct _opts    {
        ASC name;
        void *  ptr;
        size_t  sz;
        int type;
    } OPTS;

#define OPTS_NULL { {0, 0}, NULL, 0, 0}
```

Описание

Элемент таблицы параметров (options) программы: *name* - имя, *ptr* - переменная, получающая значение, *sz* - её размер, *type* - как значение преобразуется. Таблица кончается `OPTS_NULL` (элементом, у которого длина имени 0).

Тип	*ptr* указывает на	Значение
`OPTS$K_INT`	целое ровно в *sz* октетов: 1, 2, 4 или 8; *sz* 0 означает `int`	Десятичное, шестнадцатеричное с `0x` или восьмеричное с `0` число. Размеры 1, 2 и 8 – **беззнаковые**: ведущий `-` отвергается, диапазон от 0 до 2^n-1. Размеры 4 и 0 – **знаковые**: минус принимается, диапазон от -2147483648 до 2147483647. После числа ничего не должно следовать; иначе это неверное значение, и переменная не изменяется
`OPTS$K_STR`	`ASC`; *sz* - число символов, которое он может принять	Текст, обрезанный по меньшему из *sz* и `ASC$K_SZ` - 1 (254), с нулевым октетом после него
`OPTS$K_OPT`	`int`	Без значения: присутствующий параметр устанавливает `int` в 1. Заранее установите его в 0

Тип	*ptr* указывает на	Значение
`OPTS$K_PORT`	`unsigned short`	Число от 0 до 65535; **порядок байтов хоста** (чтобы выставить на провод, используйте `htons()`). Большее число - неверное значение
`OPTS$K_IP4`	`struct in_addr`	Точечный адрес; сетевой порядок байтов, как у этой структуры. Неверный адрес заносится в журнал как ошибка, и переменная остаётся прежней
`OPTS$K_CONF`	`ASC` или нуль	Только `__util$getparams()`: имя файла; файл тут же читается через `__util$readconfig()`. Внутри файла конфигурации параметр отвергается с сообщением (вложенность не допускается)
`OPTS$K_PWD`	`ASC`	Как `OPTS$K_STR`; `__util$showparams()` его не показывает

Задавайте *sz* для `OPTS$K_INT` как размер переменной: 1 для `char`, 2 для `short`, 4 для `int` или 8 для `long long`; 0 означает `int`. Любой другой размер - ошибка. 32-разрядная маска выше 2147483647 в `int` не помещается и является неверным значением (до 1.7.0 она сохранялась как отрицательное число, а минус принимался и для размеров 1, 2 и 8): дайте параметру размер 8. Рутины записывают ровно столько октетов, так что `short` больше не переполняется (в 1.6.11 каждый размер, кроме 8, записывался как `int`). `OPTS$K_PORT` всегда `unsigned short`, его *sz* не используется. Комментарий в заголовке 1.6.11 говорил о порте «сетевой порядок»; так никогда не было, и теперь комментарий говорит о порядке хоста.

Пример

```
static ASC  g_logfile, g_cfgfile;
static int  g_verbose, g_count;
static unsigned short g_port = 8080;

static OPTS s_opts [] = {
    { {$ASCINI("config")},  &g_cfgfile, ASC$K_SZ - 1,   OPTS$K_CONF },
    { {$ASCINI("logfile")}, &g_logfile, ASC$K_SZ - 1,   OPTS$K_STR },
    { {$ASCINI("verbose")}, &g_verbose, 0,      OPTS$K_OPT },
    { {$ASCINI("count")},   &g_count,   0,      OPTS$K_INT },
    { {$ASCINI("port")},    &g_port,    0,      OPTS$K_PORT },
    OPTS_NULL
};
```

__util$getparams – Чтение параметров из командной строки

Формат

```
int __util$getparams    (int, char *[], const OPTS *);
```

Возвращает

`int`: условное значение.

Аргументы

- *argc* – `int`; только чтение; по значению. Число аргументов.
- *argv* – `char *[]`; только чтение; по ссылке. Аргументы. Они никогда не изменяются (в 1.6.11 значения усекались по краям пробелов на месте).
- *opts* – `const OPTS *`; только чтение; по ссылке. Таблица параметров. Названные в ней переменные записываются.

Описание

Просматривает `argv[1]` … `argv[argc-1]`. Аргумент имеет вид `-name=value`, `/name=value` или `--name=value`; для `OPTS$K_OPT` значение необязательно. Имя ищется в таблице без учёта регистра. Имя, написанное полностью, побеждает; иначе текст может быть сокращением, начальной частью ровно одного имени параметра (правило DCL, то же, что у `__util$lookup_key()` с `UTIL$K_LOOKUP_ABBREV`). Сокращение, подходящее нескольким параметрам, неоднозначно и является ошибкой. Значение освобождается от пробельных символов по краям (в копии: *argv* не изменяется) и преобразуется по типу элемента. Если один параметр задан дважды, побеждает последний.

Пустой аргумент пропускается. Неверное значение, неизвестный или неоднозначный параметр и аргумент, не начинающийся с `-` или `/`, каждый сообщается через `$LOG()` с `STS$K_ERROR` (записи несут средство `UTILS`) и пропускается, а остальные аргументы всё равно обрабатываются; в конце статус вызова - ошибка. Параметр `OPTS$K_CONF` записывает своё значение в `ASC` (если *ptr* не нуль) и тут же читает этот файл через `__util$readconfig()`; параметры, идущие далее в командной строке, перекрывают параметры файла.

Возвращаемые условные значения

Значение	Смысл
`STS$K_SUCCESS`	Каждый аргумент принят
`STS$K_ERROR`	Неверное значение, неизвестный или неоднозначный параметр, аргумент, не являющийся параметром, либо ошибка чтения файла конфигурации. Хорошие аргументы всё равно приняты
`STS$K_INFO`	Только в файле конфигурации, названном `OPTS$K_CONF`, были неизвестные параметры (пропущены и занесены в журнал). Младший бит установлен: `1 & status` истинно

В 1.6.11 рутина возвращала `STS$K_SUCCESS` после неверного значения или неизвестного параметра и `STS$K_ERROR` только тогда, когда файл конфигурации прочитать не удавалось.

Пример

```
int main (int argc, char *argv[])
{
```

```
    if ( !(1 & __util$getparams(argc, argv, s_opts)) )
        return  STS$K_ERROR;

    __util$showparams(s_opts);

    /* the work of the program */
    return  STS$K_SUCCESS;
}
```

__util$readconfig – Чтение параметров из файла

Формат

```
int __util$readconfig   (const char *, OPTS *);
```

Возвращает

`int`: условное значение.

Аргументы

- *fconf* – `const char *`; только чтение; по ссылке. Путь к файлу.
- *opts* – `OPTS *`; только чтение; по ссылке. Таблица параметров.

Описание

Значение, которое после отсечения пробелов начинается и кончается одной и той же кавычкой, `"` или `'`, теряет обе кавычки (для любого типа); одинокая или несбалансированная кавычка остаётся в значении.

Читает файл по строкам. Всё от `!` до конца строки - комментарий, если только `!` не находится внутри части в двойных или одинарных кавычках; строка освобождается от пробельных символов по краям. Непустая строка должна быть `-name=value` или `/name=value`; любая другая сообщается через `$LOG()` с `STS$K_ERROR` и пропускается. Строка длиннее 254 октетов тоже ошибка, если только её остаток не комментарий; хвост такой строки отбрасывается и не читается как отдельная строка. Имя, которого нет в таблице, сообщается с `STS$K_WARN` и пропускается: файл может обслуживать несколько программ. Каждое сообщение заносится в журнал со средством `UTILS`, а остаток файла всё равно обрабатывается.

Имя может быть сокращено по правилу `__util$getparams()`: имя, написанное полностью, побеждает, иначе оно должно быть началом ровно одного имени параметра, без учёта регистра; сокращение, подходящее нескольким, является ошибкой. Преобразование значений то же, что в `__util$getparams()`. Элемент `OPTS$K_CONF` внутри файла отвергается с сообщением: файл конфигурации не называет другого.

Возвращаемые условные значения

Значение	Смысл
`STS$K_SUCCESS`	Каждая строка принята
`STS$K_INFO`	Параметр был неизвестен: пропущен и занесён в журнал, ничего иного неверного не было. Младший бит установлен, так что это не неудача
`STS$K_ERROR`	Нет имени файла, файл не читается, неверное значение, строка, не являющаяся параметром, неоднозначное сокращение, `OPTS$K_CONF` или слишком длинная строка

Пример

```
if ( !(1 & __util$readconfig("/etc/myapp.conf", s_opts)) )
    $LOG(STS$K_WARN, "no configuration, defaults are used");
```

__util$showparams – Запись параметров в журнал

Формат

```
int __util$showparams   (const OPTS *opts);
```

Возвращает

int: STS$K_SUCCESS.

Аргументы

- *opts* – const OPTS *; только чтение; по ссылке. Таблица.

Описание

Пишет одну запись $LOG() с STS$K_INFO для каждого параметра таблицы до OPTS_NULL: name = value. Порт показывается в десятичном и шестнадцатеричном виде, адрес - в точечной форме, флаг - как ON или OFF, строка - вместе с длиной, пароль - как <password>. Целое показывается по размеру: 1, 2 и 8 октетов как беззнаковые байт, слово и квадрослово (!UB, !UW, !UQ), 4 и 0 как знаковый longword (!SL), каждое – вместе с шестнадцатеричным видом. Элемент OPTS$K_CONF не показывается. Записи пишет библиотека, поэтому их средство - UTILS, какой бы __FAC__ ни был у программы.

Возвращаемые условные значения

Значение	Смысл
STS$K_SUCCESS	Всегда

2.9 Очереди, блокировки и атомарные операции

Очередь (queue) - двусвязный список, взаимоблокируемый спин-блокировкой в его заголовке. Элемент очереди - структура, которая *начинается* с `ENTRY`; рутины очередей считают адрес структуры адресом её `ENTRY`.

ENTRY, __QUEUE, QUEUE_INITIALIZER – Типы очереди

Формат

```
typedef struct  __entry {
    struct  __entry *left, *right;
    void    *queue;
} ENTRY;

typedef struct  __queue {
    ENTRY       *head, *tail;
    int     lock;           /* SRWLOCK on Windows */
    unsigned    count;
} __QUEUE;

#define QUEUE_INITIALIZER { (ENTRY *) 0, (ENTRY *) 0, 0, 0 }
```

Описание

left и *right* связывают элемент с соседями; на концах они нулевые. *queue* - адрес очереди, в которой находится элемент, либо нуль, когда он не в какой-либо очереди. Заголовок очереди содержит адреса первого и последнего элементов, блокировку и число элементов. Новая очередь инициализируется `QUEUE_INITIALIZER`. Структуры упакованы с выравниванием на 8 октетов.

Пример

```
typedef struct { ENTRY links; unsigned short len; char buf[128]; } MSG;

static __QUEUE s_free = QUEUE_INITIALIZER;
static MSG     s_pool [32];

for ( int i = 0; i < 32; i++ )
    {
    unsigned l_count;

    $INSQTAIL(&s_free, &s_pool[i], &l_count);
    }
```

__util$lockspin – Захват спин-блокировки

Формат

```
inline  static int __util$lockspin (void volatile * lock);
```

Возвращает

`int`: условное значение.

Аргументы

- *lock* – `void volatile *`; изменение; по ссылке. Адрес блокировки, `int`, равный 0, когда она свободна. В Windows - адрес `SRWLOCK`.

Описание

Вращается, пока блокировка не освободится, затем захватывает её (атомарным обменом устанавливает `int` в 1). Рутина сдаётся примерно после 4 миллиардов попыток. Блокировка не рекурсивна: поток, захвативший её дважды, ждёт сам себя, пока не сдастся.

Возвращаемые условные значения

Значение	Смысл
`STS$K_SUCCESS`	Блокировка захвачена
`STS$K_ERROR`	Блокировка не освободилась

__util$unlockspin – Освобождение спин-блокировки

Формат

```
inline  static  int __util$unlockspin (void * lock);
```

Возвращает

`int`: `STS$K_SUCCESS`.

Аргументы

- *lock* – `void *`; изменение; по ссылке. Блокировка, захваченная `__util$lockspin()`.

Описание

Освобождает блокировку. Не освобождайте блокировку, которую не держите.

Возвращаемые условные значения

Значение	Смысл
`STS$K_SUCCESS`	Всегда

$LOCK_LONG, $UNLOCK_LONG – Макросы блокировки

Формат

```
#define $LOCK_LONG(lock)    __util$lockspin(lock)
#define $UNLOCK_LONG(lock)  __util$unlockspin(lock)
```

Описание

Псевдонимы двух рутин.

Пример

```
static int s_lock;

if ( 1 & $LOCK_LONG(&s_lock) )
    {
    /* ... a short critical section ... */
    $UNLOCK_LONG(&s_lock);
    }
```

__util$insqtail – Вставка элемента в хвост

Формат

```
inline static int __util$insqtail (void * que, void * ent, unsigned * count);
```

Возвращает

`int`: условное значение.

Аргументы

- *que* – `void *`; изменение; по ссылке. Очередь (`__QUEUE *`).
- *ent* – `void *`; изменение; по ссылке. Элемент (`ENTRY` в его начале).
- *count* – `unsigned *`; только запись; по ссылке. Получает число элементов в очереди до вставки.

Описание

Связывает элемент после последнего. Если элемент уже в этой очереди, вызов ничего не делает и возвращает `STS$K_SUCCESS`, а *count* не устанавливается. Если он в другой очереди, вставка отвергается. `ENTRY` нового элемента должен быть обнулён (его *queue* должен быть нулевым) до первой вставки.

Возвращаемые условные значения

Значение	Смысл
`STS$K_SUCCESS`	Вставлен либо уже в этой очереди
`UTIL$S_INVARG` (0x102)	Нулевой аргумент
`UTIL$S_INQUE` (0x202)	Элемент в другой очереди
`UTIL$S_NOLOCK` (0x302)	Блокировка не захвачена

__util$insqhead – Вставка элемента в голову

Формат

```
inline static int __util$insqhead (void * que, void * ent, unsigned * count);
```

Возвращает

`int`: условное значение.

Аргументы

Как у `__util$insqtail()`.

Описание

Связывает элемент перед первым. Те же правила, что у `__util$insqtail()`, и те же условные значения (в 1.6.11 каждая неудача этой рутины была простым `STS$K_ERROR`).

Возвращаемые условные значения

Значение	Смысл
`STS$K_SUCCESS`	Вставлен либо уже в этой очереди
`UTIL$S_INVARG` (0x102)	Нулевой аргумент
`UTIL$S_INQUE` (0x202)	Элемент в другой очереди
`UTIL$S_NOLOCK` (0x302)	Блокировка не захвачена

__util$remqhead – Удаление элемента из головы

Формат

```
inline static int __util$remqhead (void * que, void **ent, unsigned * count);
```

Возвращает

`int`: условное значение.

Аргументы

- *que* – `void *`; изменение; по ссылке. Очередь.
- *ent* – `void **`; только запись; по ссылке. Получает адрес удалённого элемента либо нуль, если очередь была пуста.
- *count* – `unsigned *`; только запись; по ссылке. Получает число элементов до удаления.

Описание

Отсоединяет первый элемент и очищает его связи и его *queue*. **Для пустой очереди рутина возвращает `STS$K_SUCCESS`, устанавливает *ent* в нуль, а *count* в 0.** Пустая очередь - обычный исход, а не ошибка; вызывающий отличает «получен элемент» от «ничего нет» по *ent* (или *count*), а не по статусу. В 1.6.11 в этом случае *ent* оставался неустановленным, и вызывающий, читавший его, получал устаревший указатель.

Возвращаемые условные значения

Значение	Смысл
`STS$K_SUCCESS`	Удалён либо очередь была пуста (*ent* нуль и *count* равен 0)
`STS$K_ERROR`	Нулевой аргумент либо блокировка не захвачена (*ent* не затронут)

Пример

```
MSG     *l_msg = NULL;
unsigned l_count;

if ( (1 & $REMQHEAD(&s_free, &l_msg, &l_count)) && l_msg )
    l_msg->len = 0;
```

__util$remqtail – Удаление элемента из хвоста

Формат

```
inline static int __util$remqtail (void * que, void **ent, unsigned * count);
```

Возвращает

`int`: условное значение.

Аргументы

Как у `__util$remqhead()`.

Описание

Отсоединяет последний элемент. Те же правила, что у `__util$remqhead()`, включая пустую очередь.

Возвращаемые условные значения

Как у `__util$remqhead()`.

__util$remqent – Удаление заданного элемента

Формат

```
inline static int __util$remqent (void * que, void *ent, unsigned * count);
```

Возвращает

`int`: условное значение.

Аргументы

- *que* – `void *`; изменение; по ссылке. Очередь.
- *ent* – `void *`; изменение; по ссылке. Удаляемый элемент; сам элемент, а не адрес указателя.
- *count* – `unsigned *`; только запись; по ссылке. Получает число элементов до удаления.

Описание

Отсоединяет элемент, где бы он ни стоял, и очищает его связи и *queue*. Элемент должен находиться в этой очереди. Элемент, которого в ней нет, включая случай пустой очереди, - ошибка, поскольку удалять нечего; это отличается от удаления из головы или хвоста, где пустая очередь - обычный исход. (В 1.6.11 пустая очередь давала `STS$K_SUCCESS` и ничего не делала.)

Возвращаемые условные значения

Значение	Смысл
`STS$K_SUCCESS`	Удалён
`STS$K_ERROR`	Нулевой аргумент, элемента нет в этой очереди либо блокировка не захвачена

__util$movqhead – Перемещение элемента в голову

Формат

```
inline static int __util$movqhead (void * que, void * ent);
```

Возвращает

`int`: условное значение.

Аргументы

- *que* – `void *`; изменение; по ссылке. Очередь.
- *ent* – `void *`; изменение; по ссылке. Сам элемент (`ENTRY` в его начале), как его передаёт `$MOVQHEAD()`. В 1.6.11 параметр был объявлен как `void **`, тогда как макрос и код считали его элементом; теперь прототип говорит то, что имеется в виду. Программе, вызывавшей рутину напрямую с приведением типа, менять вызов не нужно, а программа, передававшая `void **` ради старого прототипа, должна передать элемент.

Описание

Перемещает элемент в голову очереди. Элемент должен находиться в этой очереди (его поле *queue* проверяется). Если он уже голова, вызов успешен и ничего не меняется. Рутина отсоединяет элемент и ставит его перед прежней головой; если элемент был хвостом, хвост очереди перенацеливается на его левого соседа, и устанавливается левая связь прежней головы (в 1.6.11 обе оставались устаревшими). Стоимость постоянна: поиска нет.

Возвращаемые условные значения

Значение	Смысл
`STS$K_SUCCESS`	Перемещён либо элемент уже был головой
`STS$K_ERROR`	Нулевой аргумент, элемента нет в этой очереди либо блокировка не захвачена

__util$movqtail – Перемещение элемента в хвост

Формат

```
inline static int __util$movqtail (void * que, void * ent);
```

Возвращает

`int`: условное значение.

Аргументы

Как у `__util$movqhead()`.

Описание

Перемещает элемент в хвост очереди; зеркальное отражение `__util$movqhead()`. Элемент должен находиться в этой очереди. Если он уже хвост, вызов успешен и ничего не меняется. Если элемент был головой, голова очереди перенацеливается на его правого соседа. (В 1.6.11 эта рутина портила очередь: теряла элементы после перемещаемого, не обновляла голову, а в отладочной сборке останавливалась на `assert()`; тогда советовали ею не пользоваться. Теперь она безопасна.)

Возвращаемые условные значения

Значение	Смысл
`STS$K_SUCCESS`	Перемещён либо элемент уже был хвостом
`STS$K_ERROR`	Нулевой аргумент, элемента нет в этой очереди либо блокировка не захвачена

__util$q_unlink – Изъятие элемента из цепочки очереди

Формат

```
inline static void __util$q_unlink (__QUEUE * que, ENTRY * ent);
```

Возвращает

Ничего.

Аргументы

- *que* – `__QUEUE *`; изменение; по ссылке. Очередь.
- *ent* – `ENTRY *`; изменение; по ссылке. Отсоединяемый элемент; он должен стоять в *que*.

Описание

Вспомогательная рутина рутин очереди, новая в 1.7.0. Она соединяет соседей элемента друг с другом, перенацеливает голову или хвост очереди, если элемент был первым или последним, и очищает связи *left* и *right* элемента. Вызывающий должен держать спин-блокировку очереди и сам поддерживать в актуальном состоянии счётчик и поле *queue* элемента: рутина не трогает ни то, ни другое. Программа использует рутины очереди, а не эту.

Возвращаемые условные значения

Нет; рутина не возвращает значения.

__util$clrqueue – Опустошение очереди

Формат

```
inline static int __util$clrqueue (void * que,  unsigned * count);
```

Возвращает

`int`: условное значение.

Аргументы

- *que* – `void *`; изменение; по ссылке. Очередь.
- *count* – `unsigned *`; только запись; по ссылке. Получает число элементов, которое содержала очередь.

Описание

Очищает поля *left*, *right* и *queue* каждого элемента и опустошает заголовок очереди. Элементы не освобождаются. *count* получает число элементов, которое содержала очередь, и **0, если она была пуста** (в 1.6.11 в этом случае он оставался неустановленным, а связи элементов не очищались).

Возвращаемые условные значения

Значение	Смысл
`STS$K_SUCCESS`	Сделано (также для пустой очереди)
`STS$K_ERROR`	Нулевой аргумент либо блокировка не захвачена

$INSQHEAD, $INSQTAIL, $REMQHEAD, $REMQTAIL, $REMQENT, $MOVQHEAD, $MOVQTAIL, $CLRQUE – Макросы очереди

Формат

```
#define $INSQHEAD(que, ent, count)  __util$insqhead ((__QUEUE *) que, (void *) ent,
 ↪  (unsigned *) count)
#define $INSQTAIL(que, ent, count)  __util$insqtail ((__QUEUE *) que, (void *) ent,
 ↪  (unsigned *) count)
#define $REMQHEAD(que, ent, count)  __util$remqhead ((__QUEUE *) que, (void **) ent,
 ↪  (unsigned *) count)
#define $REMQTAIL(que, ent, count)  __util$remqtail ((__QUEUE *) que, (void **)ent,
 ↪  (unsigned *) count)
#define $MOVQHEAD(que, ent)     __util$movqhead ((__QUEUE *) que, (void *) ent)
#define $MOVQTAIL(que, ent)     __util$movqtail ((__QUEUE *) que, (void *) ent)
#define $REMQENT(que, ent, count)   __util$remqent ((__QUEUE *) que, (void **)ent,
 ↪  (unsigned *) count)
#define $CLRQUE(que, count)     __util$clrqueue ((__QUEUE *) que, (unsigned *) count)
```

Возвращает

`int`: то, что возвращает рутина.

Аргументы

- *que* – адрес `__QUEUE`.
- *ent* – для `$INSQHEAD()`, `$INSQTAIL()`, `$MOVQHEAD()`, `$MOVQTAIL()` и `$REMQENT()` адрес элемента. Для `$REMQHEAD()` и `$REMQTAIL()` адрес переменной-указателя, получающей элемент.
- *count* – адрес `unsigned`.

Описание

Макросы приводят свои аргументы к нужным типам, так что принимается любой указатель на структуру, начинающуюся с `ENTRY`. Ни один аргумент они не вычисляют более одного раза. Условия и правила те же, что у рутин. `$MOVQHEAD()` и `$MOVQTAIL()` передают элемент как `void *`, что в точности принимают прототипы 1.7.0.

Пример

```
MSG     *l_msg;
unsigned l_count;

$INSQTAIL(&s_free, &s_pool[0], &l_count);
$REMQHEAD(&s_free, &l_msg, &l_count);
$REMQENT(&s_free, &s_pool[1], &l_count);
$CLRQUE(&s_free, &l_count);
```

$CMP_STORE_LONG – Сравнение и обмен

Формат

```
#define $CMP_STORE_LONG(src, old_val, new_val)  __sync_bool_compare_and_swap((int *) src,
↪  (int) old_val, (int) new_val )
```

Возвращает

Не нуль, если значение было равно *old_val* и *new_val* записано; 0, если нет.

Аргументы

- *src* – адрес `int`; изменение; по ссылке. Переменная.
- *old_val* – `int`; только чтение; по значению. Значение, которое она должна иметь.
- *new_val* – `int`; только чтение; по значению. Записываемое значение.

Описание

Атомарное сравнение и обмен одного longword (`__sync_bool_compare_and_swap`).

Пример

```
static int s_state;

if ( $CMP_STORE_LONG(&s_state, 0, 1) )
    ;   /* this thread moved the state from 0 to 1 */
```

$ATOMIC_INC, $ATOMIC_DEC – Атомарные увеличение и уменьшение

Формат

```
#define $ATOMIC_INC(val)    __sync_fetch_and_add (&val, 1)
#define $ATOMIC_DEC(val)    __sync_fetch_and_sub (&val, 1)
```

Возвращает

Значение, которое переменная имела **до** изменения.

Аргументы

- *val* – целая переменная (lvalue, а не указатель); изменение; макрос берёт её адрес.

Описание

Атомарно прибавляет или вычитает 1 с полным барьером памяти (встроенные функции GCC `__sync`).

Пример

```
static unsigned s_users;

$ATOMIC_INC(s_users);
```

2.10 Диапазоны, минимум и максимум

__util$min, __util$max, $MIN, $MAX – Меньшее и большее из двух целых

Формат

```
inline static int __util$min (int x, int y);
inline static int __util$max (int x, int y);

#define $MIN(x, y)  __util$min((int) (x), (int) (y))
#define $MAX(x, y)  __util$max((int) (x), (int) (y))
```

Возвращает

`int`: меньшее или большее.

Аргументы

- *x*, *y* – `int`; только чтение; по значению.

Описание

Макросы приводят оба аргумента к `int` и вычисляют каждый один раз. Значение, не умещающееся в `int` (`size_t` выше 2^31 или 64-разрядное число), усекается; используйте `$MIN()` и `$MAX()` для размеров, заведомо малых.

Пример

```
int l_n = $MIN(l_avail, (int) sizeof(l_buf));
```

__util$isinrange, $ISINRANGE – Проверка значения на принадлежность замкнутому диапазону

Формат

```
inline static int __util$isinrange (int x, int __min__, int __max__ );

#define $ISINRANGE(x, __min__, __max__) __util$isinrange(x, (__min__), (__max__))
```

Возвращает

`int`: 1, если ***min*** <= *x* <= ***max***, иначе 0.

Аргументы

- *x* – `int`; только чтение; по значению. Значение.
- ***min***, ***max*** – `int`; только чтение; по значению. Границы; можно задавать в любом порядке.

Описание

Границы входят в диапазон.

Пример

```
if ( !$ISINRANGE(l_port, 1, 65535) )
    return  STS$K_ERROR;
```

__util$isinrange2, $ISINRANGE2 – Проверка значения на принадлежность замкнутому диапазону (вторая форма)

Формат

```
inline static int __util$isinrange2 (int x, int __min__, int __max__ );

#define $ISINRANGE2(x, __min__, __max__)    __util$isinrange2(x, (__min__), (__max__))
```

Возвращает

`int`: 1, если ***min*** <= *x* <= ***max***, иначе 0.

Аргументы

Как у `__util$isinrange()`.

Описание

Границы входят в диапазон, как в `__util$isinrange()`, и могут задаваться в любом порядке. В 1.6.11 эта рутина исключала границы, вопреки комментарию заголовка, где говорилось, что они включены; теперь код делает то, что сказано в комментарии. Программа, полагавшаяся на строгую проверку, должна написать её вручную: `(x > lo) && (x < hi)`.

__util$range, $RANGE – Ограничение значения диапазоном

Формат

```
inline static int __util$range (int x, int  __min__, int __max__ );

#define $RANGE(x, __min__, __max__) __util$range(x, __min__, __max__)
```

Возвращает

`int`: *x*, если оно в диапазоне; иначе ближайшая граница.

Аргументы

Как у `__util$isinrange()`.

Пример

```
l_timeout = $RANGE(l_timeout, 1, 3600);
```

2.11 Списки элементов

ILE2, ILE3 – Типы элемента списка элементов

Формат

```
typedef struct  __ile2  {
    unsigned short  code, len;
    char *  ptr;
} ILE2;

typedef struct  __ile3  {
    unsigned short  code, len;
    char *  ptr;
    unsigned short * retlen;
} ILE3;
```

Описание

Список элементов (item list) - массив элементов, оканчивающийся элементом из нулей. *code* говорит, что это за элемент, *len* - длина буфера по адресу *ptr*. В `ILE3` *retlen* - адрес слова, которое получает фактически возвращённую длину.

Раскладка не такая, как в OpenVMS. `ILE3` OpenVMS начинается со слова длины, а код стоит вторым; в этом `ILE3` **код стоит первым**, а длина вторым. Оба слова 16-разрядные, поэтому размер тот же, но список, построенный здесь, нельзя отдавать системной службе OpenVMS как есть (или коду, написанному для той раскладки). То же относится к `ILE2`. Типы служат программам, определяющим списки элементов; рутины этой библиотеки, которые их принимают, описаны вместе с этими рутинами.

$ILE2INI, $ILE3INI, $ILENULL – Макросы списка элементов

Формат

```
#define $ILE2INI(code, len, ptr)        {code, len, (char *)ptr}
#define $ILE3INI(a_ile, a_code, a_len, a_ptr, a_retlen) do {           \
        (a_ile).code = (unsigned short) (a_code);           \
        (a_ile).len = (unsigned short) (a_len);         \
        (a_ile).ptr = (char *) (a_ptr);             \
        (a_ile).retlen = (a_retlen);                    \
    } while (0)
#define $ILENULL    {0}
```

Аргументы

- *code* – `unsigned short`; код элемента.
- *len* – `unsigned short`; длина буфера.
- *ptr* – адрес; буфер.
- *a_ile* – только `$ILE3INI()`: объект `ILE3`, переменная или элемент массива. Не указатель на него.
- *a_code*, *a_len*, *a_ptr*, *a_retlen* – только `$ILE3INI()`: код, длина, буфер и `unsigned short *` для возвращаемой длины.

Описание

`$ILE2INI()` - инициализатор `ILE2`. `$ILENULL` - инициализатор завершающего элемента. `$ILE3INI()` - оператор, заполняющий объект `ILE3`: он пишется как `$ILE3INI(l_item, 1, sizeof(l_name), l_name, &l_ret);` с точкой с запятой и может стоять там, где стоит оператор, например в ветви `if` без фигурных скобок. В 1.6.11 он не компилировался с обычными аргументами, потому что его параметры были названы как устанавливаемые им поля; теперь параметры называются `a_ile`, `a_code`, `a_len`, `a_ptr` и `a_retlen`, и каждый приводится к типу поля.

Пример

```
char l_name [32];

ILE2 l_list [] = {
    $ILE2INI(1, sizeof(l_name), l_name),
    $ILENULL
};

ILE3 l_item;
unsigned short l_ret;

$ILE3INI(l_item, 1, sizeof(l_name), l_name, &l_ret);
```

2.12 Помощники времени

Рутины времени главы 3 работают с `fao_time_t` (секунды). Эти работают со `struct timespec` и `struct tm`. Рутина `s___time()` (часы, используемые рутинами журнала, `CLOCK_REALTIME_COARSE` в Linux) - `static inline` в заголовке и в интерфейс не входит.

__util$timbuf – Разложение местного времени

Формат

```
inline  static void __util$timbuf   (time_t *__src, struct tm * __dst);
```

Возвращает

Ничего.

Аргументы

- *__src* – `time_t *`; только чтение; по ссылке. Время либо нуль для текущего момента.
- *__dst* – `struct tm *`; только запись; по ссылке. Результат.

Описание

Преобразует в местное время через `localtime_r()` (`localtime_s()` в Windows), а затем меняет два поля на календарные значения: *tm_mon* - от 1 до 12, а *tm_year* - полный год, 2026. Это отличается от обычной `struct tm`; не передавайте результат в `mktime()` или `strftime()`.

Пример

```
struct tm l_tm;

__util$timbuf(NULL, &l_tm);
$LOG(STS$K_INFO, "%04d-%02d-%02d", l_tm.tm_year, l_tm.tm_mon, l_tm.tm_mday);
```

__util$add_time – Сложение двух времён

Формат

```
inline static void __util$add_time(const struct timespec* time1, const struct timespec*
↪  time2, struct timespec* result_time);
```

Возвращает

Ничего.

Аргументы

- *time1*, *time2* – `const struct timespec *`; только чтение; по ссылке. Времена, с *tv_nsec* от 0 до 999999999.
- *result_time* – `struct timespec *`; только запись; по ссылке. Сумма; может совпадать с одним из операндов.

Описание

Складывает секунды и наносекунды и переносит наносекунды в секунды. Переполнение секунд не проверяется.

__util$sub_time – Вычитание двух времён

Формат

```
inline static void __util$sub_time(const struct timespec * time1, const struct timespec*
↪  time2, struct timespec* result_time);
```

Возвращает

Ничего.

Аргументы

- *time1*, *time2* – `const struct timespec *`; только чтение; по ссылке. Уменьшаемое и вычитаемое.
- *result_time* – `struct timespec *`; только запись; по ссылке. *time1* минус *time2*.

Описание

Вычитает секунды и наносекунды, поле за полем, и нормализует результат: *tv_nsec* лежит от 0 до 999999999, и когда наносекунд в *time1* меньше, чем в *time2*, занимается секунда. Отрицательная разность - это *tv_sec* меньше нуля при неотрицательном *tv_nsec*; значение равно *tv_sec* плюс *tv_nsec*/10^9. В 1.6.11 *tv_nsec* мог быть отрицательным.

__util$cmp_time – Сравнение двух времён

Формат

```
inline static int __util$cmp_time(const struct timespec * time1, const struct timespec*
 ↪  time2);
```

Возвращает

`int`: 0, если времена равны; -1, если *time1* раньше *time2*; 1, если позже. Не условное значение.

Аргументы

- *time1*, *time2* – `const struct timespec *`; только чтение; по ссылке. Времена.

Описание

Сравнивает секунды, а если они равны - наносекунды. Ничто не вычитается, так что результат не может переполниться. Интерфейс определяет только знак, но значение всегда -1, 0 или 1. В 1.6.11 результатом была разность секунд, преобразованная в `int`, которая могла иметь неверный знак для времён, отстоящих более чем на 68 лет.

Пример

```
if ( __util$cmp_time(&l_now, &l_deadline) >= 0 )
    $LOG(STS$K_WARN, "the deadline has passed");
```

2.13 Пустая рутина

__util$nope – Ничего не делать

Формат

```
static inline void __util$nope (void) {}
```

Возвращает

Ничего.

Описание

Пустая рутина. В сборке с `NDEBUG` макросы `$TRACE()` и `$IFTRACE()` раскрываются в её вызов, чтобы макрос оставался оператором. Она определена, только если определён `NDEBUG`, и только один раз (под защитой `__UTIL$_NOPE__`).

Прочие определения

Формат

```
#define likely(x)   __builtin_expect((x),1)
#define unlikely(x) __builtin_expect((x),0)
#define __PATH_SEPARATOR    '/'     /* '\\' on Windows */
#define __PATH_SEPARATOR_S  "/"     /* "\\" on Windows */
```

Описание

`likely()` и `unlikely()` - подсказки компилятору о ветвлении; в Windows они раскрываются в свой аргумент. Они определяются, только если программа не определила их сама. `__PATH_SEPARATOR` и `__PATH_SEPARATOR_S` - разделитель частей пути к файлу, как символ и как строка; программа тоже может определить их до включения заголовка. # Глава 3. Службы FAO и времени

В этой главе описаны форматировщик и процедуры времени модуля STARLET (`starlet.h`, `starlet.c`), а также вспомогательные средства работы со временем, определённые в `utility_routines.h`. Модулю не нужна никакая процедура библиотеки времени выполнения Си, кроме `time()`. Он не использует `<stdio.h>`, `<string.h>`, `<ctype.h>`, `localtime()` и `gmtime()`.

Процедура	Назначение
`__util$fao`	Форматирование управляющей строки по списку параметров переменной длины
`__util$faol`	Форматирование управляющей строки по массиву параметров
`$FAO_DESC`	Инициализация выходного дескриптора над символьным массивом
`__util$gettim`	Получение текущего двоичного времени
`__util$bintim`	Преобразование строки ASCII со временем в двоичное время
`__util$asctim`	Преобразование двоичного времени в строку ASCII
`__util$add_times`	Сложение двух двоичных времён
`__util$sub_times`	Вычитание двух двоичных времён
`__util$add_time`	Сложение двух значений `struct timespec`
`__util$sub_time`	Вычитание двух значений `struct timespec`
`__util$cmp_time`	Сравнение двух значений `struct timespec`
`__util$timbuf`	Разбор `time_t` в `struct tm` с естественными номерами года и месяца

3.1 Соглашения процедур FAO

Выходной дескриптор. Форматировщик пишет в `fao_desc_t`. Используются только два поля:

```
typedef struct __fao_desc
{
    unsigned short dsc$w_length;   /* capacity of the buffer, bytes     */
    unsigned char  dsc$b_dtype;    /* not interpreted                   */
    unsigned char  dsc$b_class;    /* not interpreted                   */
    char          *dsc$a_pointer;  /* address of the buffer             */
} fao_desc_t;
```

`dsc$w_length` – ёмкость буфера. Процедуры никогда её не меняют. Длина результата возвращается через отдельный аргумент `outlen`. Результат не завершается нулём. Ёмкость не может превышать 65535 байт.

Параметры. Каждый параметр – это `fao_prm_t`, то есть `uint64_t`, передаваемый ПО ЗНАЧЕНИЮ. Это верно для всех размеров и видов директив: байта, слова, длинного слова, квадрослова, значения времени и указателя. Указатель передаётся как `(fao_prm_t)(uintptr_t) ptr`. При вызове `__util$fao` вызывающий обязан привести каждый параметр к `fao_prm_t`. Непривёденный `int` повышается до `int`, а не до 64-битного значения, и форматировщик прочтёт неверную ширину. Макросы `$LOG_FAO` и `$PUTMSG_FAO` из `utility_routines.h` выполняют приведение и считают параметры за вас (см. главу 2).

Код завершения. Статус «липкий»: он устанавливается при возникновении ошибки и возвращается в конце. Побеждает первая ошибка; более поздняя её не заменяет. Значения – это условные значения в раскладке OpenVMS (серьёзность в битах 0-2, номер сообщения выше, средство не задано), все чётные, и начиная с версии 1.7.0 они стабильны. Они приведены в приложении A.

Имя	Значение	Смысл
`STS$K_SUCCESS`	1	Преобразование завершено
`FAO$K_BUFOVF`	0xA	Результат усечён до ёмкости буфера
`FAO$K_BADDIR`	0x12	Директива неизвестна, искажена или оборвана концом управляющей строки
`FAO$K_IVTIME`	0xC	Неверная строка времени или неверная пара операндов (процедуры времени)
`FAO$K_NEGTIM`	8	Результат вычитания времён был бы отрицательным
`FAO$K_NOPRM`	0x1A	Управляющей строке нужно больше параметров, чем есть
`FAO$K_FLDOVF`	0x22	Текст поля `!n<...!>` оказался длиннее `FAO$K_FLDBUF` и был обрезан
`STS$K_ERROR`	2	Обязательный аргумент равен NULL

Все эти значения, кроме `STS$K_SUCCESS`, чётные, поэтому проверка `1 & status` для них

ложна. В версии 1.7.0 номер изменил только FAO$K_BADDIR (раньше было 6); FAO$K_NOPRM и FAO$K_FLDOVF – новые. Программа должна сравнивать по именам, а не по числам.

Ограничения.

Имя	Значение	Смысл
FAO$K_MAXPRM	64	Число параметров, кэшируемых __util$fao; параметр с индексом 64 и выше читается как 0
FAO$K_MAXDEPTH	4	Глубина вложенности групп !n(...!) и !n<...!>
FAO$K_FLDBUF	512	Наибольшая длина текста поля !n<...!> (starlet.h)

Параметры времени компиляции starlet.c.

Макрос	По умолчанию	Действие
FAO$T_TIME	time_t	Тип, на котором основан fao_time_t; в автономной (freestanding) сборке определите его до starlet.h
FAO$GETTIME()	time(NULL)	Источник текущего времени
FAO$K_TZOFFSET	0	Секунды, прибавляемые к UTC директивами !%T, !%D и __util$asctim и вычитаемые __util$bintim

Часовой пояс фиксируется при компиляции starlet.c. Во время выполнения его изменить нельзя.

3.2 Директивы FAO

Директива начинается с `!`. Её общий вид:

```
!  [ n | # ]  directive
```

`n` – десятичное число. `#` берёт число из списка параметров: он потребляет один параметр (читаемый как 32-битное беззнаковое значение) перед параметрами самой директивы. Число является шириной поля, счётчиком повторений или счётчиком пропуска – как определяет директива. Директива, не использующая число, его игнорирует.

В таблице «Парам.» – число параметров, которые потребляет директива. «n» – необязательное число.

Директива	Парам.	Вывод	Действие n
`!AS`	1	Строка, описанная `fao_desc_t *`; длина из `dsc$w_length`	Ширина поля
`!AZ`	1	Строка, завершённая нулём (`char *`)	Ширина поля
`!AC`	1	Счётная строка (`char *`); первый байт – длина	Ширина поля
`!AD`	2	Строка; параметры – длина, затем адрес	Ширина поля
`!AF`	2	Как `!AD`, но байт вне 0x20..0x7E показывается как .	Ширина поля
`!UB !UW !UL !UQ`	1	Беззнаковое десятичное 8, 16, 32, 64 бит	Ширина поля
`!SB !SW !SL !SQ`	1	Знаковое десятичное 8, 16, 32, 64 бит	Ширина поля
`!XB !XW !XL !XQ`	1	Шестнадцатеричное, верхний регистр, с нулями до 2, 4, 8, 16 цифр	Ширина поля
`!OB !OW !OL !OQ`	1	Восьмеричное, с нулями до 3, 6, 11, 22 цифр	Ширина поля
`!ZB !ZW !ZL !ZQ`	1	Беззнаковое десятичное, с нулями до n цифр	Ширина поля
`!n*c`	0	Символ `c` (следующий байт управляющей строки) n раз	Счётчик повторений, по умолчанию 1
`!n( … !)`	0	Заключённые директивы, повторяемые	Счётчик повторений, по умолчанию 1
`!n< … !>`	0	Заключённый текст как поле	Ширина поля

Директива	Парам.	Вывод	Действие n
`!n-`	0	Ничего; возвращает указатель параметров на n позиций назад	Счётчик, по умолчанию 1
`!n+`	0	Ничего; пропускает следующие n параметров	Счётчик, по умолчанию 1
`!/`	0	Возврат каретки и перевод строки (два байта, `\r\n`)	Игнорируется
`!_`	0	Горизонтальная табуляция	Игнорируется
`!^`	0	Перевод страницы	Игнорируется
`!!`	0	Символ `!`	Игнорируется
`!%S`	0	`s`, если последнее преобразованное число не равно 1, иначе ничего	Игнорируется
`!%T`	1	Время суток `hh:mm:ss.00` из двоичного времени	Игнорируется
`!%D`	1	Дата и время `dd-MMM-yyyy hh:mm:ss.00` из двоичного времени	Игнорируется
`!%M`	1	MAC-адрес `aa:bb:cc:dd:ee:ff` по указателю на 6 октетов	Игнорируется
`!%I`	1	Адрес IPv4 `d.d.d.d` по указателю на 4 октета	Игнорируется
`!%6`	1	Адрес IPv6 в форме RFC 5952 по указателю на 16 октетов	Игнорируется
`!%U`	1	UUID `xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx` по указателю на 16 октетов	Игнорируется

Директива, которой нет в таблице, устанавливает `FAO$K_BADDIR`. Форматировщик потребляет символы директивы, ничего за них не пишет и продолжает работу. Параметр он не потребляет. То же верно для директивы, оборванной концом управляющей строки: `"abc!"`, `"abc!5"`, `"a!A"`, `"a!%"` и `"a!3*"` дают текст `abc` или `a` и `FAO$K_BADDIR`; сканер останавливается на завершающем нуле и никогда не читает за ним.

3.2.1 Строковые директивы

`!AS`, `!AZ`, `!AC`, `!AD` и `!AF` копируют байты. Дескриптор NULL или указатель NULL ничего не пишет, но параметр всё равно потребляется (`!AD` и `!AF` сначала потребляют длину, затем адрес). При ширине поля n строка выравнивается по левому краю: если она

длиннее n, справа обрезается, а если короче – справа дополняется пробелами. `!0AS` ничего не пишет.

`!AS` берёт длину из `dsc$w_length` дескриптора. `!AC` читает байт длины как беззнаковое значение, так что строка может содержать до 255 байт. `!AD` и `!AF` читают длину как беззнаковое 32-битное значение.

`!AF` заменяет каждый байт вне диапазона 0x20..0x7E точкой. Ширина применяется после замены.

```
fao_desc_t d = { 4, 0, 0, "abcdef" };
/* !AS|!8AS|!2AS  with &d three times   ->  abcd|abcd    |ab        */
/* !AZ|!6AZ       with "hello" twice    ->  hello|hello              */
/* !AC            with "\003xyz"        ->  xyz                      */
/* !AD|!AF        with 3,"abcd",4,"a\001b\n"  ->  abc|a.b.          */
```

3.2.2 Числовые директивы

Вторая буква директивы задаёт основание, третья – размер. Буква размера: `B` (8 бит), `W` (16), `L` (32) или `Q` (64). Параметр – 64-битное значение. Сначала форматировщик усекает его до размера: старшие биты отбрасываются. Для `S` результат расширяется со знаком; для остальных букв – нулями. Так, `!SB` с параметром 200 печатает `-56`, а `!XB` с 0x1FF печатает `FF`. Любая другая третья буква устанавливает `FAO$K_BADDIR`, и директива пропускается; две буквы после `!` потребляются, но параметр – нет (`"x!ZZ y"` даёт `x y`).

Без ширины раскладка такова.

Директива	Раскладка
`!U`	Естественное число цифр
`!S`	Естественное число цифр, с ведущим `-`, если значение отрицательно
`!Z`	Естественное число цифр, без заполнения нулями (то же, что `!U`)
`!X`	Цифры верхнего регистра, с нулями до 2, 4, 8 или 16 цифр
`!O`	Цифры с нулями до 3, 6, 11 или 22 цифр

При ширине n результат занимает ровно n столбцов:

Директива	n больше числа цифр	n меньше числа цифр
`!U !S`	Пробелы слева	n звёздочек
`!Z`	Нули слева, до n цифр	n звёздочек
`!X !O`	Пробелы слева от числа, дополненного нулями	Левые цифры отбрасываются; остаются младшие n цифр

Обратите внимание на разницу между `!Z` и `!X`. `!5ZL` от 42 даёт `00042`. `!10XL` от 0xAB даёт `000000AB`: число сначала дополняется нулями до 8 цифр, а два лишних столбца заполняются пробелами. Знак `!S` занимает один столбец.

`!X` и `!O` рассматривают значение как битовый образ, поэтому знак они не печатают никогда.

Каждая числовая директива запоминает преобразованное значение (после усечения до размера и расширения знака) для директивы `!%S`.

```
/* !UL|!8UL|!2UL         with 123,123,123       ->  123|     123|**          */
/* !SL|!SB|!6SL          with -5,200,-5         ->  -5|-56|    -5            */
```

```
/* !XL|!XB|!4XL|!10XL    with 0xBEEF,0xA,0x12345678,0xAB
                                          ->  0000BEEF|0A|5678|  000000AB */
/* !OB|!OL               with 8,8            ->  010|00000000010          */
/* !ZL|!5ZL|!3ZB         with 42,42,7        ->  42|00042|007             */
```

3.2.3 Ширина из списка параметров: `!#`

`!#UL` берёт ширину из списка параметров: первый параметр – ширина, второй – значение. `__util$faol("!#UL", ..., {5, 77}, 2)` даёт `77`. То же работает для счётчика повторений (`!#(`), ширины поля `!#<`, счётчика повторений `!#*` и счётчиков `!#-` и `!#+`.

3.2.4 Группы повторения и группы фиксированной ширины

`!n(` текст `!)` повторяет текст n раз (по умолчанию 1; при 0 группа пропускается). Каждый проход потребляет следующие параметры. Чтобы использовать в проходе те же параметры снова, поместите в группу `!n-`. Конец группы – последовательность `!)`. Группа может содержать другие группы, до `FAO$K_MAXDEPTH` (4) уровней. Группа глубже этого или группа без терминатора устанавливает `FAO$K_BADDIR`; затем форматировщик продолжает с байта после открывающей последовательности, так что остальной текст читается как обычный текст и директивы.

```
/* !3(!UL,!)   with 1,2,3   ->  1,2,3,                                  */
/* !2(!UL!)    nested inside !3<...!>:  !3<!2(!UL!)!>  with 1,2  ->  "12 " */
```

`!n<` текст `!>` форматирует текст в рабочем буфере в 512 байт и затем записывает его как поле в n столбцов: по левому краю, с дополнением пробелами справа, с обрезкой справа. Без n поле имеет естественную ширину. Параметры разделяются с внешней строкой. Если текст поля длиннее `FAO$K_FLDBUF` (512 байт), он обрезается там и устанавливается `FAO$K_FLDOVF`; поле не является обрезкой выходного буфера, поэтому это не `FAO$K_BUFOVF`. Любой другой статус внутренней части (например, `FAO$K_BADDIR`) передаётся вызывающему.

```
/* !5<ab!>|!5<abcdefg!>|!<x!>   ->  ab   |abcde|x                       */
```

3.2.5 Повтор символа: `!n*c`

`!n*c` записывает байт `c` n раз. `c` – байт, следующий в управляющей строке за `*`; это может быть любой байт. Без n записывается один. `!3*-` записывает `---`. Если управляющая строка заканчивается сразу после `*`, ничего не пишется, а статус – `FAO$K_BADDIR`.

3.2.6 Указатель параметров: `!n-` и `!n+`

Форматировщик хранит индекс следующего параметра. `!n-` уменьшает индекс на n (по умолчанию 1), но не ниже 0, так что те же параметры используются снова. `!n+` увеличивает индекс на n (по умолчанию 1), так что n параметров пропускаются. В `__util$faol` индекс отсчитывается по массиву; параметр за концом массива читается как 0 и устанавливает `FAO$K_NOPRM`. В `__util$fao` первые 64 параметра кэшируются, чтобы `!n-` мог вернуться к ним; параметр с индексом 64 и выше читается как 0 и тоже устанавливает `FAO$K_NOPRM`. `__util$fao` читает нужные параметры из списка аргументов и не имеет счётчика для проверки, поэтому вызывающий должен передать не меньше параметров, чем потребляет управляющая строка (недостающий читается из стека, и его значение не определено). Используйте `__util$faol` или считающие макросы `$LOG_FAO` и `$PUTMSG_FAO`, когда управляющая строка не является вашим литералом.

```
/* !UL !-!UL     with 7     ->  7 7        */
```

3.2.7 Множественное число: !%S

!%S записывает s, если последнее число, преобразованное !U, !S, !Z, !X или !O, не равно 1; если оно равно 1, не пишет ничего. Если до этого число не преобразовывалось, значение равно 0 и директива пишет s. Параметр она не потребляет. Сравнение выполняется над 64-битным значением после усечения до размера: !UB с 257 оставляет значение 1.

```
/* "!UL file!%S"  with 1  ->  1 file          with 2  ->  2 files       */
```

3.2.8 Время: !%T и !%D

Параметр – двоичное время (см. раздел 3.3), передаваемое по значению. Его знак определяет, что показывается:

- Положительное значение – абсолютное время, число секунд с 01-JAN-1970 00:00:00 UTC. Прибавляются FAO$K_TZOFFSET секунд.
- Значение 0 означает «время не задано»: показывается текущее время. Саму эпоху показать нельзя; секунду после неё – можно.
- Отрицательное значение – дельта-время в столько секунд. !%D показывает его как дни, пробел и hh:mm:ss.00 (1 01:01:01.00 для -90061); !%T показывает hh:mm:ss.00 и отбрасывает дни.

!%T записывает hh:mm:ss.00, 11 байт. !%D записывает dd-MMM-yyyy hh:mm:ss.00, 23 байта. День дополняется слева пробелом, если он меньше 10 (6-JAN-1970). Месяц – английское сокращение в верхнем регистре. Год имеет четыре цифры до 9999; более поздний год записывается полностью (31-DEC-18029), и тогда текст длиннее 23 байт. Сотые доли всегда равны 00, так как двоичное время имеет разрешение одну секунду. Ширина игнорируется. Абсолютное время до эпохи выразить нельзя, поскольку отрицательное значение есть дельта-время.

```
/* !%T with 86399        ->  23:59:59.00                              */
/* !%D with 1700000000   ->  14-NOV-2023 22:13:20.00                  */
/* !%D with 86400*5      ->   6-JAN-1970 00:00:00.00                  */
/* !%D with -90061       ->  1 01:01:01.00      (a delta of 1 d 1 h 1 m 1 s)  */
```

3.2.9 Директивы адресов: !%M, !%I, !%6, !%U

Эти четыре директивы принимают указатель на «сырые» октеты в сетевом порядке. Процедура пишет текст независимо от ширины. Указатель NULL пишет (null), и параметр потребляется. Текст строится в локальном буфере и затем записывается целиком, так что результат, не помещающийся в выходной буфер, обрезается справа и устанавливается FAO$K_BUFOVF.

Директива	Читает	Форма	Пример
!%M	6 октетов	Две шестнадцатеричные цифры нижнего регистра на октет, через :	00:1b:21:3c:4d:5e
!%I	4 октета	Десятичные, без ведущих нулей, через .	192.168.1.10
!%6	16 октетов	Каноническая форма RFC 5952 (ниже)	2001:db8::1
!%U	16 октетов	Группы 8-4-4-4-12, шестнадцатеричные нижнего регистра	550e8400-e29b-41d4-a716-446655440000

Правила `!%6` следуют RFC 5952:

- Каждая из восьми 16-битных групп записывается в нижнем регистре, без ведущих нулей. Нулевая группа – `0`.
- Самая длинная серия из двух и более нулевых групп заменяется на `::`. Если две серии одинаковой длины, заменяется левая. Одиночная нулевая группа не заменяется.
- Адрес, отображённый из IPv4 (десять нулевых октетов, затем `ff ff`, затем четыре октета), записывается как `::ffff:`, за которым следует IPv4-адрес с точками, например `::ffff:192.168.1.10`. Вывод тот же, что у `inet_ntop(AF_INET6, ...)`. Адрес в IPv4-совместимой форме (`::a.b.c.d`) записывается шестнадцатеричными группами, например `::c0a8:10a`.
- Адрес петли – `::1`; неопределённый адрес – `::`.

3.2.10 Буквальные символы

`!!` записывает `!`. `!/` записывает два байта CR и LF. `!_` записывает табуляцию. `!^` записывает перевод страницы. Любой текст вне директив копируется как есть.

3.2.11 Переполнение выходного буфера

Когда результат не помещается в буфер, форматировщик записывает столько байт, сколько помещается, устанавливает `FAO$K_BUFOVF` и идёт до конца управляющей строки, так что остальные статусы по-прежнему обнаруживаются. Запись за пределы ёмкости буфера никогда не происходит. Если ранее в строке случилась ошибка, отличная от переполнения, прежний статус сохраняется: побеждает первая ошибка.

__util$fao – Форматированный вывод ASCII (список параметров)

Процедура форматированного вывода ASCII форматирует список параметров переменной длины под управлением управляющей строки FAO.

Формат

```
int __util$fao (const char *a_ctlstr, unsigned short *a_outlen,
                fao_desc_t *a_outbuf, ...);
```

Возвращает

int, только запись, по значению. Условное значение.

Аргументы

Аргумент	Тип	Доступ	Механизм	Описание
a_ctlstr	const char *	только чтение	по ссылке	Управляющая строка, завершённая нулём
a_outlen	unsigned short *	только запись	по ссылке	Получает число записанных байт. Может быть NULL
a_outbuf	fao_desc_t *	только чтение (дескриптор), только запись (буфер)	по ссылке	Дескриптор выходного буфера; dsc$w_length – ёмкость, dsc$a_pointer не должен быть NULL
...	fao_prm_t	только чтение	по значению	Параметры, каждый приведён к fao_prm_t

Описание

__util$fao сканирует управляющую строку, преобразует директивы из раздела 3.2 и записывает результат в буфер. Результат не завершается нулём. Дескриптор не изменяется; длина возвращается в a_outlen.

Параметры читаются из списка аргументов, когда они нужны директиве, и первые 64 сохраняются в кэше. Параметры, не потреблённые управляющей строкой, игнорируются. Управляющая строка, потребляющая больше параметров, чем передал вызывающий, читает несуществующие значения: результат не определён, потому что у списка переменных аргументов нет маркера конца, который процедура могла бы увидеть. Проверяется только предел кэша: параметр после 64-го читается как 0 и устанавливает FAO$K_NOPRM. Используйте __util$faol, когда число параметров известно только во время выполнения.

Если a_ctlstr, a_outbuf или a_outbuf->dsc$a_pointer равны NULL, процедура возвращает STS$K_ERROR и ничего не пишет, даже *a_outlen.

Возвращаемые условные значения

Значение	Смысл
STS$K_SUCCESS	Строка отформатирована
FAO$K_BUFOVF	Буфер слишком мал; результат усечён
FAO$K_BADDIR	Директива неизвестна, искажена или оборвана
FAO$K_FLDOVF	Текст поля !n<...!> обрезан на 512 байтах
FAO$K_NOPRM	Потребовалось более 64 параметров
STS$K_ERROR	Обязательный аргумент равен NULL

Когда в одном вызове случается несколько из них, возвращается первый найденный статус. Текст всё равно формируется.

Пример

```
#include "starlet.h"

char           buf [128];
fao_desc_t     dsc = $FAO_DESC (buf);
unsigned short len;
unsigned char  ip [4] = { 192, 168, 1, 10 };
int            sts;

sts = __util$fao ("peer !%I got !UL file!%S, mask !XL", &len, &dsc,
                  (fao_prm_t)(uintptr_t) ip, (fao_prm_t) 2, (fao_prm_t) 0xFF00);
if ( !(1 & sts) )
    return sts;

buf [len] = '\0';   /* the result is not zero-terminated */
/* buf: "peer 192.168.1.10 got 2 files, mask 0000FF00" */
```

__util$faol – Форматированный вывод ASCII (массив параметров)

Процедура форматированного вывода ASCII с массивом форматирует массив параметров под управлением управляющей строки FAO.

Формат

```
int __util$faol (const char *a_ctlstr, unsigned short *a_outlen,
                 fao_desc_t *a_outbuf,
                 const fao_prm_t *a_prmlst, int a_prmcnt);
```

Возвращает

`int`, только запись, по значению. Условное значение, как у `__util$fao`.

Аргументы

Аргумент	Тип	Доступ	Механизм	Описание
`a_ctlstr`	`const char *`	только чтение	по ссылке	Управляющая строка, завершённая нулём
`a_outlen`	`unsigned short *`	только запись	по ссылке	Получает число записанных байт. Может быть NULL
`a_outbuf`	`fao_desc_t *`	только чтение (дескриптор), только запись (буфер)	по ссылке	Дескриптор выходного буфера
`a_prmlst`	`const fao_prm_t *`	только чтение	по ссылке	Массив параметров. Может быть NULL; тогда каждый параметр читается как 0
`a_prmcnt`	`int`	только чтение	по значению	Число элементов в `a_prmlst`

Описание

`__util$faol` делает то же, что и `__util$fao`, но параметры берутся из массива. Директива, запрашивающая параметр за концом массива, получает 0, а статус – `FAO$K_NOPRM` (`"!UL !UL !UL|"` с двумя параметрами даёт `1 2 0|`). Счётчик явный; в OpenVMS список не считается. Поскольку массив читается без кэша, ограничения в 64 параметра нет. Повторное использование параметров через `!n-` и пропуск через `!n+` работают по индексу массива.

Возвращаемые условные значения

Значение	Смысл
`STS$K_SUCCESS`	Строка отформатирована
`FAO$K_BUFOVF`	Буфер слишком мал; результат усечён

Значение	Смысл
FAO$K_BADDIR	Директива неизвестна, искажена или оборвана
FAO$K_FLDOVF	Текст поля !n<...!> обрезан на 512 байтах
FAO$K_NOPRM	Управляющей строке потребовалось больше a_prmcnt параметров
STS$K_ERROR	Обязательный аргумент равен NULL

Пример

```
char           buf [64];
fao_desc_t     dsc = $FAO_DESC (buf);
unsigned short len;
fao_prm_t      prm [2] = { 5, 77 };

__util$faol ("[!#UL]", &len, &dsc, prm, 2);    /* buf: "[   77]", len 7 */
```

$FAO_DESC – Инициализация выходного дескриптора

Формат

```
#define $FAO_DESC(buf)   { (unsigned short) sizeof(buf), 0, 0, (buf) }
```

Возвращает

Инициализатор в фигурных скобках для `fao_desc_t`. Это не выражение, поэтому его можно использовать только при инициализации.

Аргументы

Аргумент	Тип	Доступ	Механизм	Описание
`buf`	`char[N]`	только запись	по ссылке	Символьный массив, а не указатель: `sizeof(buf)` – это ёмкость

Описание

Если вместо массива передан указатель, `sizeof` даст размер указателя, и ёмкость будет неверной. Чтобы присвоить значение уже существующему дескриптору, используйте составной литерал: `dsc = (fao_desc_t) $FAO_DESC (buf);`. В `starlet.h` нет макроса `$FAO`; процедура вызывается напрямую или через `$LOG_FAO` и `$PUTMSG_FAO` из `utility_routines.h`.

Возвращаемые условные значения

Нет.

Пример

```
char       buf [256];
fao_desc_t dsc = $FAO_DESC (buf);       /* capacity 256 */
```

3.3 Формат двоичного времени

Процедуры времени этого модуля НЕ используют формат OpenVMS (квадрослово из 100-наносекундных тиков с 17-NOV-1858). Двоичное время здесь – это `fao_time_t`, то есть `time_t`, если только `FAO$T_TIME` не определён иначе:

- Значение больше 0 – АБСОЛЮТНОЕ время: число секунд с 01-JAN-1970 00:00:00 UTC. Часовой пояс, зафиксированный `FAO$K_TZOFFSET`, прибавляется при показе времени и вычитается при его чтении.
- Значение 0 для арифметических процедур является эпохой, но для процедур вывода означает «время не задано», то есть сейчас.
- Значение меньше 0 – ДЕЛЬТА-время: его модуль – длина интервала в секундах.
- Разрешение – одна секунда. Сотые доли секунды принимаются в строке времени, но отбрасываются, а время всегда показывается с `.00`.

Следствия:

- Этот формат используют `!%T`, `!%D`, `__util$gettim`, `__util$bintim` и `__util$asctim`, так что время без потерь переходит между ними туда и обратно. Ни одна процедура не преобразует его в формат OpenVMS.
- Абсолютное время до 01-JAN-1970 выразить нельзя, потому что отрицательное значение – это дельта-время. Начиная с версии 1.7.0 `__util$bintim` отвергает такую дату (`FAO$K_IVTIME`; «17-NOV-1858» – ошибка), а не возвращает значение, которое было бы прочитано обратно как дельта-время.
- Значение 0 – эпоха. Это допустимое абсолютное время для `__util$add_times` и `__util$sub_times` (как первый операнд, с дельта-временем), но `!%T`, `!%D` и `__util$asctim` читают 0 как «текущее время», а `__util$bintim` возвращает 0 для «01-JAN-1970 00:00:00» в поясе UTC.
- На целевой платформе, где `time_t` 32-битный, диапазон заканчивается в январе 2038 года.
- Год даты анализатором не ограничивается 4 цифрами, и год выше 9999 показывается полностью. Дата за пределами диапазона типа не обнаруживается.

Процедуры `__util$add_time`, `__util$sub_time` и `__util$cmp_time` из `utility_routines.h` используют другое представление: POSIX `struct timespec` (секунды и наносекунды).

__util$gettim – Получение двоичного времени

Формат

```
int __util$gettim (fao_time_t *a_timadr);
```

Возвращает

int, только запись, по значению. Условное значение.

Аргументы

Аргумент	Тип	Доступ	Механизм	Описание
a_timadr	fao_time_t *	только запись	по ссылке	Получает текущее время

Описание

Сохраняет в *a_timadr текущее время как число секунд с 01-JAN-1970 00:00:00 UTC. Смещение часового пояса не применяется. Источник – FAO$GETTIME(), по умолчанию time(NULL).

Возвращаемые условные значения

Значение	Смысл
STS$K_SUCCESS	Время сохранено
STS$K_ERROR	a_timadr равен NULL

Пример

```
fao_time_t now;

if ( 1 & __util$gettim (&now) )
    __util$fao ("started !%D", &len, &dsc, (fao_prm_t) now);
```

__util$bintim – Преобразование времени ASCII в двоичное время

Формат

```
int __util$bintim (const char *a_timstr, fao_time_t *a_timadr);
```

Возвращает

int, только запись, по значению. Условное значение.

Аргументы

Аргумент	Тип	Доступ	Механизм	Описание
a_timstr	const char *	только чтение	по ссылке	Строка времени, завершённая нулём
a_timadr	fao_time_t *	только запись	по ссылке	Получает двоичное время. При ошибке не изменяется

Описание

Пробелы и табуляции в начале и в конце строки игнорируются. Принимаются три формы.

Абсолютная дата: dd-MMM-yyyy[hh:mm[:ss[.cc]]]. Месяц – три буквы в любом регистре. День проверяется по длине месяца с учётом григорианского правила високосного года. Год – десятичное число как записано; двузначный год не расширяется. Пропущенные день или год заменяются днём или годом текущей даты, так что -JAN-2020 и 15-OCT- допустимы; дефисы остаются, а месяц остаётся обязательным (15-OCT недопустимо). Пропущенное время суток равно 00:00:00. Строка читается в поясе FAO$K_TZOFFSET.

Абсолютное время суток: hh:mm[:ss[.cc]]. Дата – текущая. Поле времени суток можно опустить: ::10 – это 00:00:10, а 12:30 – 12:30:00.

Дельта-время: dddd hh:mm[:ss[.cc]] или dddd-hh:mm[:ss[.cc]]. Число дней, затем пробел или дефис, затем время суток. Результат отрицателен: -(dddd*86400 + hh*3600 + mm*60 + ss). Время суток без дней, например 12:30, – это абсолютное время, а не дельта-время. Для дельта-времени 12 часов 30 минут используйте 0 12:30.

Ограничения: часы от 0 до 23, минуты от 0 до 59, секунды от 0 до 59, сотые от 0 до 99. Число содержит не более 9 цифр. Сотые проверяются и отбрасываются. У дельта-времени нет иного ограничения на число дней, кроме диапазона типа.

Абсолютная дата до 01-JAN-1970 отвергается с FAO$K_IVTIME: отрицательное значение было бы прочитано как дельта-время (раздел 3.3). Дата 01-JAN-1970 00:00:00 в поясе UTC даёт 0.

Возвращаемые условные значения

Значение	Смысл
STS$K_SUCCESS	Строка преобразована
FAO$K_IVTIME	Синтаксическая ошибка, поле вне диапазона, лишний текст в конце или абсолютная дата до 1970 года

Значение	Смысл
STS$K_ERROR	a_timstr или a_timadr равен NULL

Пример

```
fao_time_t t;

__util$bintim ("15-OCT-2018 10:20:30.55", &t);   /* t == 1539598830 (zone UTC) */
__util$bintim ("1 02:03:04", &t);                /* t == -93784 (a delta)      */
__util$bintim ("31-FEB-2020", &t);               /* FAO$K_IVTIME, t unchanged  */
__util$bintim ("17-NOV-1858", &t);               /* FAO$K_IVTIME, t unchanged  */
```

__util$asctim – Преобразование двоичного времени в ASCII

Формат

```
int __util$asctim (unsigned short *a_outlen, fao_desc_t *a_outbuf,
                   const fao_time_t *a_timadr, int a_cvtflg);
```

Возвращает

`int`, только запись, по значению. Условное значение.

Аргументы

Аргумент	Тип	Доступ	Механизм	Описание
`a_outlen`	`unsigned short *`	только запись	по ссылке	Получает длину результата. Может быть NULL
`a_outbuf`	`fao_desc_t *`	только чтение (дескриптор), только запись (буфер)	по ссылке	Выходной буфер
`a_timadr`	`const fao_time_t *`	только чтение	по ссылке	Время. Если NULL, используется текущее время
`a_cvtflg`	`int`	только чтение	по значению	0: полная форма. Любое другое значение: только время суток

Описание

Время	`a_cvtflg` = 0	`a_cvtflg` не равен нулю
Абсолютное (> 0)	`dd-MMM-yyyy hh:mm:ss.00` (23 байта; день дополнен пробелом)	`hh:mm:ss.00` (11 байт)
Дельта-время (< 0)	`d hh:mm:ss.00` (дни без дополнения)	`hh:mm:ss.00`, дни отброшены
Ноль или `a_timadr` равен NULL	Текущее время как абсолютное	Текущее время суток

Вывод выполняет тот же отображатель, что и для `!%D` и `!%T`, поэтому они всегда согласованы. Результат не завершается нулём. Вывод принимается `__util$bintim`, за исключением того, что значение 0 печатает текущее время, а не 01-JAN-1970, а год выше 9999 записывается полностью, так что текст длиннее.

Возвращаемые условные значения

Значение	Смысл
`STS$K_SUCCESS`	Время преобразовано
`FAO$K_BUFOVF`	Буфер слишком мал; результат усечён

Значение	Смысл
STS$K_ERROR	a_outbuf или указатель на его буфер равен NULL

Пример

```
char           buf [32];
fao_desc_t     dsc = $FAO_DESC (buf);
unsigned short len;
fao_time_t     t = -93784;

__util$asctim (&len, &dsc, &t, 0);    /* "1 02:03:04.00", len 13 */
```

__util$add_times – Сложение двух двоичных времён

Формат

```
int __util$add_times (const fao_time_t *a_tim1, const fao_time_t *a_tim2,
                      fao_time_t *a_result);
```

Возвращает

int, только запись, по значению. Условное значение.

Аргументы

Аргумент	Тип	Доступ	Механизм	Описание
a_tim1	const fao_time_t *	только чтение	по ссылке	Первый операнд
a_tim2	const fao_time_t *	только чтение	по ссылке	Второй операнд
a_result	fao_time_t *	только запись	по ссылке	Получает сумму. При ошибке не изменяется

Описание

Операнд >= 0 является абсолютным; операнд < 0 – дельта-временем. Правила:

Операнды	Результат
дельта + дельта	Дельта-время: сумма (отрицательных) значений
абсолютное + дельта	Абсолютное время: абсолютное значение плюс модуль дельта-времени
абсолютное + абсолютное	Ошибка FAO$K_IVTIME

Переполнение типа не проверяется.

Возвращаемые условные значения

Значение	Смысл
STS$K_SUCCESS	Сумма сохранена
FAO$K_IVTIME	Оба операнда абсолютные
STS$K_ERROR	Аргумент равен NULL

Пример

```
fao_time_t t0 = 1000, d = -100, r;

__util$add_times (&t0, &d, &r);     /* r == 1100 */
```

__util$sub_times – Вычитание двух двоичных времён

Формат

```
int __util$sub_times (const fao_time_t *a_tim1, const fao_time_t *a_tim2,
                      fao_time_t *a_result);
```

Возвращает

int, только запись, по значению. Условное значение.

Аргументы

Аргумент	Тип	Доступ	Механизм	Описание
a_tim1	const fao_time_t *	только чтение	по ссылке	Уменьшаемое
a_tim2	const fao_time_t *	только чтение	по ссылке	Вычитаемое
a_result	fao_time_t *	только запись	по ссылке	Получает разность. При ошибке не изменяется

Описание

Операнды	Результат
абсолютное - абсолютное	Дельта-время -(tim1 - tim2); FAO$K_NEGTIM, если tim1 раньше tim2
абсолютное - дельта	Абсолютное время: tim1 минус модуль дельта-времени; FAO$K_NEGTIM, если результат был бы меньше 0
дельта - дельта	Дельта-время tim1 - tim2 (большее дельта-время минус меньшее); FAO$K_NEGTIM, если модуль tim1 меньше модуля tim2
дельта - абсолютное	Ошибка FAO$K_IVTIME

Если два абсолютных времени равны, результат равен 0, что является абсолютным временем, а не дельта-временем нулевой длины.

Возвращаемые условные значения

Значение	Смысл
STS$K_SUCCESS	Разность сохранена
FAO$K_NEGTIM	Результат был бы отрицательным
FAO$K_IVTIME	Дельта-время минус абсолютное время
STS$K_ERROR	Аргумент равен NULL

Пример

```
fao_time_t a = 1000, b = 400, r;

__util$sub_times (&a, &b, &r);      /* r == -600: a delta of 600 seconds */
```

__util$add_time – Сложение двух значений timespec

Эта процедура и следующие три – встраиваемые (inline) функции `utility_routines.h`. Они работают со значениями POSIX `struct timespec`, а не с `fao_time_t`.

Формат

```
inline static void __util$add_time (const struct timespec *time1,
                                    const struct timespec *time2,
                                    struct timespec *result_time);
```

Возвращает

Ничего.

Аргументы

Аргумент	Тип	Доступ	Механизм	Описание
`time1`	`const struct timespec *`	только чтение	по ссылке	Первый операнд
`time2`	`const struct timespec *`	только чтение	по ссылке	Второй операнд
`result_time`	`struct timespec *`	только запись	по ссылке	Получает сумму

Описание

Складывает секунды и наносекунды и переносит целые секунды из наносекунд. У операндов `tv_nsec` должно лежать в диапазоне от 0 до 999999999. Адрес результата может совпадать с адресом операнда. Переполнение `tv_sec` не проверяется.

Возвращаемые условные значения

Нет.

Пример

```
struct timespec a = { 1, 700000000 }, b = { 2, 500000000 }, r;

__util$add_time (&a, &b, &r);       /* r = { 4, 200000000 } */
```

__util$sub_time – Вычитание двух значений timespec

Формат

```
inline static void __util$sub_time (const struct timespec *time1,
                                    const struct timespec *time2,
                                    struct timespec *result_time);
```

Возвращает

Ничего.

Аргументы

Аргумент	Тип	Доступ	Механизм	Описание
time1	const struct timespec *	только чтение	по ссылке	Уменьшаемое
time2	const struct timespec *	только чтение	по ссылке	Вычитаемое
result_time	struct timespec *	только запись	по ссылке	Получает time1 - time2

Описание

Вычитает секунды и наносекунды и нормализует результат: tv_nsec всегда лежит в диапазоне от 0 до 999999999, а когда наносекунды time1 меньше наносекунд time2, одна секунда занимается из tv_sec. До версии 1.7.0 результат оставался ненормализованным, с отрицательным tv_nsec. Отрицательная разность (time1 раньше time2) имеет отрицательный tv_sec и неотрицательный tv_nsec, как это обычно и бывает у значений timespec.

Возвращаемые условные значения

Нет.

Пример

```
struct timespec a = { 5, 100 }, b = { 3, 400 }, r;

__util$sub_time (&a, &b, &r);       /* r = { 1, 999999700 }: normalized */
```

__util$cmp_time – Сравнение двух значений timespec

Формат

```
inline static int __util$cmp_time (const struct timespec *time1,
                                   const struct timespec *time2);
```

Возвращает

`int`, только запись, по значению. Ноль (0), если времена равны, 1, если `time1` позже `time2`, -1, если `time1` раньше.

Аргументы

Аргумент	Тип	Доступ	Механизм	Описание
`time1`	`const struct timespec *`	только чтение	по ссылке	Первое время
`time2`	`const struct timespec *`	только чтение	по ссылке	Второе время

Описание

Сначала сравниваются секунды, затем наносекунды; ничего не вычитается, поэтому переполнения быть не может. Результат в точности равен -1, 0 или 1. До версии 1.7.0 процедура возвращала разность, усечённую до `int`, знак которой мог меняться на обратный для времён, отстоящих более чем на 68 лет; вызывающий, проверяющий знак (`< 0`, `> 0`), этим не затрагивается.

Возвращаемые условные значения

Нет. Возвращаемое значение не является условным значением.

Пример

```
struct timespec deadline = { 100, 0 }, now = { 100, 5 };

if ( __util$cmp_time (&deadline, &now) < 0 )
    handle_timeout ();       /* deadline is earlier than now: cmp_time gave -1 */
```

__util$timbuf – Разбор времени в календарную структуру

Формат

```
inline static void __util$timbuf (time_t *__src, struct tm *__dst);
```

Возвращает

Ничего.

Аргументы

Аргумент	Тип	Доступ	Механизм	Описание
`__src`	`time_t *`	только чтение	по ссылке	Преобразуемое время. Если NULL, используется текущее время
`__dst`	`struct tm *`	только запись	по ссылке	Получает разобранное ЛОКАЛЬНОЕ время

Описание

Процедура вызывает `localtime_r()` (`localtime_s()` в Windows), а затем прибавляет 1 к `tm_mon` и 1900 к `tm_year`. Таким образом, `tm_mon` лежит в диапазоне от 1 до 12, а `tm_year` – полный год. Это отличается от соглашения для `struct tm`, и результат нельзя передавать в `mktime()` или `asctime()`. Часовой пояс – локальный пояс процесса, в отличие от процедур `starlet.h`, которые используют UTC и фиксированный `FAO$K_TZOFFSET`.

Возвращаемые условные значения

Нет.

Пример

```
struct tm tm;

__util$timbuf (NULL, &tm);
/* tm.tm_year == 2026, tm.tm_mon == 10 in October */
```

3.4 FAO в вызовах журналирования и сообщений

Макросы `$LOG_FAO` и `$PUTMSG_FAO` из `utility_routines.h` передают управляющую строку FAO и её параметры в `__util$faol`. Они считают параметры (до 16) и приводят каждый к `fao_prm_t`, так что вызов выглядит как `$LOG_FAO (STS$K_INFO, "if=!AZ rx=!UL peer=!%I", "eth0", 777, l_ip4);`. Они описаны в главе 2. Процедура `__util$faohex`, форматирующая до 16 октетов в одну строку шестнадцатеричного и ASCII-текста, не является директивой FAO; она также описана в главе 2.

3.5 Отличия от OpenVMS

Пункт	OpenVMS	STARLET
Двоичное время	Тики по 100 нс с 17-NOV-1858	Секунды с 01-JAN-1970 UTC; дельта-время отрицательно
Параметры `SYS$FAOL`	Длинные слова	`fao_prm_t`, 64 бита, квадрослова и времена по значению
Число параметров в `$FAOL`	Не задаётся	Задаётся `a_prmcnt`
Длина вывода	Слово в дескрипторе	Аргумент `outlen`; дескриптор не изменяется
Параметр `!%T`, `!%D`	Адрес квадрослова; 0 = текущее время	Время по значению; 0 = текущее время
Сотые доли секунды	Реальные	Всегда `00`
`!%M !%I !%6 !%U`	Нет в FAO	Добавлены

3.6 Изменения в версии 1.7.0

В редакции 1.6.11 настоящего руководства перечислялись дефекты этого модуля. Они исправлены. Что может заметить потребитель:

1. Управляющая строка, оканчивающаяся внутри директивы (`"ab!"`, `"!A"`, `"!%"`, `"!5"`, `"!n*"`), больше не заставляет сканер читать за завершающим нулём. Директива устанавливает `FAO$K_BADDIR`, и сканирование останавливается в конце строки.
2. Значения `FAO$K_*` – это условные значения. `FAO$K_BADDIR` теперь равен 0x12 (был 6). `FAO$K_NOPRM` (0x1A) и `FAO$K_FLDOVF` (0x22) – новые. Побеждает первая ошибка; раньше – последняя.
3. `__util$faol` устанавливает `FAO$K_NOPRM`, когда управляющей строке нужно больше параметров, чем `a_prmcnt`; `__util$fao` устанавливает его для параметра за 64-м. Текст по-прежнему формируется.
4. Поле `!n<...!>` длиннее 512 байт обрезается с `FAO$K_FLDOVF`, а не молча.
5. `!%D`, `!%T` и `__util$asctim` используют один отображатель. Отрицательное время – дельта-время, 0 – «сейчас», а год выше 9999 записывается полностью, а не читается за пределами таблицы.
6. `__util$bintim` отвергает абсолютную дату до 01-JAN-1970 с `FAO$K_IVTIME`.
7. `__util$sub_time` возвращает нормализованный `timespec`; `__util$cmp_time` возвращает -1, 0 или 1 и не может переполниться.
8. `starlet.h` сам включает `utility_routines.h`, так что порядок двух включений больше не имеет значения, а его запасные определения `STS$K_*` удалены.

В 1.7.1 `$LOG_FAO`, `$PUTMSG_FAO` и их отладочные формы добавляют `[FAO BADDIR]`, `[FAO NOPRM]`, `[FAO FLDOVF]` или `[FAO BUFOVF]` к записи, текст которой отформатирован нечисто; возвращаемое значение не меняется (глава 2, `__util$log_fao`).

Что по-прежнему верно, по замыслу: время – это число секунд эпохи UNIX, а не квадрослово OpenVMS; эпоху саму показать нельзя, потому что 0 означает «сейчас»; `__util$fao` не может обнаружить, что вызывающий передал слишком мало параметров, поскольку список переменных аргументов Си не несёт счётчика.

Глава 4. Процедуры командного языка

В этой главе описаны процедуры CLI из `cli_routines.h` и `cli_routines.c`. Они разбирают командную строку на глагол, до восьми позиционных параметров и произвольное число квалификаторов, проверяют значения и вызывают процедуру действия глагола. Построение следует утилите определения команд (Command Definition Utility) OpenVMS: грамматика – это набор таблиц, которые определяет программа.

Грамматика такова:

```
verb [subverb] [p1 p2 ... p8] [/qualifier[=value] ...]
```

Квалификатор можно записывать и в форме GNU: `--qualifier[=value]`. Одиночный дефис (`-name`) квалификатором не является. Имена сопоставляются без учёта регистра, и имя можно сократить до любого префикса, не являющегося неоднозначным.

Модуль включает `utility_routines.h` и `starlet.h`. Он использует `ASC` – счётную строку из `utility_routines.h` – и `__util$bintim` для типа `CLI$K_TIME`.

Процедура	Назначение
`__cli$parse`	Разбор вектора аргументов по таблицам и построение контекста
`__cli$dispatch`	Вызов процедуры действия найденного глагола
`__cli$get_value`	Получение значения параметра или квалификатора
`__cli$present`	Проверка, был ли задан параметр или квалификатор, и в какой форме
`__cli$cleanup`	Освобождение контекста
`__cli$show_verbs`	Вывод таблицы глаголов в журнал
`__cli$get_keyword`	Приведение значения `CLI$K_KWD` к его записи в таблице ключевых слов
`__cli$help`	Вывод экрана справки по таблицам
`__cli$show_ctx`	Вывод контекста в журнал

4.1 Таблицы команд

Грамматику описывают четыре структуры. Таблица – массив, заканчивающийся завершающей записью; для каждого вида таблицы это указано в следующих разделах.

4.1.1 ASC: счётная строка

```
typedef struct _asc {
    unsigned char len;        /* length of the text, 0 to 255         */
    char          sts[255];   /* the text; need not be zero-terminated */
} ASC;
```

Макросы `$ASCINI("text")` (инициализатор полей `len` и `sts`), `$ASCLEN(p)`, `$ASCPTR(p)` и `$ASC(p)` (длина и указатель, для преобразования `%.*s`) относятся к `utility_routines.h` (глава 2). Значения контекста сохраняются с помощью `__util$str2asc`, которая оставляет место под завершающий нуль: текст длиннее 254 байт обрезается.

4.1.2 CLI_KEYWORD: ключевое слово

```
typedef struct __cli_keyword__ {
    ASC                name;   /* the keyword                          */
    unsigned long long val;    /* the value associated with it         */
} CLI_KEYWORD;
```

Список ключевых слов заканчивается записью, у которой `name.len` равно 0 (`{0}`).

4.1.3 CLI_PQDESC: параметр или квалификатор

```
typedef struct __cli_pqdesc__ {
    ASC            name;     /* name of the parameter or qualifier      */
    unsigned short type;     /* CLI$K_FILE ... CLI$K_MAC                */
    unsigned char  pn;       /* CLI$K_P1 ... CLI$K_P8; CLI$K_QUAL       */
    unsigned char  flag;     /* CLI$M_NEGATABLE, _LIST, _REQUIRED       */
    ASC            defval;   /* default value, empty string = none      */
    CLI_KEYWORD   *kwd;      /* keyword list for CLI$K_KWD, or NULL     */
} CLI_PQDESC;
```

Поле	Описание
`name`	Для квалификатора – имя, которое пользователь набирает после `/` или `--`. Для параметра – метка для экрана справки и сообщений
`type`	Тип значения. См. раздел 4.2
`pn`	Для параметра – его позиция, от `CLI$K_P1` до `CLI$K_P8` (1-8). Для квалификатора – по соглашению `CLI$K_QUAL` (9); анализатор его не читает
`flag`	Комбинация флагов `CLI$M_*` из раздела 4.3
`defval`	Значение, которое `__cli$get_value` возвращает, когда элемент отсутствует. Не проверяется по `type`
`kwd`	Список ключевых слов элемента `CLI$K_KWD`

Таблица ПАРАМЕТРОВ заканчивается записью, у которой `pn` равно 0. Таблица КВАЛИФИКАТОРОВ заканчивается записью, у которой `name.len` равно 0. Запись

`{ .name = {0} }` завершает оба вида таблиц. Параметры должны идти в порядке их значений `pn`, а то, какое слово командной строки связывается с параметром, определяется позицией в таблице, а не значением `pn`.

4.1.4 CLI_VERB: глагол

```
typedef struct __cli__verb__ {
    ASC                 name;      /* the verb                              */
    struct __cli__verb__ *next;    /* table of sub-verbs, or NULL           */
    CLI_PQDESC          *params;   /* parameters, or NULL                   */
    CLI_PQDESC          *quals;    /* qualifiers, or NULL                   */
    int (*act_rtn)(struct __cli_ctx__ *clictx, void *act_arg);
    void                *act_arg;  /* handed to the action routine          */
} CLI_VERB;
```

Поле	Описание
`name`	Глагол, сопоставляется без учёта регистра
`next`	Указатель на ТАБЛИЦУ подглаголов (массив, заканчивающийся записью с пустым именем). Имя `next` вводит в заблуждение: это не ссылка на следующий глагол той же таблицы. Глагол с `next` – контейнер; его собственные `params`, `quals` и `act_rtn` не используются
`params`	Позиционные параметры глагола или NULL
`quals`	Квалификаторы глагола или NULL
`act_rtn`	Процедура действия. Получает контекст и `act_arg` и возвращает условное значение
`act_arg`	Значение для процедуры действия, например код команды, приведённый к `void *`

Таблица глаголов заканчивается записью, у которой `name.len` равно 0.

4.1.5 Контекст

```
typedef struct __cli_item__ {
    struct __cli_item__ *next;
    unsigned            type;    /* 0 = verb; 1..8 = P1..P8; 9 = qualifier */
    unsigned char       state;   /* CLI$M_PRESENT, _NEGATED, _CONSUMED     */
    union { CLI_VERB *verb; CLI_PQDESC *pqdesc; };
    ASC                 val;     /* the value as typed (a keyword: in full) */
} CLI_ITEM;

typedef struct __cli_ctx__ {
    int       opts;              /* CLI$M_OPTRACE | CLI$M_OPSIGNAL         */
    CLI_ITEM *vlist;             /* the verb and sub-verbs, in order       */
    CLI_ITEM *avlist;            /* parameters and qualifiers, in order    */
} CLI_CTX;
```

Контекст строит `__cli$parse`. Программа в него не пишет. `avlist` содержит по одному элементу на каждый параметр и на каждое вхождение квалификатора (и по одному

на каждый элемент списочного квалификатора) в порядке командной строки. Флаги состояния приведены в разделе 4.3.

4.2 Типы значений

Значение параметра или квалификатора проверяется по полю `type` перед сохранением. Проверка выполняется, только если значение не пусто (у `/Q=` и `/Q` значения нет, и проверка пропускается). В таблице «ошибка» – это `STS$K_ERROR`.

Константа	Значение	Проверка
`CLI$K_FILE`	1	Нет; любой текст
`CLI$K_DATE`	2	Строгая. `dd-mm-yyyy` (одна или две цифры для дня и месяца, до четырёх для года, год не менее 1), затем необязательно разделитель (`-`, пробел или `T`) и `hh:mm` либо `hh:mm:ss`. Проверяются месяц, день (с правилом високосного года) и время суток; текст после значения – ошибка
`CLI$K_NUM`	3	Беззнаковое число: `strtoull` с основанием 0 должна потребить весь текст без ошибки диапазона: десятичное, восьмеричное (ведущий 0) и шестнадцатеричное (ведущие 0x). Знак минус или ведущий пробел – ошибка
`CLI$K_IPV4`	4	`inet_pton(AF_INET)` должна завершиться успешно (строгая десятичная запись с точками)
`CLI$K_IPV6`	5	`inet_pton(AF_INET6)` должна завершиться успешно
`CLI$K_OPT`	7	Нет; флаг. Значение, если задано, сохраняется и не проверяется
`CLI$K_QSTRING`	8	Нет. Кавычки, если есть, входят в текст; оболочка снимает их раньше
`CLI$K_UUID`	9	Ровно 36 символов вида 8-4-4-4-12 из шестнадцатеричных цифр
`CLI$K_DEVICE`	0xA	`stat()` для `/dev/<значение>` должна завершиться успешно; значение, уже начинающееся с `/dev/`, используется как есть
`CLI$K_KWD`	0xB	Значение должно выбирать одно ключевое слово из списка `kwd` (раздел 4.5); сохраняется полное ключевое слово. Если списка нет, принимается что угодно

Константа	Значение	Проверка
CLI$K_TIME	0xC	__util$bintim должна принять текст: абсолютная дата, время суток или дельта-время
CLI$K_MAC	0xD	Шесть шестнадцатеричных октетов из одной или двух цифр, разделённых : или -, и ничего больше

Значение 6 не используется. CLI$K_QUAL тоже равно 9, но это вид элемента, а не тип значения, и они никогда не смешиваются. Текст значения сохраняется так, как был набран, кроме значения CLI$K_KWD, которое сохраняется как полное ключевое слово из таблицы. Проверка не преобразует значение: преобразует его программа (с помощью strtoull, inet_pton, __util$bintim), когда использует.

4.3 Флаги и параметры

Флаги дескриптора (`CLI_PQDESC.flag`).

Константа	Значение	Смысл
`CLI$M_NEGATABLE`	0x01	У квалификатора есть и форма `/NO<name>`
`CLI$M_LIST`	0x02	Квалификатор принимает список значений: `/Q=(a,b,c)` или `/Q=a,b,c`
`CLI$M_REQUIRED`	0x10	Параметр или квалификатор должен быть задан

Состояние элемента (`CLI_ITEM.state`, возвращается `__cli$present`).

Константа	Значение	Смысл
`CLI$M_PRESENT`	0x04	Элемент был задан в командной строке
`CLI$M_NEGATED`	0x08	Он был задан в форме `/NO<name>`
`CLI$M_CONSUMED`	0x20	Значение уже возвращено `__cli$get_value` (списочные квалификаторы)

Параметры обработки (`__cli$parse`, `opts`).

Константа	Значение	Смысл
`CLI$M_OPTRACE`	1	Записать трассу разбора с помощью `$IFTRACE`
`CLI$M_OPSIGNAL`	2	Сообщать о каждой ошибке разбора в журнал через `$LOG`. Без него процедуры молчат и лишь возвращают статус. Ответ «отсутствует» от `__cli$get_value` ошибкой не является и не сообщается никогда

Размеры. `CLI$SSZ_MAXVERB` (32) – число начальных символов командного слова, которые сравниваются с глаголами. `ASC$K_SZ` (255) – размер контейнера значения; сам текст не длиннее 254 байт.

4.4 Правила разбора

1. `argv[0]` – глагол. Это первое слово после имени программы, поэтому `main` передаёт `argc - 1` и `argv + 1`.
2. Если `argv[0]` – запрос справки (`/HELP`, `-HELP`, `--help`, `/H`, `-h`, `--h` в любом регистре), печатается обзор всех глаголов, и разбор возвращает `STS$K_WARN`. Одиночное `help` – обычное слово, поэтому программа может определить собственный глагол `HELP`.
3. Глагол сопоставляется с таблицей по правилам DCL, так же как `__util$lookup_key` в режиме сокращений. Глагол, набранный полностью, побеждает: при глаголах `SET` и `SETUP` слово `SET` выбирает `SET`. Иначе слово должно быть префиксом одного глагола (`SETU` выбирает `SETUP`). Если это префикс двух разных глаголов, разбор завершается с `STS$K_FATAL` («Ambiguous»). Записи глаголов-синонимов – с одной и той же процедурой действия и аргументом, одними и теми же таблицами параметров, квалификаторов и подглаголов – считаются одним глаголом, так что два написания команды никогда не делают сокращение неоднозначным. Запись, у которой нет ничего из перечисленного, ничьим синонимом не является. Сравниваются только первые `CLI$SSZ_MAXVERB` (32) символов слова.
4. Если любое слово после глагола – запрос справки (`/HELP`, `-h`, `--help`), печатается экран справки глагола, и разбор возвращает `STS$K_WARN` до проверки каких-либо значений. Это относится к последнему глаголу вложенной команды: `SHOW STATISTICS /HELP` показывает справку по `STATISTICS` (см. раздел 4.6).
5. Если у глагола есть таблица подглаголов (`next`), следующее слово сопоставляется с этой таблицей по правилу 3. Иначе последующие слова связываются с параметрами в порядке таблицы.
6. Параметр берёт следующее слово, со следующими исключениями: сканирование параметров останавливается на слове, являющемся квалификатором глагола (`/name` или `--name`, сокращённом или нет, даже неоднозначном). Поэтому необязательный параметр можно опустить в пользу квалификатора. Слово, начинающееся с `--` и не являющееся ни одним из квалификаторов глагола, – ошибка (`Unrecognized qualifier`), всегда. Так же и слово, начинающееся с `/` и не совпадающее ни с одним квалификатором – если только ожидающий значения параметр не имеет тип `CLI$K_FILE` или `CLI$K_DEVICE`: тогда слово – это путь (`/tmp`, `/etc/passwd`) и становится значением этого параметра. Таким образом, опечатка в квалификаторе никогда не принимается за значение текстового параметра.
7. Остальные слова должны быть квалификаторами. Слово, не являющееся квалификатором (не начинается с `/` или `--`), завершает разбор с `STS$K_FATAL`. Если оно следует за квалификатором, записанным без `=`, сообщение говорит, что значение должно следовать за `=` (`/blocks 5` – ошибка, `/blocks=5` – верно); иначе это «Too many parameters», лишнее слово.
8. Квалификатор – это `/name`, `--name`, `/name=value` или `--name=value`. Имя сопоставляется с таблицей глагола: точное имя побеждает; иначе принимается префикс, подходящий одному квалификатору; префикс, подходящий нескольким, завершается с `STS$K_FATAL`. Если ничего не совпало, а имя начинается с `NO` и длиннее двух букв, остаток сопоставляется только с квалификаторами, имеющими `CLI$M_NEGATABLE`; совпадение – это отрицаемый квалификатор, у которого значения нет. Неизвестный квалификатор завершает разбор с `STS$K_FATAL`.
9. Значение квалификатора `CLI$M_LIST` разбивается по запятым; одна необязательная пара скобок снимается. Каждый элемент проверяется и сохраняется как отдельный элемент, до 254 байт каждый. Запятая внутри кавычек не защищена.
10. Когда квалификатор повторяется, сохраняется каждое вхождение. `__cli$present` сообщает о последнем. `__cli$get_value` возвращает первое (или, для списка, вхождения по очереди).
11. После того как слова израсходованы, каждый отсутствующий квалификатор и параметр с `CLI$M_REQUIRED` завершает разбор с `STS$K_FATAL`.
12. Глагол, у которого нет ни параметров, ни квалификаторов, не принимает после себя слов: лишнее слово завершает разбор с `STS$K_FATAL` («Too many parameters»), как это делает DCL.

Результат – условное значение первой ошибки. Неудачный разбор освобождает контекст и устанавливает `*clictx` в NULL.

4.5 Ключевые слова

Список ключевых слов элемента `CLI$K_KWD` просматривается функцией `strncasecmp` по набранному тексту. Текст короче ключевого слова совпадает с ним как сокращение. Точное совпадение целого ключевого слова побеждает сокращения (так что `INET` можно набрать полностью, когда существует `INET6`). Если точного совпадения нет и с текста начинаются два или более ключевых слова, разбор завершается с `STS$K_FATAL` («Ambiguous»); если ни одного – с `STS$K_ERROR`. Пустой текст – ошибка.

Значение, которое сохраняет разбор, – ПОЛНОЕ ключевое слово из таблицы, в написании таблицы, а не сокращение или регистр, набранные пользователем: `/MODE=SL` сохраняется как `slow`. Поэтому `__cli$get_value` возвращает полное имя, и программа может сравнивать его обычным сравнением строк. Чтобы получить и поле `val` ключевого слова, вызовите `__cli$get_keyword`, которая возвращает запись таблицы. В версии 1.6.11 сохранялся набранный текст, и программе приходилось самой искать его в таблице.

4.6 Вложенные глаголы

У глагола-контейнера в `next` есть таблица подглаголов. Подглагол следует за родителем сразу, без квалификатора между ними. Квалификаторы после подглагола принадлежат подглаголу. Вложенность может быть любой глубины. `__cli$dispatch` вызывает процедуру действия последнего (самого глубокого) найденного глагола.

```
static CLI_PQDESC s_stat_q [] = {
    { .name = {$ASCINI("csv")}, .type = CLI$K_OPT, .pn = CLI$K_QUAL },
    { .name = {0} }
};
static CLI_VERB s_show_subs [] = {
    { .name = {$ASCINI("statistics")}, .act_rtn = s_act_stat, .quals = s_stat_q },
    { .name = {$ASCINI("version")},    .act_rtn = s_act_ver },
    { .name = {0} }
};
static CLI_VERB s_cmds [] = {
    { .name = {$ASCINI("show")}, .next = s_show_subs },
    { .name = {0} }
};
```

Слова `show statistics --csv` вызывают `s_act_stat`, и затем `__cli$present (ctx, &s_stat_q[0], &state)` сообщает о `--csv`. Слово `show nonesuch` завершается с `STS$K_FATAL`. Запрос справки, стоящий прямо после глагола-контейнера (`show /help`), показывает список его подглаголов. Запрос справки после подглагола (`show statistics /help`) показывает справку этого подглагола: подглагол владеет остатком командной строки. До версии 1.7.0 справку по подглаголу запросить было нельзя.

__cli$parse – Разбор командной строки

Формат

```
int __cli$parse (CLI_VERB *verbs, int opts, int argc, char **argv, void **clictx);
```

Возвращает

int, только запись, по значению. Условное значение.

Аргументы

Аргумент	Тип	Доступ	Механизм	Описание
verbs	CLI_VERB *	только чтение	по ссылке	Таблица глаголов
opts	int	только чтение	по значению	CLI$M_OPTRACE и CLI$M_OPSIGNAL либо 0
argc	int	только чтение	по значению	Число слов, включая глагол. Должно быть не менее 1
argv	char **	только чтение	по ссылке	Слова; argv[0] – глагол
clictx	void **	только запись	по ссылке	Получает адрес нового CLI_CTX

Описание

Выделяет контекст, сопоставляет argv с таблицами по правилам раздела 4.4, проверяет значения и сохраняет элементы. Сначала *clictx устанавливается в NULL, так что вызывающий, вызывающий __cli$cleanup после неудачи, в безопасности. При успехе *clictx указывает на CLI_CTX, который вызывающий освобождает через __cli$cleanup. При любом статусе, не являющемся нечётным, контекст освобождается внутри процедуры, а *clictx равен NULL.

Статус STS$K_WARN означает «была напечатана справка». Это не ошибка, и контекст равен NULL, но проверка 1 & status ложна (младший бит сброшен). Вызывающий, которому нужно отличить справку от сбоя, сравнивает статус с STS$K_WARN и завершается с успехом. В любом случае программа не должна выполнять диспетчеризацию.

Возвращаемые условные значения

Значение	Смысл
STS$K_SUCCESS	Командная строка верна
STS$K_WARN	Найден запрос справки и напечатан экран справки
STS$K_FATAL	argc меньше 1; нет памяти; неизвестный или неоднозначный глагол либо квалификатор; лишнее слово («Too many parameters»); значение после пробела вместо =; отсутствует обязательный элемент; неоднозначное ключевое слово
STS$K_ERROR	Значение не прошло проверку своего типа (ключевые слова: неизвестное ключевое слово)

Пример

```
void *ctx;
int   sts;

sts = __cli$parse (g_verbs, CLI$M_OPSIGNAL, argc - 1, argv + 1, &ctx);
if ( sts == STS$K_WARN )
    return 0;                       /* help was printed */
if ( !(1 & sts) )
    return 1;                       /* the error is in the log */

sts = __cli$dispatch (ctx);
__cli$cleanup (ctx);
```

__cli$dispatch – Вызов процедуры действия

Формат

```
int __cli$dispatch (CLI_CTX *clictx);
```

Возвращает

int, только запись, по значению. Значение, возвращённое процедурой действия, или статус самой процедуры.

Аргументы

Аргумент	Тип	Доступ	Механизм	Описание
clictx	CLI_CTX *	только чтение	по ссылке	Контекст, созданный __cli$parse. Не должен быть NULL

Описание

Берёт последний глагол в списке глаголов контекста (самый глубокий подглагол) и вызывает act_rtn (clictx, act_arg). Процедура действия читает свои значения через __cli$get_value и __cli$present. Глагол без процедуры действия даёт STS$K_WARN.

Возвращаемые условные значения

Значение	Смысл
(возвращаемое значение act_rtn)	Процедура действия была вызвана
STS$K_WARN	У глагола нет процедуры действия
STS$K_FATAL	В контексте нет глагола

Пример

```
static int s_act_copy (CLI_CTX *a_ctx, void *a_arg)
{
    ASC l_src;

    if ( !(1 & __cli$get_value (a_ctx, &g_copy_p[0], &l_src)) )
        return STS$K_ERROR;
    /* ... */
    return STS$K_SUCCESS;
}
```

__cli$get_value – Получение значения

Формат

```
int __cli$get_value (CLI_CTX *clictx, CLI_PQDESC *pq, ASC *val);
```

Возвращает

int, только запись, по значению. Условное значение.

Аргументы

Аргумент	Тип	Доступ	Механизм	Описание
clictx	CLI_CTX *	только чтение	по ссылке	Контекст
pq	CLI_PQDESC *	только чтение	по ссылке	Дескриптор элемента, ТОТ ЖЕ адрес, что у записи в разобранной таблице
val	ASC *	только запись	по ссылке	Получает копию значения. Может быть NULL

Описание

Процедура ищет pq (по адресу) в списке элементов. Если элемент найден:

- Для обычного элемента возвращается первое вхождение.
- Для квалификатора CLI$M_LIST возвращается следующий ещё не возвращённый элемент, который помечается как потреблённый. Вызывайте её в цикле до отказа, чтобы прочитать все элементы.
- Если сохранённое значение имеет нулевую длину (флаг вроде /VERBOSE, отрицаемый квалификатор или /Q=), процедура возвращает STS$K_WARN, а *val устанавливается в пустую строку. Для проверки флага используйте __cli$present.

Если элемента нет, val не равен NULL, а у дескриптора непустой defval, процедура возвращает значение по умолчанию с STS$K_SUCCESS. Списочный квалификатор, который был задан и прочитан до конца, значение по умолчанию НЕ возвращает; она завершается отказом, так что цикл заканчивается.

Если clictx или pq равны NULL, процедура пишет сообщение в журнал через $LOG независимо от opts и возвращает STS$K_FATAL. Отсутствующий элемент без значения по умолчанию даёт STS$K_ERROR. Этот ответ – обычный результат зондирования, и начиная с версии 1.7.0 он никогда не записывается в журнал, ни с CLI$M_OPSIGNAL, ни без него; раньше каждое зондирование отсутствующего элемента заносило ошибку в журнал. Пустое значение присутствующего элемента (STS$K_WARN) по-прежнему сообщается с CLI$M_OPSIGNAL.

Возвращаемые условные значения

Значение	Смысл
STS$K_SUCCESS	Значение возвращено (найденное или по умолчанию)
STS$K_WARN	Элемент присутствует, но его значение пусто

Значение	Смысл
`STS$K_ERROR`	Элемент отсутствует и не имеет значения по умолчанию; или элементы списка исчерпаны
`STS$K_FATAL`	`clictx` или `pq` равен NULL

Пример

```
ASC l_tag;

while ( 1 & __cli$get_value (ctx, &g_run_q[TAGS], &l_tag) )
    printf ("tag %.*s\n", $ASC (&l_tag));
```

__cli$get_keyword – Приведение значения ключевого слова

Процедура приведения значения ключевого слова находит запись таблицы ключевых слов, которой принадлежит значение CLI$K_KWD, чтобы программа могла прочитать поле val ключевого слова.

Формат

```
int __cli$get_keyword (CLI_CTX *clictx, CLI_PQDESC *pq, const ASC *val,
                       CLI_KEYWORD **kwd);
```

Возвращает

int, только запись, по значению. Условное значение.

Аргументы

Аргумент	Тип	Доступ	Механизм	Описание
clictx	CLI_CTX *	только чтение	по ссылке	Контекст
pq	CLI_PQDESC *	только чтение	по ссылке	Дескриптор элемента со списком ключевых слов в kwd
val	const ASC *	только чтение	по ссылке	Значение, возвращённое __cli$get_value, или NULL для первого значения элемента (его значения по умолчанию, если элемент отсутствует)
kwd	CLI_KEYWORD **	только запись	по ссылке	Получает адрес записи внутри таблицы pq. Сначала устанавливается в NULL

Описание

Разбор сохраняет полное ключевое слово из таблицы (раздел 4.5), поэтому значение ищется по полному имени без учёта регистра, и сокращения не принимаются. Затем *kwd указывает в таблицу ключевых слов дескриптора: имя – в (*kwd)->name, связанное с ним значение – в (*kwd)->val. При val, равном NULL, процедура берёт первое значение элемента или значение по умолчанию из дескриптора, если элемент отсутствует. Процедура не продвигает курсор квалификатора CLI$M_LIST, поэтому её можно вызывать между вызовами __cli$get_value.

В журнал ничего не пишется независимо от opts, кроме случая NULL у clictx, pq или kwd, который является ошибкой программирования.

Возвращаемые условные значения

Значение	Смысл
`STS$K_SUCCESS`	Запись найдена
`STS$K_ERROR`	У дескриптора нет списка ключевых слов; элемент отсутствует и не имеет значения по умолчанию; значение не является ключевым словом таблицы
`STS$K_WARN`	Элемент присутствует с пустым значением (`/mode`, `/mode=`); так же говорит `__cli$get_value`. `*kwd` равно NULL
`STS$K_FATAL`	`clictx`, `pq` или `kwd` равен NULL

Пример

Программа с квалификатором `/MODE` типа `CLI$K_KWD`, таблица которого содержит `fast` (1) и `slow` (2). Команда `run /mode=SL` даёт:

```
static CLI_KEYWORD s_modes [] = {
    { .name = {$ASCINI("fast")}, .val = 1 },
    { .name = {$ASCINI("slow")}, .val = 2 },
    { .name = {0} }
};
/* ... s_quals[0] is { "mode", CLI$K_KWD, .kwd = s_modes } ... */

ASC          l_val;
CLI_KEYWORD *l_kwd;

if ( 1 & __cli$get_value (ctx, &s_quals[0], &l_val) )
    printf ("stored: %.*s\n", $ASC (&l_val));          /* stored: slow          */

if ( 1 & __cli$get_keyword (ctx, &s_quals[0], NULL, &l_kwd) )
    printf ("keyword %.*s, val %llu\n", $ASC (&l_kwd->name), l_kwd->val);
                                                      /* keyword slow, val 2   */
```

__cli$present – Проверка наличия элемента

Формат

```
int __cli$present (CLI_CTX *clictx, CLI_PQDESC *pq, unsigned *state);
```

Возвращает

int, только запись, по значению. Условное значение.

Аргументы

Аргумент	Тип	Доступ	Механизм	Описание
clictx	CLI_CTX *	только чтение	по ссылке	Контекст
pq	CLI_PQDESC *	только чтение	по ссылке	Дескриптор элемента (тот же адрес, что в таблице)
state	unsigned *	только запись	по ссылке	Получает флаги состояния. Может быть NULL

Описание

Просматривает весь список элементов и использует ПОСЛЕДНЕЕ вхождение pq. Поэтому /HIGHLIGHT /NOHIGHLIGHT сообщается как отрицаемый. *state устанавливается в 0 для отсутствующего элемента, а иначе в CLI$M_PRESENT, с CLI$M_NEGATED, если последнее вхождение было /NO<name>, и CLI$M_CONSUMED, если оно было прочитано. Значение по умолчанию процедура не использует.

Возвращаемые условные значения

Значение	Смысл
STS$K_SUCCESS	Элемент присутствует, в утвердительной или отрицаемой форме
STS$K_WARN	Элемент отсутствует
STS$K_FATAL	clictx или pq равен NULL

Пример

```
unsigned st;

if ( (1 & __cli$present (ctx, &g_run_q[VERBOSE], &st)) && !(st & CLI$M_NEGATED) )
    l_verbose = 1;
```

__cli$cleanup – Освобождение контекста

Формат

```
int __cli$cleanup (CLI_CTX *clictx);
```

Возвращает

int, только запись, по значению. Всегда STS$K_SUCCESS.

Аргументы

Аргумент	Тип	Доступ	Механизм	Описание
clictx	CLI_CTX *	изменение	по ссылке	Контекст. Может быть NULL. После вызова использовать его нельзя

Описание

Освобождает элементы обоих списков и сам контекст. Таблицы не затрагиваются. Значения ASC, скопированные __cli$get_value, остаются действительными, потому что это копии.

Возвращаемые условные значения

Значение	Смысл
STS$K_SUCCESS	Всегда

Пример

```
__cli$cleanup (ctx);
ctx = NULL;
```

__cli$show_verbs – Вывод таблицы глаголов

Формат

```
void __cli$show_verbs (CLI_VERB *verbs, int level);
```

Возвращает

Ничего.

Аргументы

Аргумент	Тип	Доступ	Механизм	Описание
`verbs`	`CLI_VERB *`	только чтение	по ссылке	Таблица глаголов
`level`	`int`	только чтение	по значению	Уровень отступа первого глагола; 0 на верхнем уровне

Описание

Записывает глаголы, их подглаголы, параметры (с типом), ключевые слова (с `val` в шестнадцатеричном виде) и квалификаторы в журнал через `$LOG (STS$K_INFO, ...)`. Предназначена для отладки грамматики. На уровне 0 строка глагола выглядит как `Show verb 'NAME' :`; на более глубоком уровне это имя с отступом в два пробела на уровень. Таблица подглаголов выводится один раз, на уровень глубже родителя; уровень вызывающего не меняется, поэтому глаголы, следующие за глаголом-контейнером в той же таблице, записываются в той же форме, что и предшествующие ему.

Возвращаемые условные значения

Нет.

Пример

```
__cli$show_verbs (g_verbs, 0);
```

__cli$help – Вывод экрана справки

Формат

```
int __cli$help (CLI_VERB *verbs, CLI_VERB *verb);
```

Возвращает

int, только запись, по значению. Всегда STS$K_SUCCESS.

Аргументы

Аргумент	Тип	Доступ	Механизм	Описание
verbs	CLI_VERB *	только чтение	по ссылке	Верхняя таблица, используется, когда verb равен NULL
verb	CLI_VERB *	только чтение	по ссылке	Описываемый глагол или NULL для обзора

Описание

Текст идёт через __util$out, без префикса. __cli$parse вызывает эту процедуру, когда находит запрос справки; программа тоже может её вызвать.

- verb равен NULL: «Available commands:» и имя каждого глагола.
- У verb есть подглаголы: «Usage: NAME ...» и список подглаголов.
- Иначе – строка «Usage:» с параметрами (обязательные без скобок, остальные в квадратных скобках) и «[/qualifier ...]», если у глагола есть квалификаторы, затем список параметров (позиция, имя, тип) и квалификаторов. Строка квалификатора содержит «required», «/NO accepted», «list» и «default=...» в соответствии с флагами и defval. Строка usage обрезается до 512 байт.

Возвращаемые условные значения

Значение	Смысл
STS$K_SUCCESS	Всегда

Пример

```
__cli$help (g_verbs, NULL);           /* the list of commands    */
__cli$help (g_verbs, &g_verbs[1]);    /* the help of one command */
```

__cli$show_ctx – Вывод контекста

Формат

```
void __cli$show_ctx (CLI_CTX *clictx);
```

Возвращает

Ничего.

Аргументы

Аргумент	Тип	Доступ	Механизм	Описание
clictx	CLI_CTX *	только чтение	по ссылке	Показываемый контекст. Не должен быть NULL

Описание

Записывает список глаголов и значение каждого параметра (Pn) и квалификатора контекста в журнал через $LOG (STS$K_INFO, ...). Предназначена для отладки.

Возвращаемые условные значения

Нет.

Пример

```
__cli$show_ctx (ctx);
```

4.7 Изменения в версии 1.7.0

В редакции 1.6.11 настоящего руководства перечислялись дефекты этого модуля. Они исправлены. Что может заметить потребитель:

1. Глагол сопоставляется по правилам DCL: набранный полностью, он побеждает (`SET` против `SETUP`), уникальный префикс выбирает, синонимы никогда не делают префикс неоднозначным.
2. Глагол без параметров и квалификаторов отвергает лишнее слово («Too many parameters»).
3. Справку по подглаголу можно запросить (`SHOW STATISTICS /HELP`).
4. Значение `CLI$K_KWD` сохраняется как полное ключевое слово; новая `__cli$get_keyword` возвращает запись таблицы, так что `val` доступно.
5. Несопоставленное `/name` принимается за путь только для ожидающего параметра `CLI$K_FILE` или `CLI$K_DEVICE`; для любого другого параметра, и всегда для `--name`, это ошибка. Раньше оно становилось значением следующего необязательного параметра.
6. Лишнее слово даёт «Too many parameters» вместо «Unrecognized token»; значение после пробела (`/blocks 5`) имеет собственное сообщение и по-прежнему является ошибкой.
7. `CLI$K_DATE` и `CLI$K_NUM` строгие (раздел 4.2).
8. `__cli$get_value` не заносит в журнал ответ «отсутствует», поэтому зондирование тихое при `CLI$M_OPSIGNAL`.
9. `__cli$show_verbs` восстанавливает свой уровень, `__cli$show_ctx` объявлена в заголовочном файле, а комментарий к `CLI_PQDESC.flag` верен.

В 1.7.1 `__cli$get_keyword` возвращает `STS$K_WARN` для элемента, присутствующего с пустым значением (`/mode`, `/mode=`), как `__cli$get_value`; отсутствие по-прежнему даёт `STS$K_ERROR`.

Что по-прежнему верно, по замыслу: для `/blocks 5` нужен `=`; `__cli$parse` возвращает `STS$K_WARN` после печати экрана справки, и младший бит этого статуса сброшен, поэтому программа должна проверять `STS$K_WARN` до проверки на сбой; `__cli$dispatch` и процедуры, не названные в этой главе, не проверяют свой контекст на NULL.

Глава 5. Процедуры протокола TLV

В этой главе описан AVProto, протокол «атрибут/значение» (тег-длина-значение, Tag-Length-Value) из `avproto.h` и `avproto.c`. Блок данных протокола (PDU) – это заголовок фиксированного размера, за которым следует полезная нагрузка из записей TLV. Все числа в PDU хранятся в сетевом порядке байтов (старший байт первым), так что PDU может пересекать машины с любым порядком байтов. Процедуры называются `__tlv$...`.

Процедура	Назначение
`__tlv$hset`	Инициализация заголовка PDU
`__tlv$hget`	Проверка сигнатуры PDU и чтение его заголовка
`__tlv$encode_tag`	Упаковка типа тега и идентификатора тега в слово тега
`__tlv$decode_tag`	Распаковка слова тега
`__tlv$put`	Добавление записи TLV в PDU
`__tlv$get`	Поиск записи по тегу и копирование её значения
`__tlv$lookup`	Поиск записи по тегу и возврат адреса её значения
`__tlv$dump`	Вывод заголовка и записей PDU в журнал

`avproto.c` включает `utility_routines.h`; программы, использующие эти процедуры, компилируют и компонуют оба модуля. В Windows `avproto.h` предоставляет `be16toh`, `htobe16` и другие преобразования через функции `_byteswap_*`, потому что `<endian.h>` там отсутствует.

5.1 Раскладка PDU

PDU – это заголовок (20 октетов) и полезная нагрузка. Полезная нагрузка – последовательность записей без чего-либо между ними: ни заполнения, ни выравнивания. Поле `len` заголовка – длина полезной нагрузки в октетах, без заголовка.

```
 octet
 0       8       12      16      20
 +-------+-------+-------+-------+---------------------------------+
 | magic | len   | csr   | seq   | payload: TLV 0, TLV 1 ... TLV N |
 | 8     | 4     | 4     | 4     | len octets                      |
 +-------+-------+-------+-------+---------------------------------+
   сигнатура, дополненная нулями до 8 октетов
           |       |       +-- порядковый номер PDU
           |       +-- слово команды / статуса
           +-- длина полезной нагрузки, октеты (старший байт первым)
```

Поле	Размер	Описание
`magic`	8 октетов	Сигнатура протокола, строка до 8 октетов, дополненная нулями
`len`	4 октета	Длина полезной нагрузки. Старший байт первым. `__tlv$put` поддерживает его актуальным
`csr`	4 октета	Регистр команды / состояния (Command / Status Register): команда в запросе, статус в ответе. Старший байт первым
`seq`	4 октета	Порядковый номер PDU, помогающий сопоставлять ответы параллельных или асинхронных обменов. Старший байт первым

Типы Си: `AVPROTO_HDR` (заголовок), `AVPROTO_TLV` (заголовок записи с массивом нулевой длины для значения) и `AVPROTO_PDU` (заголовок, за которым следует массив записей нулевой длины). Они упакованы (`#pragma pack(1)`). Поля `u_len`, `u_csr` и `u_seq` заголовка и поля `w_tag` и `w_len` записи содержат значения в сетевом порядке; преобразуйте их через `be32toh` и `be16toh` или читайте через процедуры.

5.1.1 Запись

```
 bit   0          3 4                   15 16                          31
       +----------+----------------------+------------------------------+
       |   type   |          id          |            length            |
       |  4 bits  |       12 bits        |           16 bits            |
       +----------+----------------------+------------------------------+
       |  value: `length` octets (none if length is 0)                  |
       +-----------------------------------------------------------------+
        |<------- w_tag (16 bits) ------->|<-------- w_len (16 bits) ----->|
```

Оба 16-битных слова хранятся в сетевом порядке байтов. Длина считает только октеты значения. Следующая запись начинается сразу после значения. Таким

образом, запись имеет длину 4 + `length` октетов, а значение может содержать до 65535 октетов.

Тег. `w_tag` содержит тип в 4 старших битах и идентификатор в 12 младших. Маски – `TAG$M_TYPE` (0xF000) и `TAG$M_ID` (дополнение к 0xF000). Идентификатор (от 0 до 4095) выбирает приложение. Тип сообщает, как преобразовывать значение.

Константа	Значение	Значение записи
`TAG$K_BBLOCK`	0	Блок октетов; без преобразования. Тип по умолчанию
`TAG$K_WORD`	3	16-битное число, старший байт первым, длина 2
`TAG$K_LONGWORD`	4	32-битное число, старший байт первым, длина 4
`TAG$K_QWORD`	5	64-битное число, старший байт первым, длина 8
`TAG$K_UUID`	8	16 октетов, без преобразования, длина 16
`TAG$K_MAX`	9	Маркер конца списка перечисления; не тип

У номеров типов 1, 2, 6, 7 и с 9 по 15 нет имён. `__tlv$put` и `__tlv$get` обращаются с ними как с `TAG$K_BBLOCK`.

5.1.2 Пример

Вызовы

```
__tlv$hset (pdu, (const unsigned char *) "TSTSIG", 0x1234, 42);
__tlv$put  (pdu, sizeof pdu, 1,   TAG$K_WORD,   &w,   sizeof w);    /* w == 0xABCD */
__tlv$put  (pdu, sizeof pdu, 100, TAG$K_BBLOCK, blob, 3);           /* 01 02 03   */
```

строят следующий PDU из 33 октетов:

```
 offset  octets                      meaning
 ------  --------------------------  ----------------------------------
      0  54 53 54 53 49 47 00 00      magic "TSTSIG" + два нулевых октета
      8  00 00 00 0D                  len = 13
     12  00 00 12 34                  csr = 0x1234
     16  00 00 00 2A                  seq = 42
     20  30 01                        тег: тип 3 (WORD), id 1
     22  00 02                        длина 2
     24  AB CD                        значение 0xABCD
     26  00 64                        тег: тип 0 (BBLOCK), id 100
     28  00 03                        длина 3
     30  01 02 03                     значение
```

__tlv$hset – Установка заголовка PDU

Формат

```
inline static int __tlv$hset (void *a_pdup, const unsigned char *a_sig,
                              unsigned a_u_csr, unsigned a_u_seq);
```

Возвращает

`int`, только запись, по значению. Условное значение.

Аргументы

Аргумент	Тип	Доступ	Механизм	Описание
`a_pdup`	`void *`	только запись	по ссылке	Буфер PDU; не менее 20 октетов
`a_sig`	`const unsigned char *`	только чтение	по ссылке	Сигнатура, строка из 1-8 октетов, завершённая нулём
`a_u_csr`	`unsigned`	только чтение	по значению	Слово команды/статуса в порядке хоста
`a_u_seq`	`unsigned`	только чтение	по значению	Порядковый номер в порядке хоста

Описание

Очищает 20 октетов заголовка, копирует сигнатуру в `magic` (остаток поля остаётся нулевым) и сохраняет `csr` и `seq` в сетевом порядке. Длина полезной нагрузки равна 0. Вызывайте её один раз перед первым `__tlv$put`. Сигнатура NULL или длиннее 8 октетов отвергается точно так же, как её отвергает `__tlv$hget`, и PDU не затрагивается; до версии 1.7.0 длинная сигнатура молча обрезалась до 8 октетов. Процедура не проверяет размер буфера.

Возвращаемые условные значения

Значение	Смысл
`STS$K_SUCCESS`	Заголовок установлен
`STS$K_ERROR`	`a_sig` равен NULL или длиннее 8 октетов

Пример

```
unsigned char pdu [512];

__tlv$hset (pdu, (const unsigned char *) "VCLOUD", CMD_LOGIN, ++seq);
```

__tlv$hget – Получение заголовка PDU

Формат

```
inline static int __tlv$hget (void *a_pdup, const unsigned char *a_sig,
                              unsigned *a_u_len, unsigned *a_u_csr,
                              unsigned *a_u_seq);
```

Возвращает

int, только запись, по значению. Условное значение.

Аргументы

Аргумент	Тип	Доступ	Механизм	Описание
a_pdup	void *	только чтение	по ссылке	Принятый PDU; не менее 20 октетов
a_sig	const unsigned char *	только чтение	по ссылке	Ожидаемая сигнатура, строка не более 8 октетов, завершённая нулём
a_u_len	unsigned *	только запись	по ссылке	Получает длину полезной нагрузки в порядке хоста
a_u_csr	unsigned *	только запись	по ссылке	Получает слово команды/статуса в порядке хоста
a_u_seq	unsigned *	только запись	по ссылке	Получает порядковый номер в порядке хоста

Описание

Сравнивает сигнатуру, дополненную нулями до 8 октетов, с magic. Если они равны, преобразует три поля в порядок хоста и сохраняет их. Если нет, возвращает STS$K_ERROR, и ни одна из трёх выходных переменных не записывается. (Версия 1.6.11 при несовпадении записывала STS$K_ERROR в *a_u_csr.) Все три указателя должны быть действительными: ни один не может быть NULL.

Процедура не проверяет ни того, что принятый буфер содержит 20 октетов, ни того, что len укладывается в принятое. Используйте реальный размер буфера с __tlv$get и __tlv$lookup, которые ограничивают len этим размером.

Возвращаемые условные значения

Значение	Смысл
STS$K_SUCCESS	Сигнатура совпала; три поля сохранены
STS$K_ERROR	Сигнатура равна NULL или длиннее 8 октетов, либо не совпадает

Пример

```
unsigned char rxbuf [512], sig [] = "VCLOUD";   /* rxbuf: a received PDU */
unsigned      len, csr, seq;

if ( !(1 & __tlv$hget (rxbuf, sig, &len, &csr, &seq)) )
    return STS$K_ERROR;             /* not our protocol */
```

__tlv$encode_tag – Упаковка тега

Формат

```
inline static unsigned short __tlv$encode_tag (unsigned short a_v_type,
                                               unsigned short a_v_tag);
```

Возвращает

`unsigned short`, только запись, по значению. Слово тега в СЕТЕВОМ порядке байтов, готовое для записи в `w_tag`.

Аргументы

Аргумент	Тип	Доступ	Механизм	Описание
`a_v_type`	`unsigned short`	только чтение	по значению	Тип, константа `TAG$K_*`. Помещаются только младшие 4 бита
`a_v_tag`	`unsigned short`	только чтение	по значению	Идентификатор, от 0 до 4095

Описание

Возвращает `htobe16 ((a_v_type << 12) | a_v_tag)`. Процедура не маскирует аргументы. Идентификатор больше 4095 изменяет биты типа, а тип больше 15 теряется при сдвиге.

Возвращаемые условные значения

Нет. Возвращаемое значение – слово тега, а не условное значение.

Пример

```
unsigned short w = __tlv$encode_tag (TAG$K_WORD, 1);   /* octets 30 01 in memory */
```

__tlv$decode_tag – Распаковка тега

Формат

```
inline static unsigned short __tlv$decode_tag (unsigned short a_w_tag,
                                               unsigned short *a_v_type,
                                               unsigned short *a_v_tag);
```

Возвращает

`unsigned short`, только запись, по значению. Слово тега в порядке байтов ХОСТА.

Аргументы

Аргумент	Тип	Доступ	Механизм	Описание
`a_w_tag`	`unsigned short`	только чтение	по значению	Слово тега в том виде, как оно хранится в PDU (сетевой порядок)
`a_v_type`	`unsigned short *`	только запись	по ссылке	Получает тип, от 0 до 15
`a_v_tag`	`unsigned short *`	только запись	по ссылке	Получает идентификатор, от 0 до 4095

Описание

Преобразует слово в порядок хоста, затем разделяет его: старшие 4 бита – тип, младшие 12 бит – идентификатор. Оба указателя должны быть действительными.

Возвращаемые условные значения

Нет.

Пример

```
unsigned short type, id;

__tlv$decode_tag (rec->w_tag, &type, &id);
```

__tlv$put – Добавление записи

Формат

```
int __tlv$put (void *a_pdu, unsigned a_pdusz, unsigned a_v_tag,
               unsigned a_v_type, void *a_val, unsigned a_valsz);
```

Возвращает

`int`, только запись, по значению. Условное значение.

Аргументы

Аргумент	Тип	Доступ	Механизм	Описание
`a_pdu`	`void *`	изменение	по ссылке	PDU с заголовком, установленным `__tlv$hset`
`a_pdusz`	`unsigned`	только чтение	по значению	Размер всего буфера, включая заголовок (`unsigned` начиная с версии 1.7.0; раньше был `unsigned short`)
`a_v_tag`	`unsigned`	только чтение	по значению	Идентификатор, от 0 до 4095
`a_v_type`	`unsigned`	только чтение	по значению	Тип, константа `TAG$K_*`, от 0 до 15
`a_val`	`void *`	только чтение	по ссылке	Значение, для числовых типов в порядке XOCTA. Может быть NULL, только если запись пуста
`a_valsz`	`unsigned`	только чтение	по значению	Размер значения в октетах. Для типа фиксированного размера не менее размера типа

Обратите внимание на порядок двух аргументов: сначала идентификатор, затем тип.

Описание

Добавляет одну запись в конец полезной нагрузки (по смещению `len` заголовка), а затем прибавляет размер записи к `len`. Записи нужно `4 + n` октетов, где `n` – реальная длина значения (ниже), а оставшееся место равно `a_pdusz - 20 - len`. Проверка выполняется по реальной длине, поэтому процедура никогда не пишет за пределы буфера, и при ошибке ничего не записывается.

По типам:

- `TAG$K_WORD`, `TAG$K_LONGWORD`, `TAG$K_QWORD`: значение – 16-, 32- или 64-битное число в порядке хоста. Процедура преобразует его в сетевой порядок. Поле длины равно 2, 4 или 8. `a_valsz` должно быть не меньше этого размера; сохраняется только размер типа. `a_valsz`, равное 0, что допускалось до версии 1.7.0, теперь является ошибкой.
- `TAG$K_UUID`: 16 октетов копируются как есть. Поле длины равно 16, а `len` заголовка увеличивается на 20. `a_valsz` должно быть не меньше 16.
- Любой другой тип: `a_valsz` октетов копируются как есть; поле длины равно `a_valsz`, от 0 до 65535.

Идентификатор должен быть от 0 до 4095, а тип – от 0 до 15; иначе ничто не помещается в свои биты, и вызов завершается отказом (раньше идентификатор молча маскировался). Процедура проверяет, что `len` заголовка помещается в буфер. Сигнатуру она не пишет. О каждой ошибке она записывает сообщение в журнал через `$LOG` независимо от параметров.

Возвращаемые условные значения

Значение	Смысл
`STS$K_SUCCESS`	Запись добавлена
`STS$K_ERROR`	`a_pdusz` меньше 20; идентификатор больше 4095 или тип больше 15; `a_valsz` слишком мало для типа фиксированного размера либо больше 65535 для блока; `a_val` равен NULL при непустом значении; `len` заголовка больше области полезной нагрузки; запись не помещается

Пример

```
uint32_t id  = 4711;
char     name[] = "Ruslan";

__tlv$hset (pdu, sig, CMD_LOGIN, ++seq);
__tlv$put  (pdu, sizeof pdu, TAG_ID,   TAG$K_LONGWORD, &id,  sizeof id);
__tlv$put  (pdu, sizeof pdu, TAG_NAME, TAG$K_BBLOCK,   name, strlen (name));
```

__tlv$get – Получение значения

Формат

```
int __tlv$get (void *a_pdu, unsigned a_pdusz, int *a_context, unsigned a_v_tag,
               unsigned *a_v_type, void *a_val, unsigned *a_valsz);
```

Возвращает

int, только запись, по значению. Условное значение.

Аргументы

Аргумент	Тип	Доступ	Механизм	Описание
a_pdu	void *	только чтение	по ссылке	PDU
a_pdusz	unsigned	только чтение	по значению	Реальный размер буфера в октетах: число принятых октетов
a_context	int *	изменение	по ссылке	Позиция поиска; -1 для начала. Может быть NULL
a_v_tag	unsigned	только чтение	по значению	Искомый идентификатор
a_v_type	unsigned *	только запись	по ссылке	Получает тип записи. Не должен быть NULL
a_val	void *	только запись	по ссылке	Буфер, получающий значение
a_valsz	unsigned *	изменение	по ссылке	На входе – размер a_val; на выходе – число сохранённых октетов. Может быть NULL только для числовых типов

Описание

Просматривает записи с позиции в *a_context (или с начала) в поисках первой, чей идентификатор равен a_v_tag; тип не сравнивается. Предел поиска – меньшее из длины полезной нагрузки в заголовке и числа реально присутствующих октетов (a_pdusz - 20), поэтому ложный заголовок не может заставить процедуру читать за пределами буфера. Процедура останавливается на первой записи, не укладывающейся в предел, и сообщает о ней как о STS$K_ERROR: усечённая запись – это ошибка, а не «отсутствует» (до версии 1.7.0 давался STS$K_WARN).

Контекст – смещение в октетах от начала полезной нагрузки. Значение -1 (или вызов с NULL) означает начало с первой записи. После каждого вызова *a_context устанавливается на смещение сразу за последней просмотренной записью, так что следующий вызов с тем же контекстом находит следующую запись с тем же идентификатором. Контекст NULL всегда начинает с начала.

Когда запись найдена, значение копируется в `a_val`:

- `TAG$K_WORD`, `TAG$K_LONGWORD`, `TAG$K_QWORD`: преобразуется в порядок хоста; сохраняемый размер равен 2, 4 или 8.
- `TAG$K_UUID`: ровно 16 октетов.
- Любой другой тип: копируется как есть; сохраняемый размер равен длине записи.

`*a_valsz` устанавливается в число сохранённых октетов. Для всех типов, кроме числовых, `a_valsz` обязателен и должен содержать размер получателя; значение длиннее него не копируется, и процедура возвращает `STS$K_ERROR`. Для числовых типов NULL в `a_valsz` означает, что буфер достаточен для типа.

Поле длины числовой записи должно быть равно 2, 4 или 8, а записи UUID – 16. Любая другая длина означает искажённую запись, и процедура возвращает `STS$K_ERROR`, ничего не копируя. До версии 1.7.0 такая запись читалась за пределами своего значения.

Возвращаемые условные значения

Значение	Смысл
`STS$K_SUCCESS`	Запись найдена, её значение скопировано
`STS$K_WARN`	В остатке PDU нет записи с этим идентификатором
`STS$K_ERROR`	`a_pdusz` меньше 20; запись выходит за пределы PDU; неверна длина записи фиксированного размера; `*a_valsz` меньше записи; `a_valsz` равен NULL для типа, которому он нужен; `a_val` или `a_v_type` равен NULL

Пример

```
int      ctx = -1;
unsigned type, sz;
char     item [64];

for ( ;; )
{
    sz = sizeof item;
    if ( !(1 & __tlv$get (rx, rxlen, &ctx, TAG_ITEM, &type, item, &sz)) )
        break;
    printf ("item: %.*s\n", (int) sz, item);
}
```

__tlv$lookup – Поиск записи

Формат

```
int __tlv$lookup (void *a_pdu, unsigned a_pdusz, int *a_context, unsigned a_v_tag,
                  unsigned *a_v_type, void **a_val, unsigned *a_valsz);
```

Возвращает

int, только запись, по значению. Условное значение.

Аргументы

Аргумент	Тип	Доступ	Механизм	Описание
a_pdu	void *	только чтение	по ссылке	PDU
a_pdusz	unsigned	только чтение	по значению	Реальный размер буфера в октетах
a_context	int *	изменение	по ссылке	Позиция поиска; -1 для начала. Может быть NULL
a_v_tag	unsigned	только чтение	по значению	Искомый идентификатор
a_v_type	unsigned *	только запись	по ссылке	Получает тип. Не должен быть NULL
a_val	void **	только запись	по ссылке	Получает адрес значения внутри PDU. Не должен быть NULL
a_valsz	unsigned *	только запись	по ссылке	Получает длину значения. Может быть NULL

Описание

Поиск такой же, как в __tlv$get, но ничего не копируется и не преобразуется: возвращаемый адрес указывает внутрь PDU, а значение находится в сетевом порядке. Адрес действителен, пока действителен буфер PDU. Он может быть невыровненным, поэтому читайте из него число через memcpy с последующим be16toh, be32toh или be64toh. Эта процедура – правильный выбор для большого блока. Длина записи фиксированного размера проверяется так же, как в __tlv$get (слово 2, длинное слово 4, квадрослово 8, UUID 16): запись другой длины – STS$K_ERROR, и адрес не выдаётся. (До 1.7.0 возвращался адрес короткой записи, и вызывающий должен был сам проверять *a_valsz.)

Возвращаемые условные значения

Значение	Смысл
STS$K_SUCCESS	Запись найдена
STS$K_WARN	В остатке PDU нет такой записи

Значение	Смысл
`STS$K_ERROR`	`a_pdusz` меньше 20; запись выходит за пределы PDU; неверна длина записи фиксированного размера

Пример

```
void    *p;
unsigned type, sz;
int      ctx = -1;

if ( 1 & __tlv$lookup (rx, rxlen, &ctx, TAG_BLOB, &type, &p, &sz) )
    process_blob (p, sz);
```

__tlv$dump – Вывод PDU

Формат

```
void __tlv$dump (void *a_pdu, unsigned a_pdusz);
```

Возвращает

Ничего.

Аргументы

Аргумент	Тип	Доступ	Механизм	Описание
a_pdu	void *	только чтение	по ссылке	PDU
a_pdusz	unsigned	только чтение	по значению	Реальный размер буфера в октетах

Описание

Записывает в журнал через `$LOG (STS$K_INFO, ...)` одну строку для заголовка (`PDU [len=..., csr=..., seq=...]`) и по одной для каждой записи. Строка записи показывает индекс, слово тега, идентификатор и тип, длину, а также заголовок записи и значение в шестнадцатеричном виде. Значение длиннее 64 октетов выводится кусками по 64 октета, строки продолжения начинаются с `+offset`, так что длинное значение не может переполнить буфер (до версии 1.7.0 значение в 256 октетов и более записывало один байт за его пределы). Запись, не укладывающаяся в предел, сообщается как усечённая и завершает вывод. Предназначена для отладки.

Возвращаемые условные значения

Нет.

Пример

```
__tlv$dump (pdu, sizeof (AVPROTO_HDR) + len);
```

5.2 Изменения в версии 1.7.0

В редакции 1.6.11 настоящего руководства перечислялись дефекты этого модуля. Они исправлены. Что может заметить потребитель:

1. `__tlv$put` проверяет свободное место по реальной длине записи и никогда не пишет за пределы буфера. Типу фиксированного размера нужно `a_valsz` не менее его размера; 0 отвергается. Запись UUID прибавляет к `len` 20 октетов, так что заголовок и запись согласованы.
2. `__tlv$put` отвергает идентификатор больше 4095 и тип больше 15, а не маскирует их. Размер её буфера – `unsigned`.
3. `__tlv$get` проверяет длину числовой записи или записи UUID в потоке и сообщает о неверной, а также о записи, выходящей за пределы PDU, как о `STS$K_ERROR`. `__tlv$lookup` так же сообщает о записи, выходящей за пределы PDU.
4. `__tlv$dump` выводит длинное значение кусками по 64 октета.
5. `__tlv$hset` отвергает сигнатуру длиннее 8 октетов (или NULL); `__tlv$hget` при ошибке не трогает свои выходные переменные.
6. `avproto.h` больше не отключает предупреждения компилятора (`-Wparentheses`, `-Wdate-time`, `-Wunused-variable`) для каждого включающего его файла.

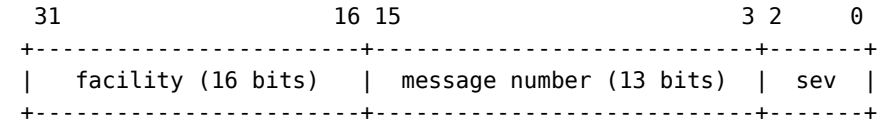
В 1.7.1 `__tlv$lookup` проверяет длину записи фиксированного размера, как `__tlv$get`.

Что по-прежнему верно: `__tlv$lookup` возвращает адрес без преобразования, а `__tlv$hset` и `__tlv$hget` не проверяют размер переданного им буфера.

Приложение A. Условные значения и серьёзность

Процедуры STARLET возвращают 32-битное условное значение в `int` или `unsigned`. Соглашение то же, что в OpenVMS: установленный бит 0 означает успех, сброшенный бит 0 – неуспех. Проверка – `1 & status`. Константы серьёзности и макросы определены в `utility_routines.h`. Начиная с версии 1.7.0 заголовок `starlet.h` сам включает `utility_routines.h` и больше не повторяет константы серьёзности. Заголовок `starlet.h` определяет значения `FAO$K_*`, а `utility_routines.h` – значения `UTIL$S_*` процедур очередей и `__util$inimsg`; заголовки `cli_routines.h` и `avproto.h` не определяют условных значений и используют значения из этого приложения.

A.1 Раскладка условного значения



Поле	Биты	Ширина	Описание
Серьёзность	0-2	3	Одно из значений `STS$K_*` ниже
Номер сообщения	3-15	13	Номер сообщения в пределах его средства
Средство	16-31	16	Номер средства. Средство по умолчанию служебного модуля, `__FAC$_UTILS__`, равно 0x01

Управляющих битов нет: поле средства занимает 16 старших бит. Таблица сообщений, зарегистрированная через `__util$inimsg`, находится по полям средства и сообщения; см. главу 2.

Поскольку значения серьёзности 1, 3, 5 и 7 нечётны, все они читаются как успех при проверке `1 & status`. На практике как успех используются только `STS$K_SUCCESS` и `STS$K_INFO`.

A.2 Значения серьёзности

Константа	Значение	Буква (STS$C__*)	Успех?	Смысл
STS$K_WARN	0	W	нет	Предупреждение; в нескольких процедурах также «не найдено» и «отсутствует»
STS$K_SUCCESS	1	S	да	Успех
STS$K_ERROR	2	E	нет	Ошибка
STS$K_INFO	3	I	да	Информация
STS$K_FATAL	4	F	нет	Тяжёлая ошибка
STSK_UNDEF, STSK_UNDEF5	5	? (STS$C__UNDEF)	да	Не определено; __util$log и подобные процедуры показывают неизвестную серьёзность как ? и возвращают для неё STS$K_ERROR (до 1.7.0 – это значение)
STS$K_UNDEF6	6	нет	нет	Не определено
STS$K_UNDEF7	7	нет	да	Не определено
STS$K_MAX	8	нет	нет	Маркер конца списка

STS$M_SYSLOG (16) не является частью условного значения. Это флаг для аргумента серьёзности процедур журналирования: он требует отправить сообщение также службе syslog.

A.3 Макросы

Все эти макросы находятся в `utility_routines.h`. Аргумент – условное значение или число, как указано.

Макрос	Результат
`$SEV(code)`	`(code) & 0x7`: серьёзность
`$MSG(code)`	`((code) >> 3) & 0x1FFF`: номер сообщения
`$FAC(code)`	`(code) >> 16`: средство. Не маскируется, поэтому аргумент должен быть 32-битным беззнаковым
`$DEFSEV(n)`	`(n) & 0x7`: серьёзность, для построения значения
`$DEFMSG(n)`	`((n) << 3) & 0xFFF8`: номер сообщения, помещённый в биты 3-15
`$DEFFAC(n)`	`(n) << 16`: средство, помещённое в биты 16-31
`$STS_ERROR(code)`	Истинно, если серьёзность равна `STS$K_ERROR` или `STS$K_FATAL`

Значение строится сложением трёх частей:

```
#define MY$_NOFILE   ( $DEFFAC (0x20) | $DEFMSG (3) | $DEFSEV (STS$K_ERROR) )

$SEV (MY$_NOFILE) == STS$K_ERROR;   /* 2    */
$MSG (MY$_NOFILE) == 3;
$FAC (MY$_NOFILE) == 0x20;
```

A.4 Значения библиотечных модулей

Коды завершения процедур FAO и времени (`starlet.h`) и коды `UTIL$S_*` процедур очередей (`utility_routines.h`) следуют раскладке раздела A.1 без средства: серьёзность – в трёх младших битах, номер сообщения – выше. Все они чётные, поэтому `1 & status` для них ложна. Они выбраны именно такими (в истории `starlet.c` записан переход от нечётных значений). Их номера стабильны начиная с версии 1.7.0, с одним исключением: `FAO$K_BADDIR` в версии 1.6.11 был равен 6, а `UTIL$S_NOLOCK` – 0x202.

Константа	Значение	`$SEV`	`$MSG`	Где используется
`FAO$K_NEGTIM`	0x08	0 (warn)	1	`__util$sub_times`: отрицательный результат
`FAO$K_BUFOVF`	0x0A	2 (error)	1	`__util$fao`, `__util$faol`, `__util$asctim`: усечённый результат
`FAO$K_IVTIME`	0x0C	4 (fatal)	1	`__util$bintim`, `__util$add_times`, `__util$sub_times`: неверное время
`FAO$K_BADDIR`	0x12	2 (error)	2	`__util$fao`, `__util$faol`: неизвестная, искажённая или оборванная директива
`FAO$K_NOPRM`	0x1A	2 (error)	3	`__util$fao`, `__util$faol`: слишком мало параметров
`FAO$K_FLDOVF`	0x22	2 (error)	4	`__util$fao`, `__util$faol`: поле `!n<...!>` обрезано
`UTIL$S_INVARG`	0x102	2 (error)	0x20	Процедуры очередей и `__util$inimsg`: аргумент NULL или непригодный
`UTIL$S_INQUE`	0x202	2 (error)	0x40	`__util$insqhead`, `__util$insqtail`: элемент находится в другой очереди
`UTIL$S_NOLOCK`	0x302	2 (error)	0x60	Процедуры очередей: спин-блокировка не захвачена

Константа	Значение	$SEV	$MSG	Где используется
UTIL$S_DUPFAC	0x400	0 (warn)	0x80	__util$inimsg: каталог сообщений этого средства уже зарегистрирован

Статус процедур FAO – первая ошибка, найденная в вызове (раздел 3.1). Процедуры очередей возвращают STS$K_SUCCESS также для пустой очереди в __util$remqhead и __util$remqtail; вызывающий узнаёт о пустой очереди по тому, что *ent равен NULL (глава 2).

Файл конфигурации, читаемый __util$readconfig или названный параметром OPTS$K_CONF процедуры __util$getparams, может содержать параметр, которого нет в таблице (файл может обслуживать несколько программ). Такой параметр пропускается и заносится в журнал, а процедура возвращает STS$K_INFO; его младший бит установлен, поэтому проверка 1 & status по-прежнему говорит об успехе. Неверное значение, строка, не начинающаяся с - или /, и неизвестный параметр в командной строке дают STS$K_ERROR (глава 2).

Процедуры CLI и модуля TLV возвращают только STS$K_SUCCESS, STS$K_WARN, STS$K_ERROR и STS$K_FATAL.

Модуль	STS$K_SUCCESS	STS$K_WARN	STS$K_ERROR	STS$K_FATAL
__cli$parse	верно	напечатана справка	значение не прошло проверку типа или неизвестное ключевое слово	синтаксис, поиск, неоднозначность, лишнее слово или отсутствующий элемент
__cli$get_value	значение возвращено	пустое значение	отсутствует или исчерпано (в журнал не пишется)	аргумент NULL
__cli$get_keyword	запись найдена	пустое значение	не ключевое слово, отсутствует, нет списка	аргумент NULL
__cli$present	присутствует	отсутствует	не используется	аргумент NULL
__tlv$hset	заголовок установлен	не используется	неверная сигнатура	не используется
__tlv$hget	сигнатура совпала	не используется	нет совпадения или неверная сигнатура	не используется
__tlv$put	добавлено	не используется	не помещается или неверный аргумент	не используется
__tlv$get	скопировано	не найдено	неверная область, буфер или запись	не используется

Модуль	STS$K_SUCCESS	STS$K_WARN	STS$K_ERROR	STS$K_FATAL
__tlv$lookup	найдено	не найдено	неверная область или запись	не используется

Приложение B. Указатель процедур и макросов

В таблице перечислены процедуры, макросы, типы и константы всех четырёх модулей в алфавитном порядке (при упорядочении ведущие подчёркивания и `$` игнорируются). «Гл.» – глава, описывающая элемент. Четыре помощника времени `__util$add_time`, `__util$sub_time`, `__util$cmp_time` и `__util$timbuf` принадлежат модулю `utility_routines`, но описаны в главе 3.

Имя	Назначение	Модуль	Гл.
`$ARRSZ`	Число элементов массива	utility_routines	2
`ASC`	Счётная строка (байт длины и 255 символов)	utility_routines	2
`$ASC`	Длина и адрес ASC для преобразования %.*s	utility_routines	2
`ASC$K_SZ`	Наибольшая счётная строка (255)	utility_routines	2
`$ASCDECL`	Объявление и инициализация переменной ASC	utility_routines	2
`$ASCINI`	Поля-инициализаторы ASC	utility_routines	2
`$ASCLEN`	Длина ASC	utility_routines	2
`$ASCNIL`	Инициализатор пустой ASC	utility_routines	2
`$ASCPTR`	Адрес текста ASC	utility_routines	2
`$ATOMIC_DEC`	Атомарное уменьшение (выборка и вычитание 1)	utility_routines	2
`$ATOMIC_INC`	Атомарное увеличение (выборка и прибавление 1)	utility_routines	2
`AVPROTO_HDR`	Структура: заголовок PDU (20 октетов)	avproto	5

Имя	Назначение	Модуль	Гл.
AVPROTO_PDU	Структура: заголовок, за которым следуют записи	avproto	5
AVPROTO_TLV	Структура: заголовок записи TLV	avproto	5
$BIN2HEX	Макро-форма __util$bin2hex	utility_routines	2
__cli$cleanup	Освобождение контекста разбора	cli_routines	4
__cli$dispatch	Вызов процедуры действия найденного глагола	cli_routines	4
__cli$get_keyword	Приведение значения ключевого слова к его записи в таблице	cli_routines	4
__cli$get_value	Получение значения параметра или квалификатора	cli_routines	4
__cli$help	Вывод экрана справки по таблицам	cli_routines	4
CLI$K_DATE	Тип значения: дата	cli_routines	4
CLI$K_DEVICE	Тип значения: устройство в /dev	cli_routines	4
CLI$K_FILE	Тип значения: имя файла	cli_routines	4
CLI$K_IPV4	Тип значения: адрес IPv4	cli_routines	4
CLI$K_IPV6	Тип значения: адрес IPv6	cli_routines	4
CLI$K_KWD	Тип значения: ключевое слово	cli_routines	4
CLI$K_MAC	Тип значения: MAC-адрес	cli_routines	4
CLI$K_NUM	Тип значения: число (десятичное, восьмеричное, шестнадцатеричное)	cli_routines	4
CLI$K_OPT	Тип значения: флаг без значения	cli_routines	4
CLI$K_P1 ... CLI$K_P8	Позиции параметров (1-8)	cli_routines	4
CLI$K_QSTRING	Тип значения: строка в кавычках	cli_routines	4
CLI$K_QUAL	Вид элемента: квалификатор (9)	cli_routines	4
CLI$K_TIME	Тип значения: абсолютное время или дельта-время	cli_routines	4
CLI$K_UUID	Тип значения: UUID	cli_routines	4

Имя	Назначение	Модуль	Гл.
CLI$M_CONSUMED	Состояние: значение списка уже возвращено	cli_routines	4
CLI$M_LIST	Флаг: квалификатор принимает список	cli_routines	4
CLI$M_NEGATABLE	Флаг: у квалификатора есть форма /NO	cli_routines	4
CLI$M_NEGATED	Состояние: элемент задан как /NO	cli_routines	4
CLI$M_OPSIGNAL	Параметр: сообщать об ошибках в журнал	cli_routines	4
CLI$M_OPTRACE	Параметр: трассировать разбор	cli_routines	4
CLI$M_PRESENT	Состояние: элемент задан	cli_routines	4
CLI$M_REQUIRED	Флаг: элемент должен быть задан	cli_routines	4
__cli$parse	Разбор вектора аргументов по таблицам команд	cli_routines	4
__cli$present	Проверка, был ли задан элемент и в какой форме	cli_routines	4
__cli$show_ctx	Вывод контекста в журнал	cli_routines	4
__cli$show_verbs	Вывод таблицы глаголов в журнал	cli_routines	4
CLI$SSZ_MAXVERB	Длина сравниваемой части глагола (32)	cli_routines	4
CLI_CTX	Структура: контекст разбора	cli_routines	4
CLI_ITEM	Структура: разобранный элемент	cli_routines	4
CLI_KEYWORD	Структура: ключевое слово и его значение	cli_routines	4
CLI_PQDESC	Структура: параметр или квалификатор	cli_routines	4
CLI_VERB	Структура: глагол	cli_routines	4
$CLRQUE	Макро-форма __util$clrqueue	utility_routines	2
$CMP_STORE_LONG	Атомарное сравнение с записью длинного слова	utility_routines	2
$DEFSEV, $DEFMSG, $DEFFAC	Построение полей условного значения	utility_routines	2

Имя	Назначение	Модуль	Гл.
`$DUMPHEX`	Макро-форма __util$dumphex	utility_routines	2
`EMSG_RECORD, EMSG_RECORD_DESC`	Структуры таблицы сообщений	utility_routines	2
`ENTRY, __QUEUE`	Структуры очереди и её элементов	utility_routines	2
`FAO$GETTIME`	Источник текущего времени, задаваемый при компиляции	starlet	3
`FAO$K_BADDIR`	Условное значение: неверная, неизвестная или оборванная директива (0x12)	starlet	3
`FAO$K_BUFOVF`	Условное значение: вывод усечён	starlet	3
`FAO$K_FLDBUF`	Ограничение: наибольшая длина текста поля !n<...!> (512)	starlet	3
`FAO$K_FLDOVF`	Условное значение: поле !n<...!> обрезано	starlet	3
`FAO$K_IVTIME`	Условное значение: неверное время	starlet	3
`FAO$K_MAXDEPTH`	Ограничение: глубина вложенности групп FAO (4)	starlet	3
`FAO$K_MAXPRM`	Ограничение: число параметров, кэшируемых __util$fao (64)	starlet	3
`FAO$K_NEGTIM`	Условное значение: отрицательный результат времени	starlet	3
`FAO$K_NOPRM`	Условное значение: слишком мало параметров	starlet	3
`FAO$K_TZOFFSET`	Смещение часового пояса в секундах, задаваемое при компиляции	starlet	3
`FAO$T_TIME`	Тип, на котором основан fao_time_t, задаваемый при компиляции	starlet	3
`$FAO_DESC`	Инициализатор выходного дескриптора над символьным массивом	starlet	3
`fao_desc_t`	Дескриптор выходного буфера	starlet	3

Имя	Назначение	Модуль	Гл.
`fao_prm_t`	Тип параметра FAO (64 бита, по значению)	starlet	3
`fao_time_t`	Тип двоичного времени (секунды с 01-JAN-1970; отрицательное = дельта-время)	starlet	3
`$IFLOG`	Макрос: запись в журнал, если условие истинно	utility_routines	2
`$IFTRACE`	Макрос: запись трассы, если условие истинно	utility_routines	2
`ILE2, ILE3, $ILE2INI, $ILE3INI, $ILENULL`	Структуры списков элементов; `$ILE3INI` – макрос-оператор	utility_routines	2
`$INSQHEAD`	Макро-форма __util$insqhead	utility_routines	2
`$INSQTAIL`	Макро-форма __util$insqtail	utility_routines	2
`$ISINRANGE`	Макро-форма __util$isinrange	utility_routines	2
`$ISINRANGE2`	Макро-форма __util$isinrange2	utility_routines	2
`KWDENT`	Структура: запись таблицы ключевых слов	utility_routines	2
`$LOCK_LONG`	Макро-форма __util$lockspin	utility_routines	2
`$LOG`	Макрос: запись в журнал в месте вызова; возвращает серьёзность	utility_routines	2
`$LOG_FAO`	Макрос: запись в журнал с управляющей строкой FAO	utility_routines	2
`$MAX`	Макро-форма __util$max	utility_routines	2
`$MIN`	Макро-форма __util$min	utility_routines	2
`$MOVQHEAD`	Макро-форма __util$movqhead	utility_routines	2
`$MOVQTAIL`	Макро-форма __util$movqtail	utility_routines	2
`OPTS$K_INT ... OPTS$K_PWD`	Типы значения параметра	utility_routines	2
`OPTS, OPTS_NULL`	Структура записи таблицы параметров	utility_routines	2
`$PUTMSG`	Макрос: запись сообщения условного значения	utility_routines	2

Имя	Назначение	Модуль	Гл.
`$PUTMSG_FAO`	Макрос: запись сообщения условного значения с аргументами FAO	utility_routines	2
`$RANGE`	Макро-форма __util$range	utility_routines	2
`$REMQENT`	Макро-форма __util$remqent	utility_routines	2
`$REMQHEAD`	Макро-форма __util$remqhead	utility_routines	2
`$REMQTAIL`	Макро-форма __util$remqtail	utility_routines	2
`s___time`	Внутреннее: текущее время как timespec	utility_routines	2
`$SEV, $MSG, $FAC`	Извлечение серьёзности, номера сообщения, средства (приложение A)	utility_routines	2
`STS$C__WARN ...` `STS$C__UNDEF`	Однобуквенные коды серьёзности	utility_routines	2
`STS$K_WARN ...` `STS$K_FATAL, STS$K_UNDEF, STS$K_MAX`	Значения серьёзности (приложение A)	utility_routines	2
`STS$M_SYSLOG`	Флаг аргумента серьёзности: отправить также в syslog	utility_routines	2
`$STS_ERROR`	Истинно для условного значения ошибки или тяжёлой ошибки	utility_routines	2
`SYM_REC, $SYM_REC_INI, $SYM_REC_EOL`	Запись таблицы символов для mask2sym и val2sym	utility_routines	2
`TAG$K_BBLOCK`	Тип тега: блок октетов (0)	avproto	5
`TAG$K_LONGWORD`	Тип тега: 32-битное число (4)	avproto	5
`TAG$K_MAX`	Маркер конца списка типов тегов	avproto	5
`TAG$K_QWORD`	Тип тега: 64-битное число (5)	avproto	5
`TAG$K_UUID`	Тип тега: UUID (8)	avproto	5
`TAG$K_WORD`	Тип тега: 16-битное число (3)	avproto	5
`TAG$M_ID`	Маска битов идентификатора слова тега	avproto	5
`TAG$M_TYPE`	Маска битов типа слова тега (0xF000)	avproto	5

Имя	Назначение	Модуль	Гл.
`__tlv$decode_tag`	Распаковка слова тега	avproto	5
`__tlv$dump`	Вывод PDU в журнал	avproto	5
`__tlv$encode_tag`	Упаковка типа и идентификатора тега	avproto	5
`__tlv$get`	Поиск записи и копирование её значения	avproto	5
`__tlv$hget`	Проверка сигнатуры и чтение заголовка PDU	avproto	5
`__tlv$hset`	Инициализация заголовка PDU	avproto	5
`__tlv$lookup`	Поиск записи и возврат адреса её значения	avproto	5
`__tlv$put`	Добавление записи TLV в PDU	avproto	5
`$TRACE`	Макрос: запись трассы	utility_routines	2
`$UNLOCK_LONG`	Макро-форма __util$unlockspin	utility_routines	2
`__util$add_time`	Сложение двух значений timespec	utility_routines	3
`__util$add_times`	Сложение двух двоичных времён	starlet	3
`__util$asc2str`	Копирование счётной строки в строку, завершённую нулём	utility_routines	2
`__util$asctim`	Преобразование двоичного времени в строку ASCII	starlet	3
`__util$base64`	Кодирование октетов в Base64 (RFC 4648)	utility_routines	2
`__util$bin2hex`	Преобразование октетов в шестнадцатеричный текст	utility_routines	2
`__util$bintim`	Преобразование строки ASCII со временем в двоичное время	starlet	3
`__util$bzero`	Очистка блока памяти	utility_routines	2
`__util$clrqueue`	Опустошение очереди	utility_routines	2
`__util$cmp_time`	Сравнение двух значений timespec (-1, 0, 1)	utility_routines	3
`__util$cmpasc`	Сравнение двух ASC	utility_routines	2

Имя	Назначение	Модуль	Гл.
`__util$cmpasc_blind`	Сравнение двух ASC без учёта регистра	utility_routines	2
`__util$collapse`	Удаление из строки всех пробелов и табуляций	utility_routines	2
`__util$cpyasc`	Копирование ASC в ASC	utility_routines	2
`__util$crc32c`	CRC-32 буфера (полином IEEE, несмотря на имя)	utility_routines	2
`__util$deflog`	Выбор файла журнала или хоста syslog	utility_routines	2
`__util$dumphex`	Запись шестнадцатеричного дампа буфера в журнал	utility_routines	2
`__util$fao`	Форматирование управляющей строки по списку параметров переменной длины	starlet	3
`__util$faohex`	Форматирование до 16 октетов в одну строку из шестнадцатеричного и ASCII-текста	utility_routines	2
`__util$faol`	Форматирование управляющей строки по массиву параметров	starlet	3
`__util$fill`	Заполнение буфера повторением образца	utility_routines	2
`__util$getmsg`	Поиск записи сообщения для условного значения	utility_routines	2
`__util$getparams`	Заполнение таблицы параметров из командной строки	utility_routines	2
`__util$gettim`	Получение текущего двоичного времени	starlet	3
`__util$hex2bin`	Преобразование шестнадцатеричного текста в октеты	utility_routines	2
`__util$inimsg`	Регистрация таблицы сообщений	utility_routines	2
`__util$insqhead`	Вставка элемента в голову очереди	utility_routines	2
`__util$insqtail`	Вставка элемента в хвост очереди	utility_routines	2

Имя	Назначение	Модуль	Гл.
`__util$isinrange`	Проверка, что значение лежит в диапазоне, границы включены	utility_routines	2
`__util$isinrange2`	Проверка, что значение лежит в диапазоне, границы включены, в любом порядке	utility_routines	2
`__util$iszero`	Проверка, что все октеты буфера нулевые	utility_routines	2
`UTIL$K_LOOKUP_ABBREV`	Параметр __util$lookup_key: разрешить сокращения	utility_routines	2
`UTIL$K_LOOKUP_NCASE`	Параметр __util$lookup_key: игнорировать регистр	utility_routines	2
`__util$lockspin`	Захват спин-блокировки	utility_routines	2
`__util$log`	Запись в журнал (формат printf)	utility_routines	2
`__util$log2buf`	Форматирование записи журнала в буфер вызывающего	utility_routines	2
`__util$log_fao`	Запись в журнал (управляющая строка FAO)	utility_routines	2
`__util$logd`	Как __util$log, с модулем, функцией и строкой	utility_routines	2
`__util$logd_fao`	Как __util$log_fao, с модулем, функцией и строкой	utility_routines	2
`__util$lookup_key`	Перевод ключевого слова в значение по таблице	utility_routines	2
`__util$mask2sym`	Перевод битовой маски в текст	utility_routines	2
`__util$max`	Большее из двух целых	utility_routines	2
`__util$min`	Меньшее из двух целых	utility_routines	2
`__util$movc5`	Перемещение символов с заполнением (LIB$MOVC5)	utility_routines	2
`__util$movqhead`	Перемещение элемента (`void *`) в голову его очереди	utility_routines	2
`__util$movqtail`	Перемещение элемента (`void *`) в хвост его очереди	utility_routines	2

Имя	Назначение	Модуль	Гл.
`__util$nope`	Пустая функция, используемая, когда трассировка исключена при компиляции	utility_routines	2
`__util$out`	Запись строки без префикса в приёмник журнала	utility_routines	2
`__util$pattern_match`	Сопоставление строки с образцом-шаблоном	utility_routines	2
`__util$putmsg`	Запись сообщения условного значения (аргументы printf)	utility_routines	2
`__util$putmsg_fao`	Запись сообщения условного значения (аргументы FAO)	utility_routines	2
`__util$putmsgd`	Как __util$putmsg, с модулем, функцией и строкой	utility_routines	2
`__util$putmsgd_fao`	Как __util$putmsg_fao, с модулем, функцией и строкой	utility_routines	2
`__util$q_unlink`	Внутреннее: изъятие элемента из цепочки его очереди	utility_routines	2
`__util$range`	Ограничение значения диапазоном	utility_routines	2
`__util$readconfig`	Заполнение таблицы параметров из файла конфигурации	utility_routines	2
`__util$remqent`	Удаление заданного элемента из очереди	utility_routines	2
`__util$remqhead`	Удаление головного элемента очереди	utility_routines	2
`__util$remqtail`	Удаление хвостового элемента очереди	utility_routines	2
`__util$rewindlogfile`	Усечение файла журнала при превышении размера	utility_routines	2
`__util$rotatelogfile`	Ротация файла журнала при превышении размера	utility_routines	2

Имя	Назначение	Модуль	Гл.
`UTIL$S_DUPFAC`	Условное значение: каталог сообщений средства уже зарегистрирован (0x400)	utility_routines	2
`__util$setlogfd`	Направление журнала в открытый файловый дескриптор	utility_routines	2
`__util$showparams`	Показ таблицы параметров	utility_routines	2
`UTIL$S_INQUE`	Условное значение: элемент в другой очереди (0x202)	utility_routines	2
`UTIL$S_INVARG`	Условное значение: непригодный аргумент (0x102)	utility_routines	2
`UTIL$S_NOLOCK`	Условное значение: спин-блокировка не захвачена (0x302)	utility_routines	2
`__util$str2asc`	Копирование строки, завершённой нулём, в ASC	utility_routines	2
`__util$strcat_rx`	Склейка списка строк в одну область	utility_routines	2
`__util$strelem`	Получение элемента строки, ограниченного разделителями	utility_routines	2
`__util$strstr`	Поиск подстроки в счётном буфере	utility_routines	2
`__util$strxor`	Исключающее ИЛИ строки с ключом	utility_routines	2
`__util$sub_time`	Вычитание двух значений timespec (нормализованное)	utility_routines	3
`__util$sub_times`	Вычитание двух двоичных времён	starlet	3
`__util$syslog`	Отправка сообщения серверу syslog (тихий успех, если хоста нет)	utility_routines	2
`__util$timbuf`	Разбор time_t в локальный struct tm с естественными номерами года и месяца	utility_routines	3
`__util$trace`	Запись трассы, если условие истинно	utility_routines	2

Имя	Назначение	Модуль	Гл.
`__util$trim`	Удаление ведущих и замыкающих пробелов	utility_routines	2
`__util$uint32_2_dec`	Преобразование беззнакового 32-битного числа в десятичный текст	utility_routines	2
`__util$unbase64`	Декодирование Base64	utility_routines	2
`__util$uncomment`	Удаление комментария из строки, части в кавычках пропускаются	utility_routines	2
`__util$unlockspin`	Освобождение спин-блокировки	utility_routines	2
`__util$uuid2str`	Преобразование UUID в текст	utility_routines	2
`__util$val2sym`	Перевод значения в текст	utility_routines	2